

From Vector to Machine Code



Algorithms for generating code for laser, cutting and plotting machines

v1.0.0 · EN · 2026-07-02

Contents

0 The core idea: the Chain model

1 Architecture & Pipeline

2 Geometry Foundations

3 Fill Algorithms & Automatic Selection

4 Outline & Contour

5 Single-Stroke (Centerline) Text

6 Pattern, Texture & Shade Fill

7 Toolpath Ordering

8 Machine-Code Emission

9 Raster Engraving

10 Curve Fidelity & Continuity

11 Performance & Complexity

12 Inputs, Units & Edge Cases

The core idea: the Chain model

Before we discuss curves, fills, or text, we need one simple truth as a foundation: everything a point-to-point machine can do reduces to a single data structure - a **chain of points**. Grasp this, and the whole system becomes clear.

1. Every machine speaks the same "abstract language"

A pen plotter, a laser, a vinyl/knife cutter, a CNC router - however different their physics - all accept the same basic instruction set:

Move while the tool is DISENGAGED

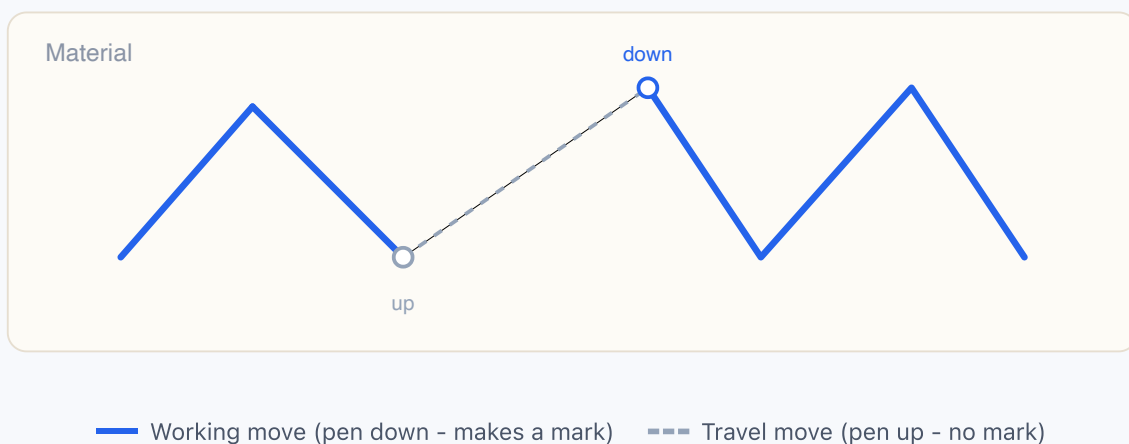
Pen up / laser off / blade up. This is a **travel** (**rapid**) move: it produces nothing on the material.

Move while the tool is ENGAGED

Pen down / laser on / blade down. This is a **working** move: it produces a mark, a cut, or a burn.

Modulate the tool (optional)

During a working move, the intensity can vary: laser power, spindle RPM, blade force.



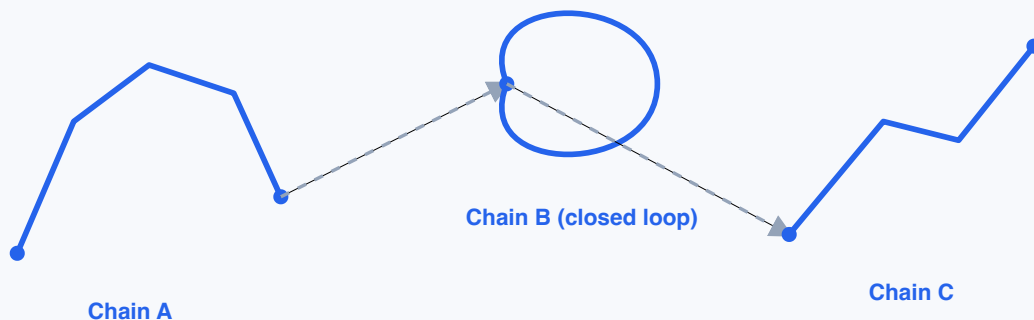
All machine output, however complex, is just an interleaving of these two kinds of motion.

2. Defining the "Chain"

Because there are only two kinds of motion, every drawing reduces to a single data structure:

☆ Chain

A **chain** = an *ordered* list of 2-D points (a polyline) that the tool traces in *one continuous engaged pass*. Between chains, the tool disengages and travels.



Three chains joined by **travel moves** (dashed). Curves, fills, text and images are simply *different sources of chains*.

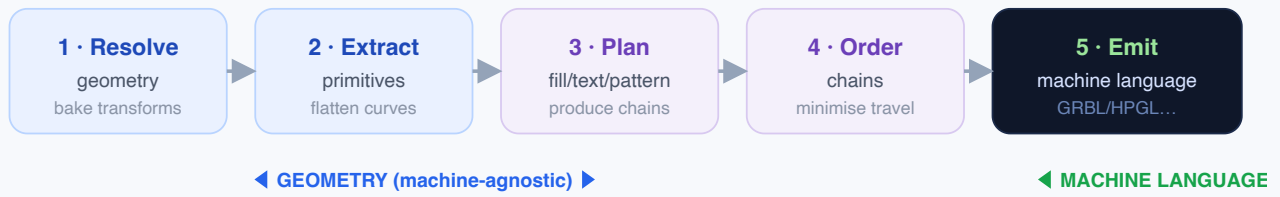
The entire job fits in one sentence: *produce a set of chains that reproduce the artwork, order them to minimise travel, then serialise them into the target machine language*. This is the single most important abstraction in the whole system.

❶ The core separation principle

Keep the **geometry** (chain production) completely independent of the **machine language** (chain serialisation). Only the final emission stage - thin and unique - knows about a specific machine. If a function computing a hatch pattern mentions **G1** or **PD**, the design is wrong.

3. The five-stage pipeline

Turning a drawing into code passes through five distinct stages. Each has a single responsibility and communicates with the next through a clear contract.



The first four stages are **pure geometry**, entirely unaware of the machine. Only stage 5 - thin - translates chains into a concrete language. Swapping GRBL for HPGL changes only this stage.

1 · Resolve geometry

Bake every transform (translate/rotate/scale/shear) into world space; choose the fill rule; identify holes.
→ Ch. 2

2 · Extract primitives

Flatten Bézier/arc curves to polylines at a controlled tolerance; separate the *outline* from the *interior area*. → Ch. 2, Ch. 4

3 · Plan chains

Per source: solid area → fill engine; pattern → tile & clip; text → skeleton; image → scanline modulation.
→ Ch. 3, 5, 6, 9

4 · Order chains

Minimise travel with nearest-neighbour, choose entry points, re-root closed loops. → Ch. 7

5 · Emit dialect

Modal tool state, coordinate formatting, preamble/postamble, coordinate-system transforms. → Ch. 8

4. Glossary

These terms recur throughout the book. You need not memorise them now - return here when you hit an unfamiliar word.

Term	Meaning
Chain / polyline	Ordered point list traced in one engaged pass.
Working move	Motion with the tool engaged (marks the material).
Travel / rapid	Motion with the tool disengaged (no mark).
Pen-up / pen-down	Generic names for disengage / engage, whatever the tool.
Flattening	Approximating curves (Bézier/arc) by short line segments.
Tolerance	Max allowed deviation between the true curve and the flattened polyline.
Fill rule	How nested subpaths decide inside/outside (even-odd vs non-zero winding).
Hole / counter	An interior region that must NOT be filled (e.g. the hole in "O").
Hatch	Parallel straight fill lines.
Boustrophedon	"Ox-plough" back-and-forth path; alternating scanline direction.
Contour / offset fill	Concentric inward copies of the boundary.
Kerf / tool offset	Physical width the tool removes/leaves; requires geometric offsetting.
Overcut	A small extension past a closed loop's start so a blade fully severs it.
Modal state	A machine setting that persists until changed (units, power, feed).
Dialect	A concrete command language (GRBL, HPGL, Marlin...).
Distance field	A grid where each interior cell holds its distance to the boundary; its iso-lines are the inward offsets (used for robust Contour/Spiral, Ch. 2).
Iso-contour	A curve of constant value in a field; here, the set <code>{distance = d}</code> = the inward offset by <code>d</code> . Extracted with <i>marching squares</i> .
Nest	A maximal group of concentric offset rings reachable without a lift; a hole or pinch starts a new nest. Spiral emits one stroke per nest.

Term	Meaning
Lift	A pen-up / disengage-travel-re-engage cycle; the dominant time cost, hence fills are optimised to minimise it (Ch. 3).
Luminance	Perceptual brightness $0.299R+0.587G+0.114B$ of a fill colour; drives shade-by-colour fill density (Ch. 3).

5. Seven principles for generating this code

1. **Never couple geometry to a dialect.** Geometry returns chains; a separate emitter turns chains into text.
2. **A chain is the contract.** Design each feature to *append chains to a list*. Ordering and emission then work uniformly for all of them.
3. **Every tunable has a rationale.** Spacing, tolerance, thresholds – state *why* the value is what it is, so it can be re-tuned intelligently.
4. **Prefer connected paths over disjoint ones.** Time cost is dominated by tool lifts and travel, not by engaged distance (see the cost model in Ch. 3).
5. **Always have a fallback.** Any strategy can degenerate on a pathological shape; a robust generator falls back to the simplest correct method rather than emitting nothing.
6. **Prefer topology-robust primitives.** A primitive that quietly fails on holes or concavity (e.g. a naive per-vertex inset) makes the selector above it *lie*: it picks the right strategy, which then silently degrades. Build the offset on a **distance field** (Ch. 2) so it works for any topology, and the whole fill stack becomes honest.
7. **An auto-selector encodes its strategies' current capability.** When you strengthen a strategy (make Spiral topology-robust, see Ch. 3), **re-tune the selector** to exploit it – otherwise the improvement is left unused. Selector rules are not permanent; they track what the strategies behind them *can now do*.

✓ Grasp this chapter and you have half the picture

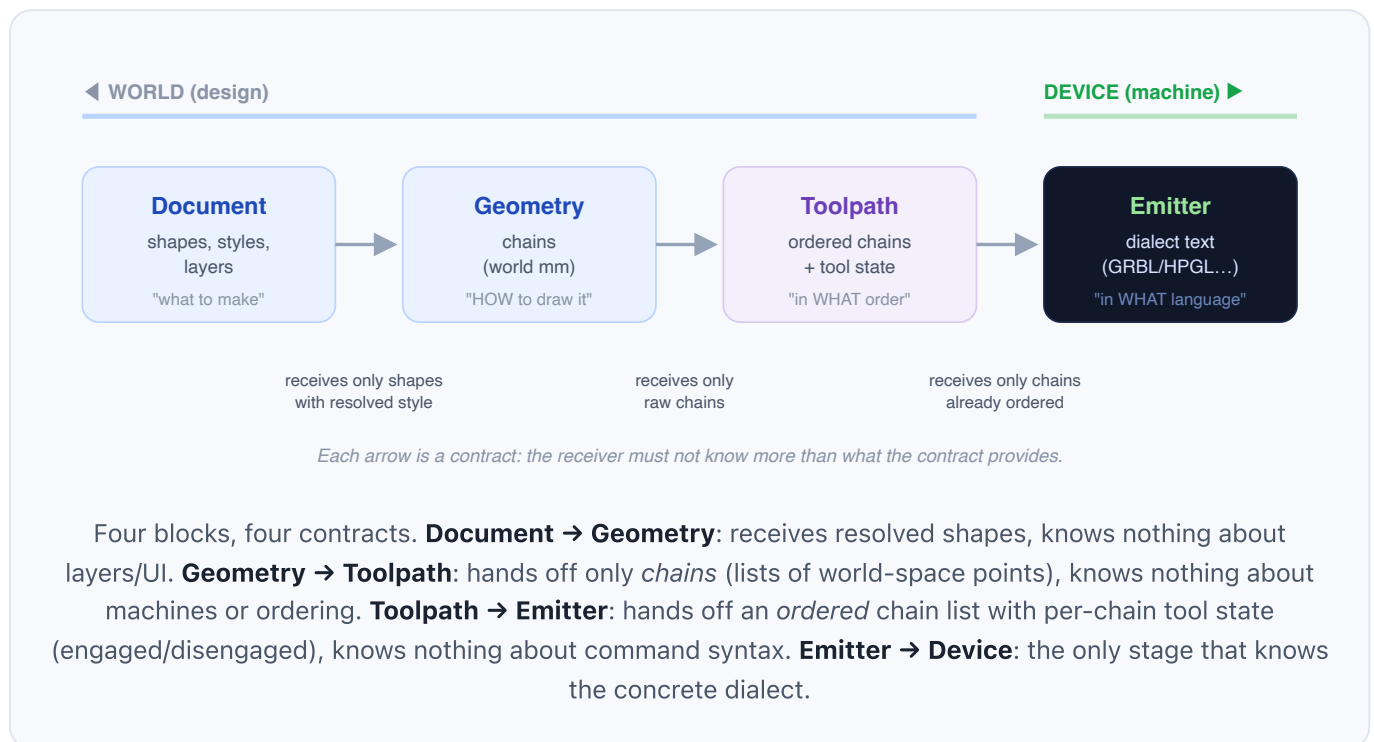
The rest of the book simply answers: *how to produce the best chains* for each source type (fill, outline, text, image), *how to order* them, and *how to translate* them into code. It all rests on the chain model you just learned.

Architecture & Pipeline

Chapter 0 told us the destination: every drawing reduces to a set of **chains of points**. This chapter answers the next question: what system produces that set of chains in an organised way, so that adding a new machine or a new fill algorithm doesn't bring the whole building down? The answer is a strictly layered architecture, with five sequential stages and four boundary contracts that must never be broken.

1. The layered responsibility model

Picture the whole system as a pipeline of four functional blocks, running from **WORLD** space (the design, in the mm units the user sees) to **DEVICE** space (the machine, in the units its controller understands). Each block knows exactly what it needs to know, and nothing more:



The most important feature of this model is that it is **unidirectional**: information only flows left to right, and no block ever "looks back" at an earlier one for more detail. Geometry doesn't need to know which layer a shape sits on; Toolpath doesn't need to know whether the machine is GRBL or HPGL; Emitter doesn't need to know the fill rule, because by the time it receives data, every geometric decision has already been finalised.

☆ The layer-correctness test

If you **cannot write a unit test** for a stage without instantiating a specific machine dialect, the architectural layering has leaked. A fill-pattern function that reads `machine.type == "grbl"` somewhere inside it is a clear warning sign.

2. Stage 1 - Resolve geometry

The goal of the first stage is to turn "a shape with a transform and a style" into **world-space outlines**, together with the knowledge of which regions are interior. Three things must happen, in this order:

1 · Bake transforms

Compose the shape's local transform with the transforms of every ancestor (group/layer), then apply it to the local outline to produce a path in world space. Rotation, scale, and shear must all be baked *before* flattening, so tolerance is measured in world units, not in a distorted local unit.

2 · Choose a fill rule

even-odd or **non-zero winding**. Even-odd is the safe default: "a hole is wherever subpaths overlap an odd number of times" - it is independent of subpath winding direction. Details in Ch. 2.

3 · Classify the geometry role

Which of the three roles in §5 below does this shape belong to - this decision drives the entire downstream branch.

The output of this stage is a resolved path in world coordinates, together with its style: does it have a fill? a stroke? which pattern? what shade/level?

3. Stage 2 - Extract primitives

The goal is to convert continuous curves into polylines, and split the geometry into the two things every downstream stage consumes:

Outline geometry

The boundary polylines themselves - the shape's edge/outline.

Interior area

The closed interior region, used by fill and pattern engines.

The key operation here is **curve flattening** at a controlled tolerance (see Ch. 2 §1). Every downstream stage operates only on polylines, never on parametric curves - so this stage centralises all curve math in one place, instead of scattering it across the system.

4. Stage 3 - Plan chains (the dispatch)

For each resolved shape, the system must decide *which chain source(s)* apply - purely based on that shape's geometry and style, not on any "machine mode". This is a style-driven dispatch, independent of the machine:

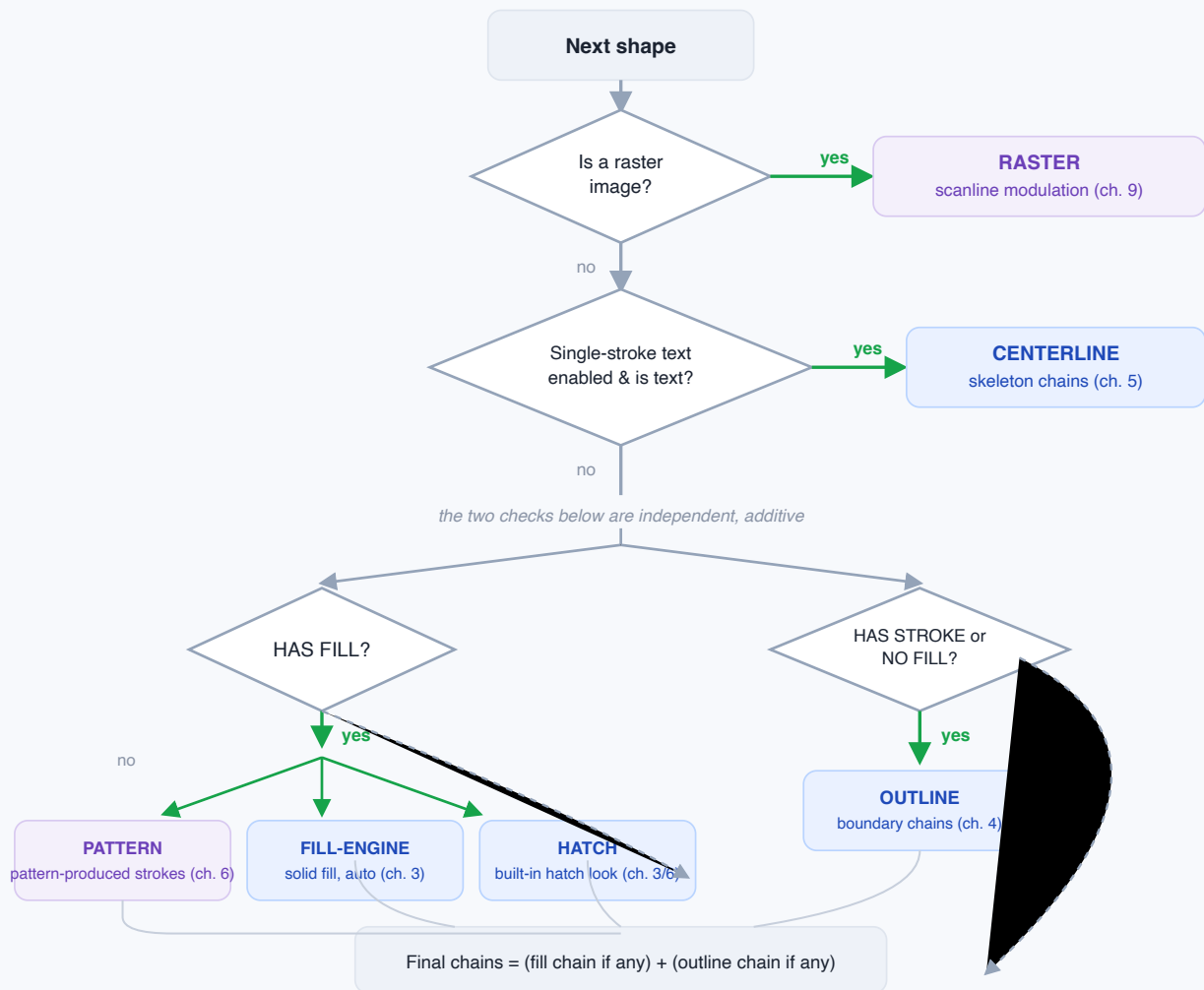
```
for each shape in draw_order:
    if shape is raster_image:
        → RASTER path (scanline modulation)    # ch. 9; NOT chains
        continue

    if single_stroke_text and shape is text:
        → CENTERLINE chains                    # ch. 5
        continue

    if shape.has_fill:                          # covers the interior
        if fill_is_pattern and tiling_produced_strokes:
            → PATTERN chains (tile + clip)      # ch. 6
        elif fill_is_solid:
            → FILL-ENGINE chains (auto strategy) # ch. 3
        else: # built-in hatch look, or texture producing nothing
            → HATCH chains at the pattern's angles # ch. 3/6

    if shape.has_stroke or not shape.has_fill: # draws the boundary
        → OUTLINE chains                        # ch. 4
```

This is the whole decision. Note that it is **additive**, not exclusive: the last two `if` blocks can both be true at once for the same shape.



The dispatch tree: raster and centerline are two **overriding exceptions** that fully replace the normal branch; fill and stroke below are **two independent, additive** checks - a shape with both a fill and a stroke passes through both and emits both kinds of chains.

i Three additive cases to distinguish

A shape with both fill and stroke: emits *both* fill chains *and* outline chains - the interior is covered and the border is drawn separately (for the concrete order/settings/shared-edge decisions this leaves open, see Ch. 4 §2b).

A shape with fill, no stroke: emits only fill chains - the design does not call for a border, so none is invented. Note: *not* because "the fill reaches the boundary" - inset-based fills typically leave a thin unfilled rim, see Ch. 4 §2.

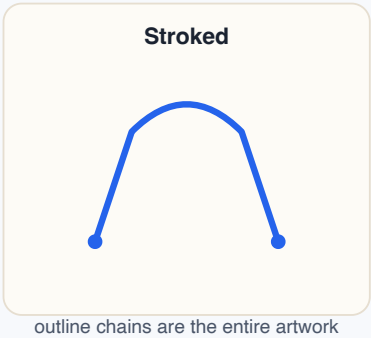
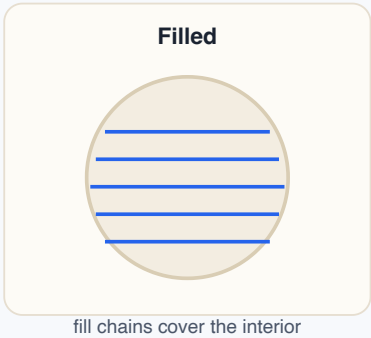
An unfilled shape (an open path, a shape with only a stroke, or one with neither): emits only outline chains - with no interior, the boundary is the entire artwork.

The dispatch order also encodes two overriding exceptions: single-stroke text *entirely replaces* the normal outline/fill handling of a text shape (Ch. 5); and raster is handled completely separately because it is not chain-based.

5. The three geometry roles

Every shape falls into exactly one of three **geometry roles**. Finding the best toolpath for each role is the entire job of Stages 3-5:

Role	Detection	Chains produced	Best-fit strategy
Filled	has a valid fill	fill/pattern chains (+ outline if also stroked)	fill engine picks the time-optimal covering path from the region's shape (Ch. 3)
Stroked (outline)	has a stroke, or has no fill	outline chains	flatten boundary → chains; order for minimal travel (Ch. 4, Ch. 7)
Text (single-stroke)	text + option enabled	centerline chains	skeleton extraction so each glyph is a single pass (Ch. 5)



Three roles, mutually exclusive **for chain-generation strategy**: solid regions use the fill engine, boundaries use outline chains, single-stroke text uses skeleton extraction. A filled-and-stroked shape is just the Filled role plus an added outline chain, not a fourth role.

□ No "cut vs draw" branch

This architecture has **no** separate branch for "cutting machine" versus "drawing machine". The dispatch depends only on what the geometry *is* (fill / stroke / neither), so the same logic produces the correct chains for any point-to-point machine. Tool-specific refinements a cutting tool may want (overcut, kerf offset) are optional post-processing on the already-chosen chains (Ch. 4 §4), not a separate pipeline.

6. Stage 4 - Order chains

Take the unordered chain list from Stage 3 and produce an order plus a direction for each chain that minimises pen-up travel. Detailed in Ch. 7. This stage is **pure geometry**: it never emits a machine command.

Ordering is performed **per layer / per tool group**, because different layers carry different machine settings (power, speed, force, passes) and must be emitted as contiguous blocks, not interleaved.

7. Stage 5 - Emit dialect

Serialise the ordered chains into machine commands. This is the only stage that knows about the dialect. It has four responsibilities:

Preamble (units, absolute mode, tool init, home) →

Track modal tool state (engaged/disengaged, power, feed) →

Coordinate-system transforms (home corner, mirror, rotation, rotary wrap) →

Postamble (disengage tool, return home, end)

Tracking modal state ensures redundant commands are never emitted - if the laser power is already at 800, there's no need to resend a power-800 command for every segment. Full details in Ch. 8.

8. Four coordinate spaces

Bugs tend to cluster at coordinate-space boundaries. So all four spaces must be named explicitly, never conflated:

1 • Local space

A shape's own coordinates, before its transform is applied.

2 • World / design space

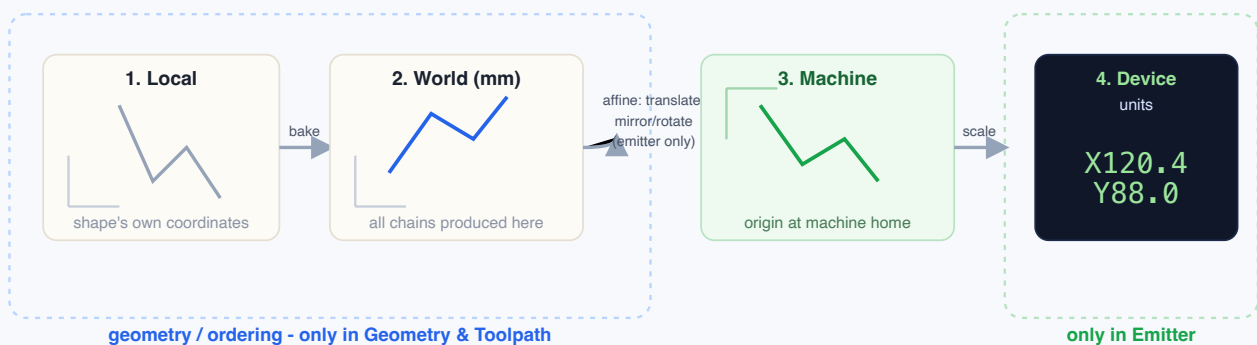
After baking every transform; a common mm plane with a defined origin corner and axis directions (commonly +X right, +Y down, like screen space). **All** chain production happens here.

3 • Machine space

The device's own coordinate system: origin at the machine's home corner, axis directions per the hardware, possibly mirrored or rotated relative to world space.

4 • Device units

Machine space scaled to the controller's integer/decimal unit (e.g. mm for GRBL; 1/40 mm "plotter units" for basic HPGL).



Chains are produced and ordered **entirely** in World space. The world → machine transform (a small affine map) is applied only **in the emitter**, one point at a time, exactly once - so the preview always shows world space while the machine receives machine space.

Rule

Produce and order chains entirely in world space. Convert to machine space + device units **exactly once**, per point, at emission time. Keep travel-to-home moves in raw machine coordinates, so "home" is always the true device origin, never passed through the world transform.

9. Preview / simulator parity

If the product displays a simulated toolpath, drive the simulator from the **exact same ordered chain list** the emitter consumes, tagging each move as engaged or travel. This guarantees that "what you preview is what the machine will do". Keep the simulator geometry in world space (readable to the user) and let only the emitted text carry the machine transform - never maintain two independent path computations.

10. Robustness invariants

The following three principles apply to every chain-generation stage, regardless of the specific algorithm:

Never emit "nothing" for a non-empty area

Every planning strategy must have a fallback to a trivially-correct method - ugly beats empty.

Skip incompatible sources gracefully

A blade machine silently drops raster images; a pen machine drops power modulation. Don't error - just omit.

Bounded work

Every generator must cap its output (max rings, max sites, max raster cells, an ordering-optimisation cutoff), so that a tiny spacing on a huge area cannot hang the machine or exhaust memory. When a cap forces the result to be coarsened, record that as a human-readable annotation.

✓ **Architecture is the frame; the rest is content**

This chapter gave you the frame: four blocks, five stages, four coordinate spaces, three robustness invariants. The following chapters fill in each cell of this frame - starting with the geometry foundations (curve flattening, fill rule, offsetting) in Chapter 2.

Geometry Foundations

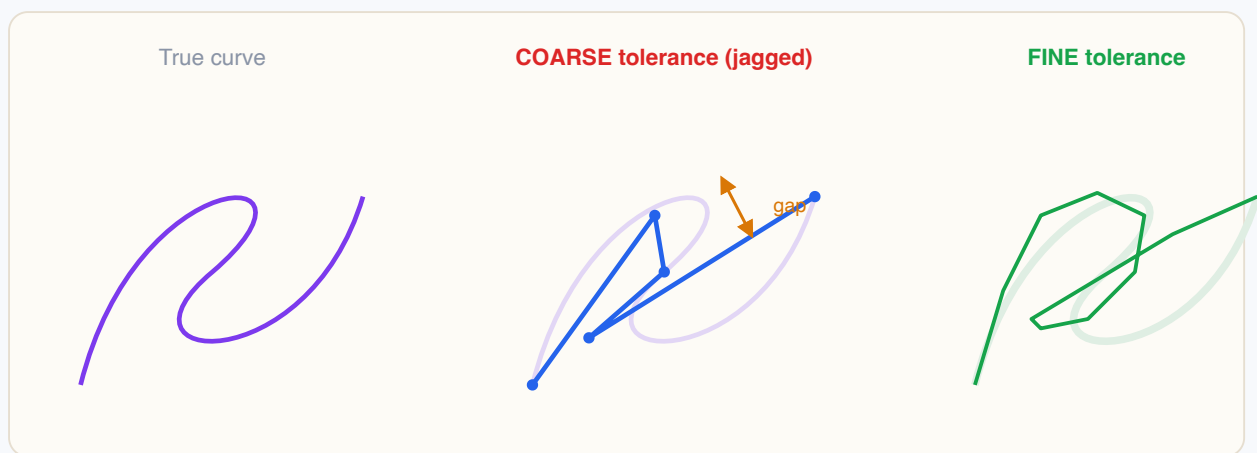
Every chain source - fill, outline, text, pattern - is built on the same small set of geometric primitives. Implement these once, correctly, and the rest becomes straightforward. All operations here are **dialect-independent**: they are pure 2-D computational geometry.

i Where this chapter sits in the pipeline

This is the toolkit for *stage 1 (resolve geometry)* and *stage 2 (extract primitives)* introduced in Chapter 1. After this chapter, every drawing has become clean polylines plus inside/outside information - ready for the fill engines in Chapter 3 and the outlines in Chapter 4.

1. Curve flattening (Bézier / arc → polyline)

Machines move point-to-point, so every curve must be approximated by short straight segments. The only parameter is **tolerance**: the maximum distance allowed between the true curve and the polyline chord.



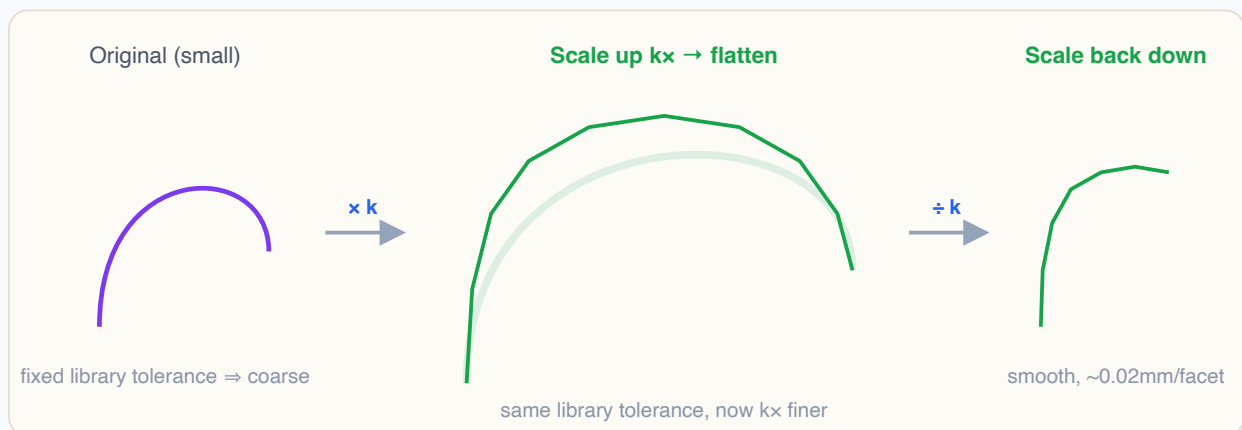
— True curve — Flattened polyline — Chord/arc gap = tolerance

Tolerance is the largest distance between the **chord** (straight segment) and the **arc** (true curve). Coarse tolerance ⇒ few points but visibly flat facets (jagged); fine tolerance ⇒ many points but hugging the curve closely.

The scale-up trick (uniform world-space tolerance)

Many vector libraries flatten with a tolerance fixed in *path units*, which is too coarse at millimetre scale and produces visibly faceted curves (the flat sides of an "O" or "S"). The robust, library-agnostic fix:

- 1 · Scale the path up by a factor **k** (e.g. **k = 25**) →
- 2 · Flatten in the enlarged space →
- 3 · Scale every resulting point back down by **1/k**



Effective tolerance $\approx \text{libraryTolerance} / k$. With **k = 25** this yields facets of about $\sim 0.02\text{mm}$ - smaller than the resolution of any pen, blade, or laser, so the curve reads as smooth.

Keyword to remember: *adaptive / subdivision flattening with tolerance control*.

⚠ Trade-off: smaller tolerance, larger output

Smaller tolerance $\Rightarrow$ more points $\Rightarrow$ larger output and (on some controllers) slower streaming. Pick tolerance at "just below the tool's mark width". **$\sim 0.02\text{mm}$ is a safe universal default** for mm-scale artwork.

Fine vs coarse contexts

Use the **fine** tolerance for any geometry whose points become *emitted output* or must *align with other output*: outlines, glyph curves, and any region boundary whose intersections produce visible fill/pattern endpoints. A **coarser** tolerance is acceptable only for boundaries used in *decisions* that never emit a vertex and tolerate $\sim$ sub- 0.05mm error: shape-metric estimation, tile-count bounding, coarse inside/outside tests.

⚠ **Never clip a visible fill line against a coarse boundary**

Its endpoints would land off the true edge and misalign with a fine outline (this is exactly the seam bug described in 10 §2.4). **Centralise** both tolerances in one place so the whole system stays consistent.

2. Polyline / polygon representation

- A **polyline** is an ordered point list. **Open** = distinct endpoints; **closed** = first point $\approx$ last point (within an epsilon).
- Test *closed* by `|first - last| <  $\epsilon$` **and** at least 3 (often ≥ 4) points.
- A shape with holes is a **set of subpaths** (outer boundary + inner boundaries). Keep them as separate polylines and let the *fill rule* decide inside/outside; do not try to merge them into one ring.

★ **Epsilon: compare using squared distance**

Use a squared-distance epsilon (ϵ^2) for closeness tests to avoid `sqrt` . A typical value is `$\epsilon = 1e-3$` mm.

3. Fill rules: even-odd vs non-zero winding

To decide which points are "inside" a multi-subpath shape:

Even-odd (parity)

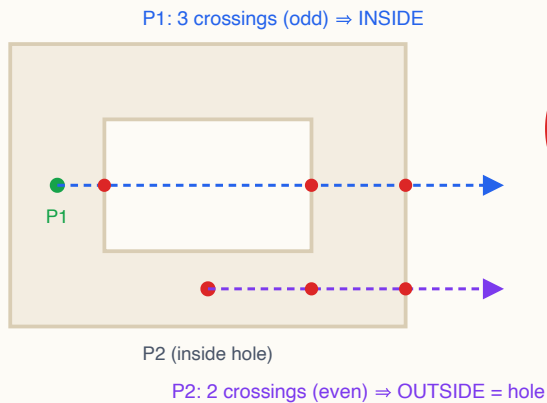
A point is inside if a ray cast from it crosses the boundary an **odd** number of times. Direction-independent - subpath winding does not matter.

This is the safe default for arbitrary imported artwork, glyph counters, and concentric rings.

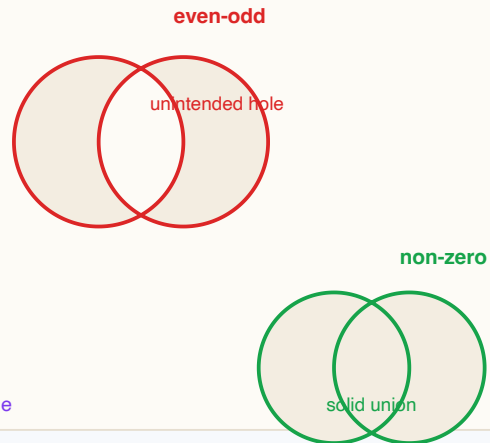
Non-zero winding

Sum the **signed** crossings; inside if the sum $\neq 0$. Needs consistent winding direction; can fill regions that even-odd would treat as holes.

Shape with a hole · counting ray crossings



Two OVERLAPPING regions



■ Inside point ■ Boundary crossing --- Test ray

Left: for a shape with a hole, an **odd** crossing count $\Rightarrow$ inside, **even** $\Rightarrow$ outside (hole). Right: when two same-direction regions *overlap*, even-odd carves an **unintended hole** in the intersection, while non-zero keeps the region solid.

☆ The precise decision

The hole/nesting model in §4 is an **even-odd** model, and even-odd is the safe default for *nested* holes (counters, donuts) regardless of winding. But even-odd carves an **unintended hole wherever two contours overlap** (rather than nest) – for example two overlapping same-direction blobs, or an accent overlapping a glyph, that the designer meant to read as **solid**. So the precise rule is:

- Subpaths that are **nested** (a hole fully inside another) $\rightarrow$ even-odd is correct and winding-independent; use it.
- Subpaths that **overlap** and should union into a solid $\rightarrow$ honor the source's declared **non-zero winding** fill rule (most fonts and SVGs use it), or **pre-union** the overlapping same-winding contours (12 §3.2) *before* the even-odd fill.

This is why glyph rasterisation (05 §3.1) fills with **non-zero winding**: a font's overlapping contours must read solid. The nesting-depth test (§4) is winding-independent, so it stays correct under either rule.

Point-in-polygon test

The standard **ray-casting / crossing-number** algorithm: cast a horizontal ray and count edge crossings, using the half-open vertical span rule $(a.y > p.y) \neq (b.y > p.y)$ to count a shared vertex exactly once. $O(n)$ per query. Used for hole detection and for the "is this midpoint inside?" test when clipping (§6).

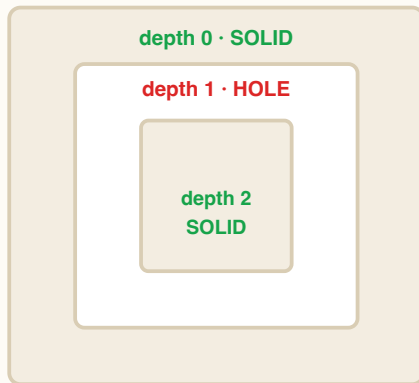
```
// Ray-casting: is point p inside polyline poly?
function pointInPolygon(p, poly) {
  let inside = false;
  for (let i = 0, j = poly.length - 1; i < poly.length; j = i++) {
    const a = poly[i], b = poly[j];
    // half-open rule on the vertical axis: count each shared vertex exactly once
    if ((a.y > p.y) !== (b.y > p.y)) {
      const xCross = a.x + (p.y - a.y) / (b.y - a.y) * (b.x - a.x);
      if (p.x < xCross) inside = !inside;
    }
  }
  return inside;
}
```

4. Hole & nesting detection (even-odd nesting depth)

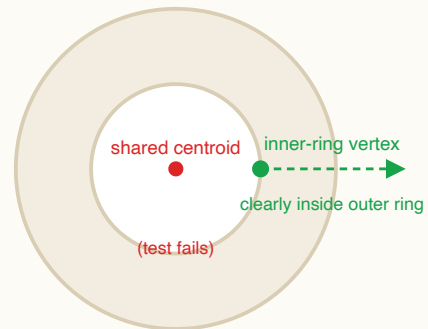
To know which subpaths are solid and which are holes:

1. For each subpath, compute its **signed area** using the **shoelace formula**: $A = \frac{1}{2} \sum (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i)$. $|A|$ is the area; the sign gives the winding.
2. Compute a **nesting depth** for each subpath by testing *one of its vertices* against every *larger* subpath with point-in-polygon. Depth = number of larger subpaths that contain it.
3. **Even depth** $\Rightarrow$ **solid** (adds to the net area); **odd depth** $\Rightarrow$ **hole** (subtracts).

Concentric rings · nesting depth



Test a VERTEX, not the centroid



Left: nesting depth increases through the nested rings; even is solid, odd is a hole. Right: two concentric rings (a donut, or the outer/inner of a letter "O") **share a centroid**, so a centroid test fails. A **boundary vertex** is clearly inside or outside the other ring.

☆ Why test a vertex, not a centroid

Two concentric rings (a donut, or the outer/inner of a letter "O") can share the same centroid, so a centroid test fails. A boundary vertex is unambiguously inside or outside the other ring. This makes nesting **winding-independent** and correct for both donuts and letter counters.

Net area = $\Sigma(\text{solid areas}) - \Sigma(\text{hole areas})$. Hole count = number of odd-depth subpaths.

5. Convex hull, oriented bounding box & shape metrics

The automatic fill selector (Chapter 3) classifies a region from cheap scalar metrics. Compute them once:

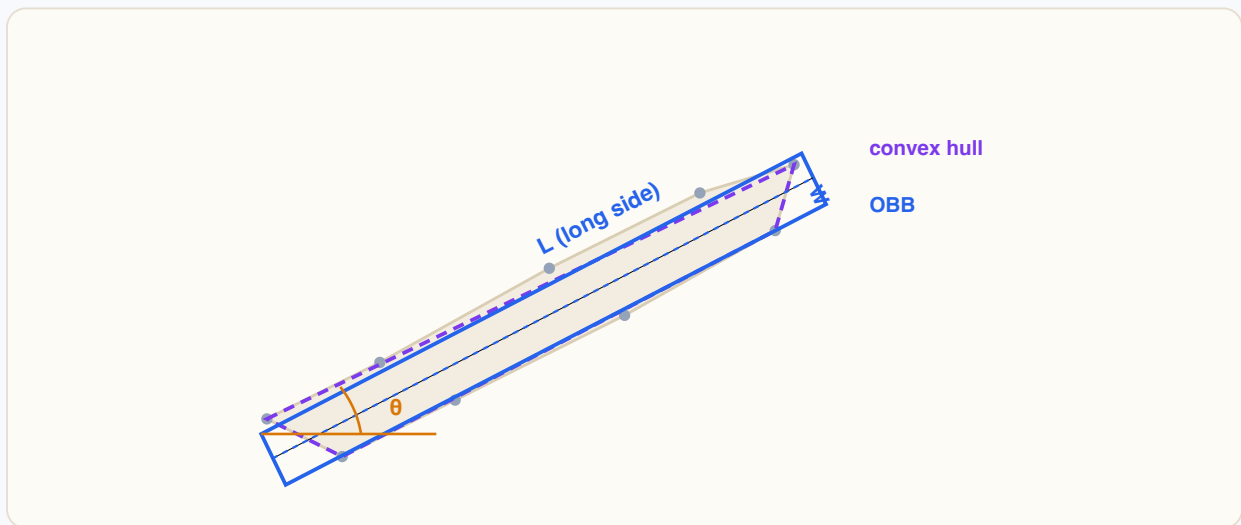
Convex hull

The **Andrew's monotone-chain** algorithm: sort points by (x, then y), build the lower then upper hull with a cross-product turn test, $O(n \log n)$. The hull gives both a convexity measure and the basis for the oriented bounding box.

Minimum-area oriented bounding box (OBB)

Rotating calipers, based on the theorem that a side of the minimum-area rectangle must be collinear with a hull edge: for each hull edge, project all hull vertices onto that edge's direction and its perpendicular, take the extents, compute the rectangle's area, and keep the minimum. Record:

- **L** = long side length, **W** = short side length,
- **θ** = orientation of the long axis (degrees),
- **center**.



--- Convex hull (monotone chain) — OBB (rotating calipers) — θ · long-axis orientation

The convex hull tightly wraps the point set; the OBB is the minimum-area rectangle, one side collinear with a hull edge. This "principal axis" orientation θ is exactly what connected fills align to (filling along the long axis $\Rightarrow$ the longest runs, the fewest direction changes).

Derived metrics (all normalised, cheap, comparable)

Metric	Formula	Meaning
area	net area (§4)	true filled area
perimeter	Σ segment lengths	boundary length
elongation	L / W	1 = square-ish, large = a thin bar
rectangularity	$area / (L \cdot W)$	how fully it fills its OBB (1 = a rectangle)
convexity (solidity)	$area / hullArea$	1 = convex, <1 = concave / spiky
compactness	$4\pi \cdot area / perimeter^2$	1 = a disc, $\rightarrow 0$ = long / spidery
mean stroke width	$2 \cdot area / perimeter$	average local thickness of the region
width in lines	$meanWidth / spacing$	how many fill passes fit across it
holes	count of odd-depth subpaths	topology

Just geometry, not machine learning

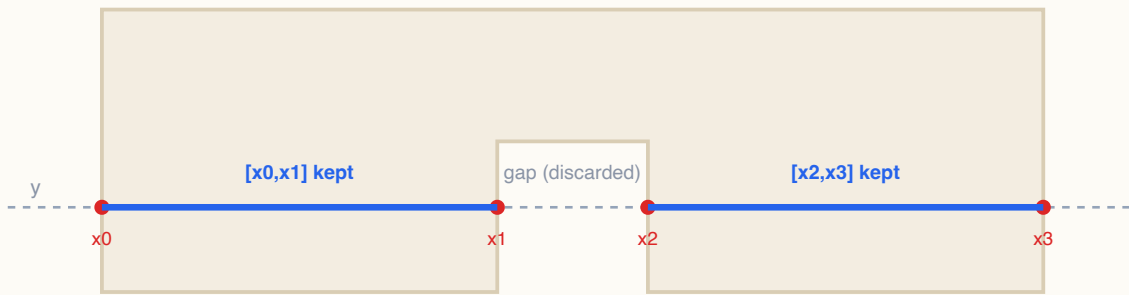
These metrics are the *entire* input to the auto-fill decision - no ML, no heuristic soup, just interpretable geometry. The identity $mean\ stroke\ width = 2A/P$ is the "average thickness" for a long thin region ($area \approx length \times width$, $perimeter \approx 2 \times length$).

6. Clipping a polyline to a region

Fills and patterns generate lines across the whole bounding box, then keep only the parts *inside* the region. Two common needs:

Straight scanline against subpaths (fast path)

For a horizontal scanline at height y , find every edge crossing, sort the x values, and pair them: $[x_0, x_1] [x_2, x_3] \dots$. Each pair is an inside span (this is exactly the even-odd rule applied to a single line). Used by hatch / zigzag (Chapter 3); $O(edges)$ per scanline.



The scanline crosses the boundary at 4 points; sorting the x values and pairing them as `[x0,x1]` and `[x2,x3]` gives two inside spans. The notch in the middle (between x1 and x2) is discarded because it lies outside the region.

Arbitrary polyline against an arbitrary region (general path)

For each segment of the polyline:

1. Find all intersection parameters `t ∈ (0,1)` with the region boundary.
2. Sort them; they split the segment into sub-segments.
3. Keep a sub-segment if its **midpoint** lies inside the region. Any even-odd inside-test works here - the hand-rolled ray-cast from §3, or a geometry library's own even-odd `contains()`.
4. Concatenate adjacent kept sub-segments into output chains; a gap flushes the current chain.

Keyword: *segment–polygon clipping*. Used by space-filling curves and pattern tiling.

Half-plane clipping (for cells)

Sutherland–Hodgman polygon clipping against a half-plane is the primitive for Voronoi cells (Chapter 3): clip a polygon to the side of a bisector line closer to one site.

7. Offsetting a region inward (inset)

Contour / spiral fills and kerf compensation need an **inward offset** of a closed region by a distance `d`. Repeatedly inseting by the line spacing yields the concentric rings those fills draw. There are two robust constructions; pick according to what your environment provides.

Method A - stroke-and-subtract (boolean erosion)

A library-agnostic construction using a stroker:

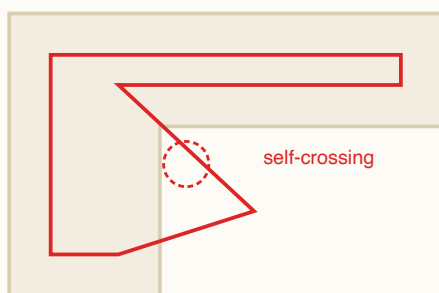
1. Stroke the boundary with pen width $2d$ (the stroke straddles the outline, d inside and d outside), using **miter** joins and **flat** caps.
2. **Subtract** that stroke band from the original region.
3. The result is the region shrunk by d on every side; holes grow inward correctly, and thin necks automatically pinch off into separate pieces.

Keyword: *polygon offsetting* / *erosion* (a.k.a. Minkowski erosion by a disc; stroke-and-subtract approximates it with the chosen join style). This is exact and resolution-independent, but it **requires a robust boolean/offset engine** (a Clipper-class library).

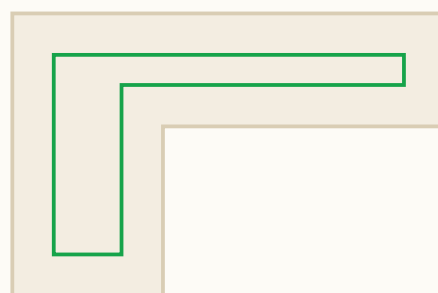
⚠ Do not substitute a naive per-vertex inset

Do not use the naive inset that "moves each vertex a distance d along its angle-bisector normal": it **self-intersects** the moment the region is genuinely concave (a star point, the inside of an "L"), and it has no notion of holes or of a region splitting into several pieces - it silently produces crossing, garbage rings. If a per-vertex inset is all you have, it is safe **only** for a single, near-convex ring; detect anything else and fall through to Method B.

Naive per-vertex · self-intersects at concave corner



Correct offset (Method A/B)



At a **concave** corner (the inside of an "L"), the naive per-vertex inset moves vertices past each other and creates a self-crossing loop (left). A correct offset (right) keeps a valid boundary evenly parallel to the edge.

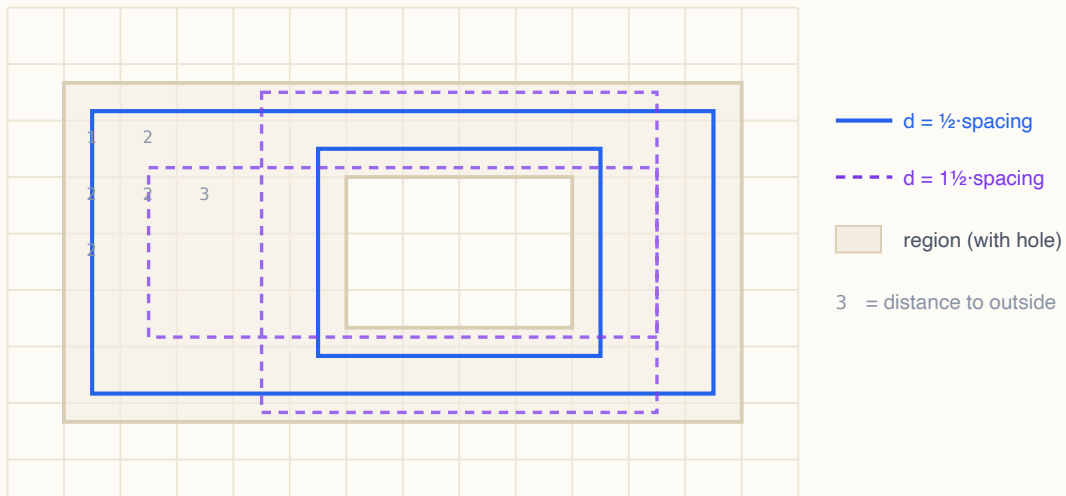
Method B - distance field + iso-contours (topology-robust, dependency-free)

When no boolean-offset library is available - or the artwork routinely has holes, concavity, or many disjoint regions - offset *without any boolean operations* by reading the inset off a distance field.

This is the standard basis for CAM "contour-parallel" / "peel" machining, and it is completely topology-agnostic.

1. *RASTERISE* the even-odd interior (§3, §6) into an inside/outside mask on a grid with step $h \approx \text{spacing} / 3$. (h a fraction of the line spacing keeps the iso-lines smooth rather than staircased; pad the grid by a couple of cells of guaranteed-outside border so boundary cells resolve. Cap total cells and coarsen h if the region is large - see 11 §3.)
2. *DISTANCE TRANSFORM*: compute, for every interior cell, the distance to the nearest exterior cell. A two-pass CHAMFER transform with weights $(1, \sqrt{2}) \cdot h$ differs from true Euclidean by at most ~8% (it always over-estimates, worst near the ~22.5° diagonal) - negligible against the line spacing - and needs no libraries. (An exact separable Euclidean DT - Felzenszwalb-Huttenlocher - or a library EDT is a drop-in upgrade when available.)
3. *ISO-CONTOURS*: the inward offset by distance d is exactly the iso-line $\{ \text{distance} = d \}$. Extract iso-lines at $d = \frac{1}{2} \cdot \text{spacing}, 1\frac{1}{2} \cdot \text{spacing}, 2\frac{1}{2} \cdot \text{spacing} \dots$ with MARCHING SQUARES (linear interpolation of the crossing point on each cell edge), LINK the resulting segments into closed rings by matching quantised endpoints, and apply one CHAIKIN pass to shave the grid staircase.

Distance field on a grid → iso-lines are inward offsets



— First ring at $d = \frac{1}{2} \cdot \text{spacing}$ - - - Ring at $d = 1\frac{1}{2} \cdot \text{spacing}$

The even-odd interior is rasterised into a mask; the distance transform assigns each cell its distance to the nearest outside cell. Iso-lines $\{\text{distance} = d\}$ are exactly the inward offsets. Note both the outer boundary and the **hole** grow inward at the same time - they are simply features of the same scalar field.

✓ Why it is robust

Holes, concavities, thin necks, and several disjoint blobs are all just features of one scalar field - the iso-lines split, merge, round and vanish correctly with **no special cases and no self-intersection**. The first ring at $d = \frac{1}{2} \cdot \text{spacing}$ sits half a line-spacing inside the boundary (the standard contour start point, 03 §3.4). Each level can yield several rings (one per nest); return them all, closed (first point == last point).

Cost: the transform is $O(\text{cells})$; marching squares is a single pass, $O(\text{cells} + \text{total contour length})$. Both are bounded by the cell cap. See 11 §3.

ⓘ Method A vs Method B trade-off

Offsetting via a distance field is an *approximation* at grid resolution (corners read slightly rounded when h is coarse), but it **never** produces invalid geometry and needs no dependencies. Method A is exact but needs a real offset engine. A good default: **Method B as the always-correct primitive**, with Method A (or a library offset) as an exact fast path for the simple single-ring case.

Keywords: *distance transform, chamfer distance, marching squares, iso-contour, contour-parallel machining.*

⚠ Degenerate case (either method)

If the region is thinner than `d` on every side, the inset is empty - detect this and stop (or keep the last non-empty ring). With Method B this is simply "no cell has distance $\geq d$ ".

8. Numerical hygiene

- Use squared distances for comparisons; only take `sqrt` when a real length is needed.
- Offset scanlines by **half a step** (`y0 = [top/spacing]·spacing + spacing/2`) so a line never lands exactly on a vertex (which would double-count crossings).
- Nudge "inside?" probe points a hair ($\approx 0.25 \cdot \text{spacing}$) toward the region's bounding-box center (a cheap approximate centroid) when testing points that lie *on* a boundary (connectors that run along an edge), so a valid edge is not rejected by a strict `contains()` that treats the boundary as outside.
- **Scrub every flattened polyline before use:** drop **non-finite** coordinates (`NaN` / `±inf` from a malformed transform or a degenerate curve) and collapse **consecutive duplicate points** (within a tiny ϵ). A stray `NaN` would otherwise reach the emitter as `X nan`; a zero-length segment causes a divide-by-zero in every downstream tangent/normal/offset computation. Do this once, at extraction time, so no later stage has to defend against it.
- Cap iterative constructions (rings, curve order, site count, **grid cell count**) so pathological input (tiny spacing, huge area) always terminates.

✓ Wrapping up this chapter

You now have a complete set of primitives: tolerance-controlled flattening, polyline representation, two fill rules with their point-in-polygon test, nesting depth to identify holes, convex hull + OBB + shape metrics, polyline clipping, and two ways to offset inward. Chapter 3 uses these metrics directly to auto-select a fill strategy; Chapter 4 uses offsetting to build contours and kerf compensation.

Fill Algorithms & Automatic Selection

A machine that only knows how to draw straight lines cannot "flood-fill" a region the way graphics software does; it must **cover** that region with a set of line **chains**. This chapter specifies every fill strategy, the **cost model** used to rank them, the **automatic selector** that picks the time-optimal strategy from a region's shape - plus how fill **density** encodes shade. Everything here returns **chains in world space** and is completely independent of the machine dialect.

i Where this sits in the pipeline

This is stage 3 - *chain planning* - of the five-stage pipeline (Ch. 0, Ch. 1). It consumes the geometric measurements (hull, OBB, the six metrics) from Ch. 2 and produces chains for the ordering stage and then emission. It has no idea what the target machine is.

1. The cost model (why one strategy beats another)

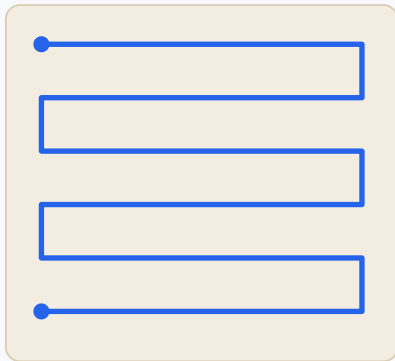
For a fixed area **A** and a given line spacing **s**, the **engaged (pen-down) length is essentially constant** for *any* line fill:

engaged length is almost independent of strategy
 engaged length $\approx A / s$

So engaged distance does **not** distinguish techniques. What differs - and what dominates real machine time - is:

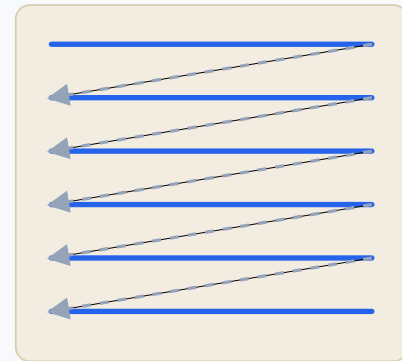
1. **Number of tool lifts** (disengage → travel → re-engage). Each lift costs a fixed dwell plus an acceleration penalty.
2. **Travel distance** between disjoint segments (pen-up moves produce nothing).
3. **Number of direction reversals** (decelerate → accelerate at each turn).

Zigzag · 0 lifts



1 continuous chain · 2 pen-down/up points

Hatch · 6 lifts



6 disjoint chains · 1 lift per line

=
same engaged
length

— Working stroke - - - Travel move (lift) ■ Material

The core idea of the whole chapter. The same region, the same number of lines, **the same engaged length** - but Zigzag joins the lines into *one continuous stroke* (0 lifts), while Hatch leaves them *disjoint* (one lift per line). It is the number of lifts, not the engaged length, that decides machine time.

★ Conclusion

A **connected** fill that draws the same lines with almost no lifts always beats a **disjoint** fill on time. This single fact drives the entire automatic selector (§4): it only ever chooses among the **connected** fills.

Class	Lifts	Best for
Connected (Zigzag, Spiral, Contour*)	~0 to a few	time-optimal; these are the Auto choices
Disjoint (Hatch, CrossHatch)	one lift per line	a specific look, manual only
Decorative (Hilbert, Peano, Voronoi)	varies	texture/aesthetics, manual only

* Contour draws separate rings (one lift per ring); Spiral connects them into one stroke.

ⓘ The lift floor is Spiral

Think of the tool as a pen plotter: the number of pen-lifts is the metric to minimise. Zigzag lifts ~0 times on a convex region but *fragments* (one chain per disjoint span) across holes and concavities. Contour lifts once per ring. **Spiral, given a topology-robust offset (§3.4) and a nest-aware connector (§3.5), lifts exactly once per nested region** - the theoretical floor, because a separate nest genuinely cannot be reached without a lift. On a donut this is the difference between ~2 lifts (Spiral) and dozens (Contour, one per ring) or many (Zigzag, one per span per scanline). This is why the selector in §4 prefers Spiral wherever the offset engine can produce rings.

2. Density: mapping shade to line spacing

Line **spacing** controls how dark a filled area reads and how long it takes to complete.

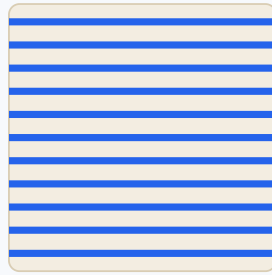
```
spacing = shadeFactor(level) × lineWidth
```

- `lineWidth` - the tool's mark width (pen nib, beam kerf). Lines closer than this overlap; lines much farther apart leave visible gaps.
- `shadeFactor(level)` - a multiplier per **fill level / shade**. Darker ⇒ smaller factor ⇒ tighter lines ⇒ more coverage. A representative four-level map:

Level	Factor	Effect
Black (100%)	0.8	lines slightly overlap → solid
Dark gray	1.5	small gaps
Gray	2.5	medium gaps
Light gray	5.0	sparse



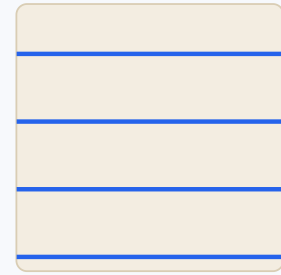
0.8 · Black



1.5 · Dark gray



2.5 · Gray



5.0 · Light gray

The *same* fill strategy, changing only the **line spacing**: factors $0.8 \rightarrow 1.5 \rightarrow 2.5 \rightarrow 5.0$ give a tonal ramp from dense black to light gray. Shade is a function of geometric density, not ink colour.

i Separation of concerns - but not numerically independent

Density (spacing) and *strategy* (path shape) are chosen by **separate mechanisms**: the user sets shade via density; the selector sets the path shape. They are not, however, fully independent numerically: the selector reasons about the shape *relative to the current spacing* (via `widthInLines` and the tiny-area rule, §4), so a large change in shade - hence spacing - **can legitimately move a shape into a different strategy**. A region many lines wide at a dark shade may be only one line wide at a very light shade, where a single contour genuinely *is* the right fill. This is intended: the strategy adapts to how many passes actually fit. What is guaranteed is (a) at a *fixed* shade the strategy is deterministic, and (b) shade never *overrides* an explicit user strategy choice - only `Auto` re-evaluates. If you need the path shape frozen across shades, pick an explicit strategy instead of `Auto`.

For a dense engraving/burn fill (laser), spacing is instead driven by the **beam/material interval** (e.g. ~ 0.15 mm), deliberately set far below any pen spacing floor, so the burn fully covers the area.

Deriving the shade level from fill colour (automatic tone)

When the shade is not set by hand, derive it from the region's **fill colour**: darker colour $\Rightarrow$ denser lines, lighter $\Rightarrow$ sparser - so a greyscale drawing engraves as a believable tonal image with no per-region tuning.

1. Compute the colour's **perceptual luminance** (0 = black ... 1 = white):

```
# R,G,B in the range 0..1
Y = 0.299·R + 0.587·G + 0.114·B
```

The weights matter: a pure red and a pure yellow of equal RGB magnitude read as very different darkness, and this luminance ranks them correctly.

2. Map luminance to a **spacing multiplier** on the base (darkest) spacing:

```
spacing(region) = baseSpacing · ( 1 + Y · (Kmax - 1) )
```

with the base spacing being the value for a **fully dark** fill (multiplier 1, densest) and **Kmax** the sparsest multiplier (a representative value is ~5, i.e. a light grey ends up ~5× sparser - matching the shade-factor table above). This is a continuous form of that table: black → 1×, mid-grey → ~3×, light → ~5×.

⚠ Where it lives

Apply the multiplier **per region, upstream of the selector**, so a lighter region re-classifies legitimately (its **widthInLines** shrinks - a region many lines wide when dark may be only one line wide when light, §2 above). Keep it a toggle: some jobs want uniform density regardless of colour. A near-white or **none** fill means "not filled" and produces no fill at all (only its outline, if any). Parse the colour with the host's colour model; fall back to a small hex / **rgb()** / named-colour parser so the mapping still works standalone.

3. The strategies

Each strategy is described as: *idea* → *algorithm* → *connectivity* → *when to use*.

3.1 Hatch (parallel scanlines) - disjoint

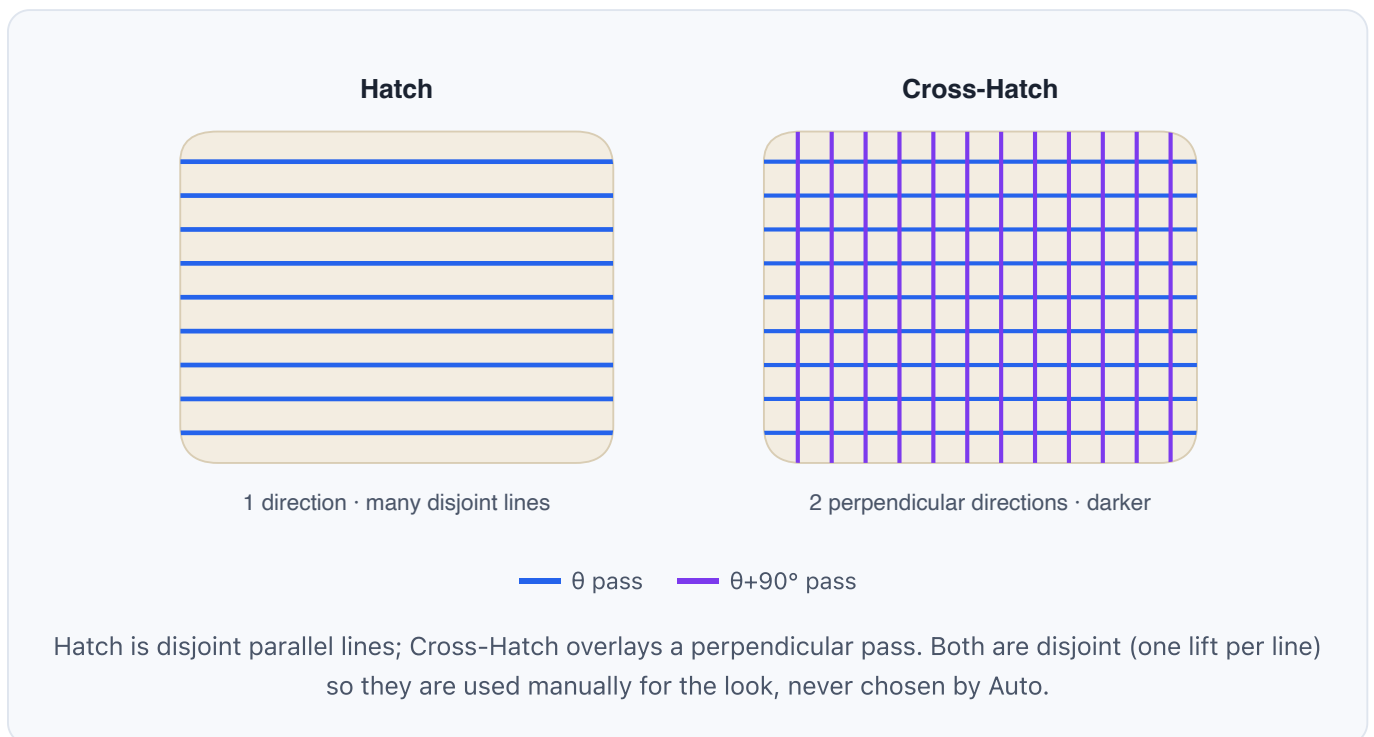
- **Idea:** fill with parallel straight lines at angle θ , spacing s .
- **Algorithm:**
 1. Rotate the region into a frame where the hatch lines are horizontal (rotate by $-\theta$ about the region center).
 2. For each scanline **y** (starting from $\lfloor \text{top}/s \rfloor \cdot s + s/2$, stepping by **s**), find all boundary crossings, sort the x's, and pair them into inside spans (even-odd on a line, Ch. 2 §6).

3. Emit each span as a 2-point chain; rotate back to world space.

- **Connectivity:** one chain per span $\Rightarrow$ **many lifts**.
- **Use:** manual only - when the user wants a literal "hatch look", a specific angle, or an architectural shading convention. Multiple angle sets give cross-hatch/grid.
- **Keyword:** *scanline polygon fill*.

3.2 Cross-Hatch - disjoint

Two Hatch passes at perpendicular angles (θ and $\theta+90^\circ$), or any angle set. Same disjoint cost; a darker/textured look. Manual only.

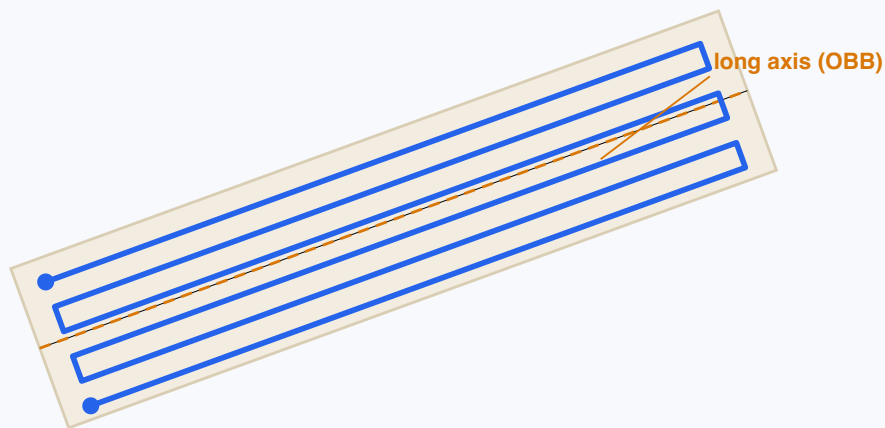


3.3 Zigzag (boustrophedon) - connected ★ Auto candidate

- **Idea:** the *same* scanlines as Hatch, but consecutive spans are joined end-to-end into one continuous back-and-forth stroke.
- **Algorithm:**
 1. Compute per-scanline inside spans (as Hatch).
 2. Walk scanlines top $\rightarrow$ bottom, **alternating** the left-to-right / right-to-left ordering of the spans (this is the *boustrophedon* reversal).
 3. Append the next span to the current chain **iff** the connector between the previous span's end and this span's start is short ($\leq \sim 2.5\cdot s$) **and** lies inside the region. Otherwise flush the chain

and start a new one.

4. Concavities and holes naturally split the fill into a few chains where a connector would leave the region.
- **Connectivity:** near-zero lifts on convex/rectangular shapes.
 - **Orientation:** align the scanlines to the region's **long axis** (from the OBB, Ch. 2 §5) so runs are longest and reversals fewest.



Scanlines run along the long axis → longest runs, fewest reversals, one continuous stroke

Boustrophedon (the "ox-plough" pattern): the scanline reverses direction each pass and joins into one continuous stroke. Aligning to the **long axis** of the oriented bounding box (OBB) gives the longest runs and the fewest reversals.

⚠ Edge-connector caveat

Connectors run *along* the boundary, where a strict `contains()` treats the boundary itself as outside. Nudge the probe point a hair (≈ 0.25 -spacing) toward the region's **bounding-box center** (a cheap approximate centroid) before the inside-test, or convex fills fracture into many strokes. This nudge is valid for **convex-ish** regions. On a concave or holed region the bbox center may lie *outside* the material - the nudge then only makes the inside-test *more* likely to **reject** a connector, so it errs toward fragmenting the fill, **never toward burning across a gap**. Because Auto routes concave/holed shapes to Spiral (not Zigzag, §4), Zigzag normally sees only convex-ish regions; if you *force* Zigzag on a holed shape, replace the bbox-center nudge with a local inward test (sample several points along the connector, or offset along the segment's inward normal).

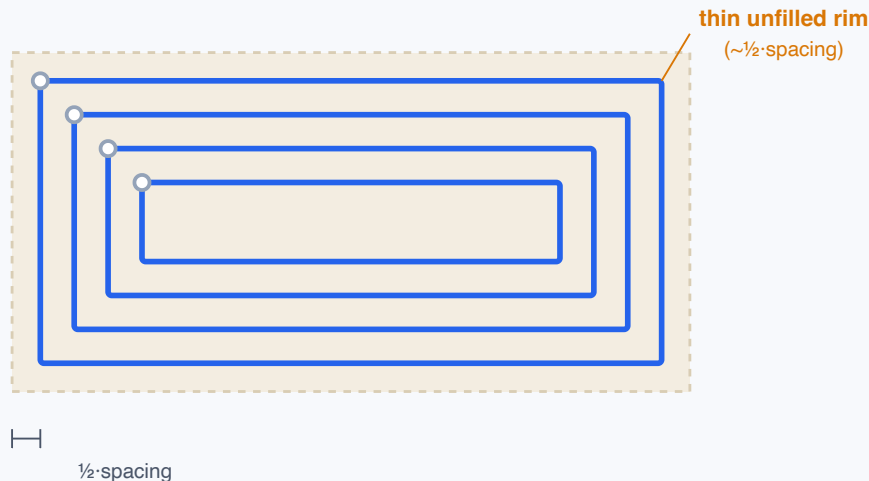
- **Use:** open convex / rectangular / thin straight regions. The general-purpose, time-optimal fill.

3.4 Contour (concentric offsets) - connected, a few lifts ★ Auto candidate

- **Idea:** fill by repeatedly inseting the boundary; each inset is a closed ring.
- **Algorithm:**
 1. Start half a spacing inside the boundary (so rings sit centred and do not retrace the exact outline).
 2. Emit the current region's boundary polylines as closed rings.
 3. Inset by `s` (Ch. 2 §7); repeat until empty.
 4. Guard with a max-ring cap.
- **Generation:** the ring set comes straight from the inward-offset primitive (Ch. 2 §7). Use the **distance-field / iso-contour** method (Method B) as the default: it produces the concentric rings for *any* topology - holes, concavity and multiple nests - in one shot (each iso-level yields every ring at that depth), which is precisely what a naive per-vertex inset cannot do. A boolean-offset library (Method A) is an exact alternative when present. This robustness is what lets the selector (§4) actually use Contour / Spiral on real artwork instead of silently degrading to a hatch look.

⚠ Edge-coverage caveat - the thin unfilled rim

Because the first ring is inset by half a spacing, the outermost drawn line sits $\sim \frac{1}{2} \cdot \text{spacing}$ inside the true boundary, leaving a thin **unfilled rim** (negligible at dense shades, visible at sparse ones). When a *filled-no-stroke* shape must have its edge fully covered, start the first ring at $\frac{1}{2} \cdot \text{lineWidth}$ inside instead, so the outer ring's outer edge lands on the boundary. (This is why "the fill reaches the boundary" is *not* a safe reason to suppress an outline - see Ch. 4 §2.)



Concentric rings inset progressively inward, the first ring starting $\frac{1}{2}$ spacing inside the boundary. Each ring is a separate closed chain $\Rightarrow$ **one lift per ring**. The inset leaves a thin unfilled rim at the outer edge.

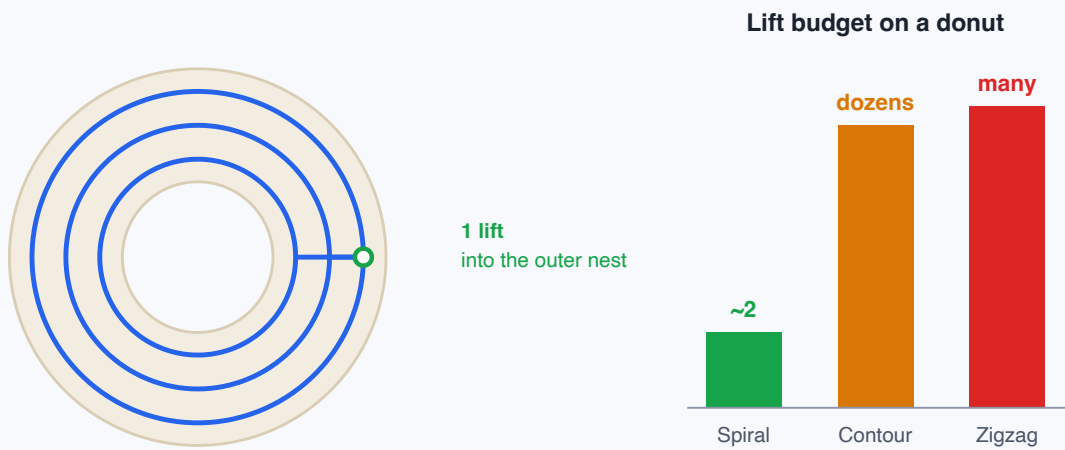
- **Connectivity:** one lift per ring (rings are separate), but the rings hug the shape so travel between them is tiny.
- **Use:** shapes with **holes**, strong **concavity**, or **hairline** thickness (a few lines wide) where following the outline reproduces the form better than straight hatching. Also the natural choice for text/logos where "echo the outline" looks intentional.
- **Keyword:** *inward polygon offsetting / contour-parallel machining.*

3.5 Spiral (connected contours) - connected ★ Auto candidate

- **Idea:** Contour, but connect the concentric rings into **as few continuous strokes as possible** - one per nested region - by re-rooting each ring to start nearest where the tool currently is.
- **Algorithm** (a greedy nearest-*ring* walk - the ordering logic of Ch. 7 §2-3 applied to the ring set):
 1. Generate contour rings (as §3.4), each as an *open* ring (drop the closing duplicate). Start the first stroke at the outermost ring.
 2. From the current stroke's end, find the **nearest remaining ring and the nearest vertex on it**; rotate that ring to start at that vertex and append it (tracing the full ring back to its entry). Re-rooting keeps the burned step between consecutive rings as short as one spacing.
 3. If that nearest jump exceeds a few spacings ($\approx 3 \cdot s$), the remaining rings belong to a **separate nest** (split off by a hole or a pinch): flush the current stroke and start a new one from the nearest remaining ring.

4. Cap the per-ring nearest-vertex scan on very large rings (sample every k-th vertex) so the connector search stays cheap.

- **Connectivity: exactly one lift per nested region** - the lift floor (§1).



Outer material band: the rings connect into ONE stroke

— Continuous Spiral stroke (rings + connectors) ◻ Entry into a nest = one lift ◻ Material

Spiral connects the contour rings with a *nearest-ring walk* and re-rooting, keeping each connector only one spacing long. The result: **one lift per nested region**. On a donut, the lift budget drops from dozens (Contour) or many (Zigzag) to just ~2 (Spiral).

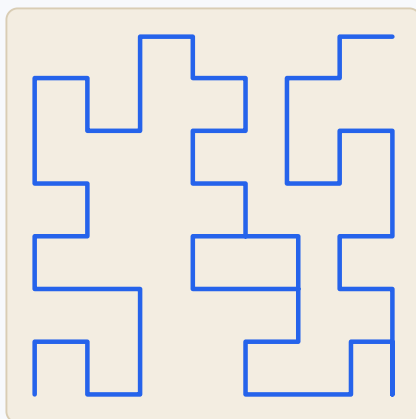
- **Use:** the broad default for filled regions with a real interior - round/convex blobs (zero reversals, it just spirals inward), **and** concave shapes, holed shapes and glyph interiors, now that the offset engine (Ch. 2 §7, Method B) produces valid rings for those too. The connectors between adjacent rings stay within the material band, so they add negligible burned length and never bridge a hole (the far-jump flush guarantees it).
- **Fallback:** if ring generation degenerates (region too thin for any ring), fall back to Contour, then to the scanline fill.

3.6 Hilbert & Peano (space-filling curves) - decorative, manual only

- **Idea:** clip a **space-filling curve** to the region; the curve visits every cell of a grid once, giving a uniform, isotropic, single-stroke fill with a distinctive woven texture.
- **Hilbert:** generate via the classic $d \rightarrow (x, y)$ bit-manipulation mapping; choose the curve order so cell size $\approx$ spacing (`order` s.t. $side / 2^{order} \approx s$), then clip the polyline to the region (Ch. 2 §6).

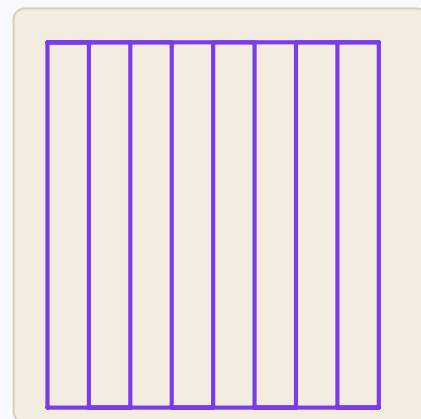
- **Peano:** generate via its **L-system** ($X \rightarrow XFYFX-F-YFXFY+F+XFYFX$, etc.; F = step, $+/-$ = 90° turn), walk the string to a polyline, scale to the bounding box, clip.
- **Connectivity:** the curve is one stroke but clipping fragments it at holes; still few lifts. Cap the order to bound the point count.
- **Use:** aesthetic/texture only; never auto-selected (a connected line fill is faster for plain coverage).
- **Keywords:** *Hilbert curve, Peano curve, L-system, space-filling curve.*

Hilbert



one stroke, isotropic woven texture

Peano

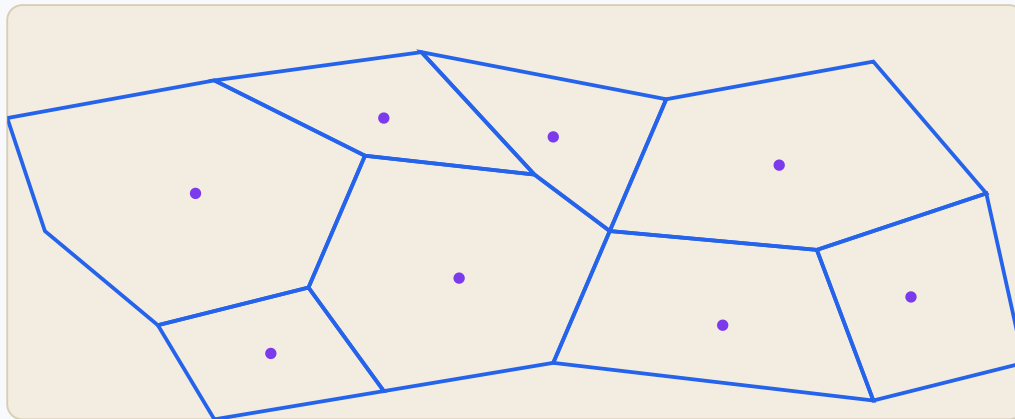


generated from an L-system, fills denser

Space-filling curves visit every grid cell exactly once, giving a distinctive woven texture. Decorative only - Auto never chooses them because a connected line fill covers faster.

3.7 Voronoi (cellular) - decorative, manual only

- **Idea:** scatter sites on a jittered grid, build each site's Voronoi cell, clip the cells to the region, draw the cell boundaries → an organic crackle pattern.
- **Algorithm:** for each site, start with a large square and clip it by the perpendicular **bisector half-plane** against every other site (Sutherland-Hodgman, Ch. 2 §6); intersect the resulting cell with the region.
- **Determinism:** derive the jitter from a hash of the grid indices so output is reproducible (no RNG). Cap the site count (cell clipping is $O(\text{sites}^2)$).
- **Use:** decorative only; never auto-selected.
- **Keywords:** *Voronoi diagram, perpendicular bisector, half-plane clipping.*



— Cell boundary (stroke) ● Site

Each cell is the intersection of the perpendicular-bisector half-planes between one site and every other, then clipped to the region. An organic crackle look – decorative only, never Auto.

4. The automatic selector ("Auto")

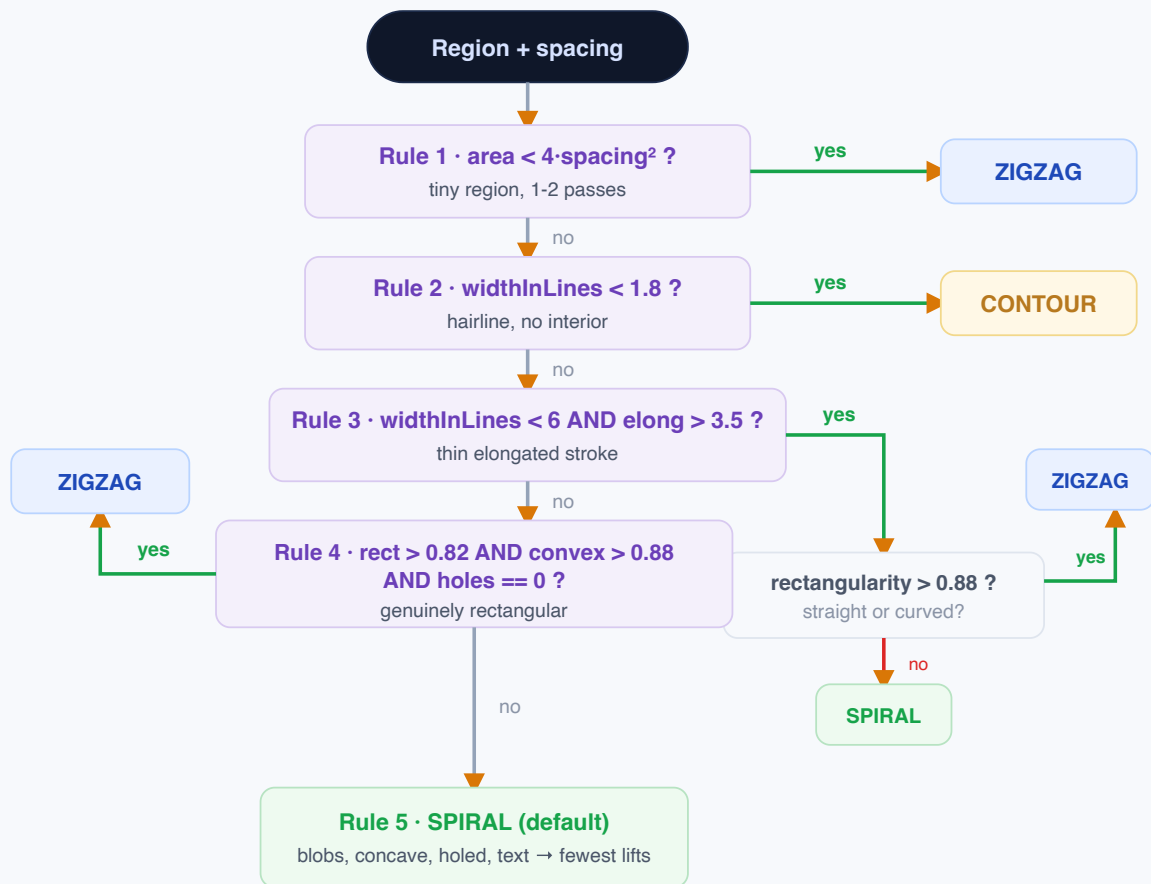
Given a region, classify it from its metrics (Ch. 2 §5) and return the **time-optimal connected** strategy. Auto **never** returns Hatch/CrossHatch/Hilbert/Peano/Voronoi – those trade time for a look and are reachable only by explicit override.

Inputs: `area`, `elongation`, `rectangularity`, `convexity`, `compactness`, `widthInLines` (`= meanWidth/spacing`), `holes`, and `spacing`.

Decision rules (first match wins)

The selector is tuned to be **lift-optimal** (§1): with a topology-robust offset (Ch. 2 §7, Method B) and a nest-aware Spiral connector (§3.5), Spiral reaches the lift floor on essentially any region with a real interior, so it is the **default**. Zigzag is kept where it is genuinely as good and more natural (rectangles / straight bars, where boustrophedon along the long axis gives the longest runs and ~0 lifts); Contour is kept for hairlines (nothing to spiral).

```
1. area < 4·spacing2                                → ZIGZAG    # tiny: 1-2
passes, any fill ≈ same
2. widthInLines < 1.8                                → CONTOUR   # hairline: no
interior, hug the outline
3. widthInLines < 6 AND elongation > 3.5:              # thin elongated
stroke
    rectangularity > 0.88 → ZIGZAG                    # straight bar:
hatch along long axis
    else → SPIRAL                                     # curved/bent
sliver: follow + connect
4. rectangularity > 0.82 AND convexity > 0.88 AND holes == 0
    → ZIGZAG    # square /
rectangle: boustrophedon
5. otherwise → SPIRAL    # blobs, concave,
holed, text: fewest lifts
```



Every rule is reachable and none is shadowed.

Rectangles are pinned to Zigzag at rule 4; everything else with a real interior reaches Spiral at rule 5.

□ Rule test
 □ Zigzag
 □ Contour
 □ Spiral

The Auto selector's decision tree: five rules, **first match wins**. Tiny region → Zigzag; hairline → Contour; thin elongated stroke branches on straightness; genuine rectangle → Zigzag; everything else → Spiral (the default, the lift floor).

Rationale per rule

Rule	Why
1	Below a few passes the strategy is irrelevant; Zigzag is the simplest correct choice.
2	A region only ~1 line wide has no interior to fill; a single contour <i>is</i> the fill, and there is nothing for a spiral to wind through.
3	A long thin shape: a straight bar (high rectangularity) hatches along its length with the longest possible runs → Zigzag; a curved/bent sliver is followed better - and connected with fewer lifts - by a Spiral of its offset.
4	A near-rectangle fills fastest with boustrophedon along the long axis: ~0 lifts and the fewest reversals (2 per scanline). A square <i>spiral</i> would turn 4× per ring for no gain, so squares/rectangles are pinned to Zigzag. <code>convexity > 0.88 AND holes == 0</code> keeps genuinely rectangular shapes here and lets concave/holed rectangles-with-bites fall through to Spiral.
5	The lift-optimal default. Round blobs (zero-reversal inward spiral), concave shapes, holed shapes and glyph interiors all connect into one stroke per nest (§3.5) - fewer lifts than Contour (one per ring) and far fewer than Zigzag fragmenting across the concavity/hole. Requires the robust offset engine; if it cannot produce rings the fill falls back (Contour → scanline, §6).

Worked examples (the rules are reachable and route as claimed)

Shape	convexity	rectangularity	holes	First rule that fires	→
Filled square	1.0	~1.0	0	rule 3 skips → rule 4 (rect>0.82, convex>0.88)	Zigzag (along long axis)
Filled disc	1.0	~0.79	0	rules 1-4 skip (rect<0.82) → rule 5	Spiral (winds inward, ~1 lift)
"L" bracket	~0.6	~0.55	0	rule 4 fails (convex<0.88) → rule 5	Spiral (one stroke, ~1 lift)
Donut (ring)	-	-	1	rule 4 fails (holes>0) → rule 5	Spiral (one stroke per band, ~2 lifts)
Letter "o"	-	-	1	rule 4 fails (holes>0) → rule 5	Spiral (~1-2 lifts)
Thin straight bar	~1.0	~0.95	0	rule 3 (elong>3.5, rect>0.88)	Zigzag (hatch along length)
Tiny dot (area < 4s ²)	-	-	-	rule 1	Zigzag (1-2 passes)

Every rule is reachable and none is shadowed: rectangles are pinned to Zigzag by rule 4, everything else with a real interior reaches Spiral at rule 5.

☆ Design evolution - why this supersedes the earlier rule set

An earlier version of this selector routed **holed / strongly-concave** shapes to *Contour* and reserved *Spiral* for round convex blobs only. That was correct **for a convex-only Spiral** - one built on a naive per-vertex inset that self-intersects on concavity and cannot connect across holes. Once the offset engine is topology-robust (Method B) and the Spiral connector is nest-aware (§3.5), that restriction disappears: Spiral produces valid rings and connects them with **one lift per nest** on *any* topology, strictly beating Contour on the very lift metric the cost model (§1) is built to minimise. The lesson worth recording: **an auto-selector's rules encode the current capability of the strategies behind them** - strengthen a strategy (here, make the offset robust) and the selector must be re-tuned to exploit it, or it leaves the win on the table. Measured effect: a donut fill dropped from dozens of lifts (Contour) to ~2 (Spiral).

i Design note

The thresholds are interpretable, ordered, and independent of any dialect. Re-tune them by changing the *rationale* (e.g. a machine with a very cheap lift can relax the preference for connected fills), not by trial and error.

5. "Architecture fill" override (shade-by-hatch)

A useful product mode: force **every non-darkest** level to a plain **Hatch** in a fixed user direction, while the **darkest** level keeps the chosen (or Auto) strategy. Because spacing already encodes the shade (§2), the lighter levels still read at the correct darkness - but as consistent parallel lines, which is the conventional way to indicate materials/tones in technical/architectural drawing. This is a dispatch-level decision layered *on top of* the selector: it swaps the strategy and the angle set, nothing else.

6. The `fill()` contract & fallback

```
fill(region, strategy, params) -> list<chain>:
  region.fillRule = EVEN_ODD
  spacing = max(floor, params.spacing)           # never zero or negative
  metrics = computeMetrics(flatten(region), spacing)
  if strategy == AUTO: strategy = chooseAuto(metrics, spacing)

  chains = dispatch(strategy)                     # §3
  if strategy == SPIRAL and chains empty: chains = contour(region, spacing)

  # SAFETY NET: never leave a real area uncovered
  if chains empty and metrics.area > spacing2:
    chains = hatch(region, metrics.longAxis, spacing)
  return chains
```

⚠ Two different area thresholds - do not unify them

Note the two area thresholds are **deliberately different**: $4 \cdot \text{spacing}^2$ is the "too tiny to bother choosing" cutoff in the selector (rule 1), while spacing^2 is the "is there any real area left to rescue?" test for the safety-net fallback. **Do not unify them.**

Invariants

- Always run on the **even-odd** interior so holes are respected.
- Always clamp spacing to a positive floor.
- Always produce output for a non-trivial area (the hatch fallback).
- Closed rings are returned **closed** (first == last) so the ordering stage can re-root them (Ch. 7).

7. Implementation checklist (for an AI generating a fill engine)

✓ Checklist

- Flatten curves with a uniform world-space tolerance (Ch. 2 §1).
- Run the engine on the **even-odd** interior; compute net area & hole count via even-odd depth (Ch. 2 §4). (Overlaps that must read solid are resolved *upstream* - non-zero/union per Ch. 2 §3 - before the region reaches the engine.)
- Compute hull → min-area OBB → the six metrics (Ch. 2 §5).
- Implement scanline span extraction once; reuse it for both Hatch and Zigzag.
- Implement inward offset once (Ch. 2 §7); reuse it for Contour and Spiral. Prefer the **distance-field / iso-contour** method so holes / concavity / multiple nests work; keep a boolean-offset fast-path if available. Never ship a bare per-vertex inset as the only offset.
- Connect Spiral rings with a **nest-aware nearest-ring walk** (§3.5): one stroke per nest, flush on a far jump - this is where the lift savings live.
- Implement the auto selector as the ordered rule list (§4), and **re-tune it whenever a strategy's capability changes** (e.g. once the offset is robust, prefer Spiral broadly). Stale rules waste a strengthened strategy.
- Keep density and strategy as *separate inputs*: density feeds the selector's spacing-relative metrics, but the reverse never holds - a strategy must not bake in its own spacing (§2). Optionally derive the shade level from **fill-colour luminance** (§2) - darker ⇒ denser - as a toggle.
- Provide the hatch fallback and per-strategy caps.
- Return world-space chains; let ordering + emission handle the rest.

✓ In summary

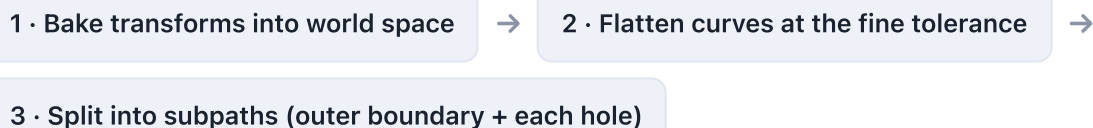
A line-drawing machine covers a region with *chains*, not by flood-filling. The cost model (§1) says the number of **lifts** dominates time, so the selector only chooses among the **connected** fills and prefers **Spiral** (the lift floor) wherever the offset can produce rings. Density (§2) encodes shade separately from the path shape. The rest - outline, text, pattern - all build on this same chain foundation, then get ordered and emitted uniformly.

Outline & Contour

The outline seems like the simplest chain source of all: just trace the shape's boundary. But "trace the boundary" hides a few decisions that are anything but trivial - when it should actually be emitted, how it interacts with fills, and the physical compensations a cutting blade needs (overcut, kerf/tool offset). All of this is independent of the target machine language.

1. What an outline is

The **outline chains** of a shape are the polylines of its boundary path after the shape has gone through three processing steps:



Specifically: (1) every transform (translate/rotate/scale/shear) is **baked** into world space (see Ch. 1 §2); (2) curves are **flattened** at the fine tolerance (Ch. 2 §1); (3) the path is split into its **subpaths** - the outer boundary plus each hole boundary - each subpath becoming one chain.

★ Rule: subpath → chain

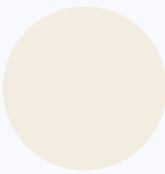
Every subpath with **2 or more points** becomes exactly one chain. A **closed** subpath becomes a closed chain (first point $\approx$ last point); an **open** subpath - an unclosed stroke, or a path imported from a file that was never closed - stays an open chain. Nothing else is required: an outline is purely the *geometry of the boundary*, independent of any fill rule or winding direction.

2. When outlines are emitted (the presence rules)

An outline is emitted purely from the shape's **geometry/style** - independent of the machine mode - when **either** of the following holds:

Condition	Reason
Shape has a stroke style	There is an explicit border line that needs to be drawn.
Shape has no fill	With nothing to fill, the outline <i>is</i> the artwork.

Conversely, a shape that is **filled but has no stroke** emits only its fill chains (Ch. 3), **not** an outline - the design specifies no border, so none is invented.

Stroke, no fill	Fill, no stroke	Both fill and stroke
		
Emits: outline (is the entire artwork)	Emits: fill chains only , no outline	Emits: fill + outline (see §2b)

Presence rules: an outline is emitted when there is a stroke, or when there is no fill (the outline is the artwork itself). Only when a shape has a fill but *no* stroke is the outline skipped.

⚠ Don't rely on "the fill already reaches the boundary"

It's true that a scanline fill does reach all the way to the boundary - but inset-based fills (contour/spiral, Ch. 3 §3.4) stop about *half a line spacing* short of the boundary and leave a thin unfilled rim. If exact edge coverage matters on a borderless filled shape, make the fill hug the boundary more tightly rather than bolting on a stray outline.

2b. Shapes with both fill and stroke - emission decisions

When a shape has **both** a fill and a stroke, it produces two chain sets over the same boundary region. The additive dispatch rule says "emit both" - but leaves three concrete decisions that the theory alone doesn't pin down:

Settings

A shape's fill and outline live on the *same* layer/tool-group, so they inherit the **same** power/speed/pass-count (Ch. 8 §7). If the outline needs different settings (darker, or cut all the way through, while the fill is only lightly marked), put it on a **separate layer**.

Order

Prefer **fill first, then outline**, so a crisp border isn't smeared by later fill passes. Greedy ordering (Ch. 7) does *not* guarantee this on its own - use an affinity/grouping key (Ch. 7 §5) to keep a shape's fill-then-outline pair travelling together.

Shared edge / double pass

The outermost lines of a scanline fill and the outline both sit close to the boundary; on a marking tool, this shared edge may get passed over twice. Usually harmless (even beneficial - a crisper edge). To avoid it, **inset the fill by $\frac{1}{2} \cdot \text{lineWidth}$** so the outline is the sole "owner" of the boundary. Note this is *not* the endpoint **welding** of Ch. 10 §3.1: welding joins fragments *within the same* chain source; it never merges a fill with a separate outline.

3. Outline vs Contour fill: don't confuse them

They share the same primitive - trace the boundary - but the two features answer two different questions, and must be kept distinct:

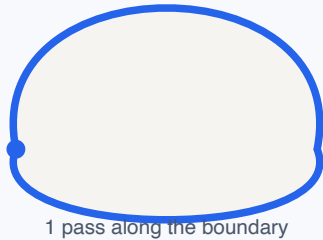
Outline (this chapter)

The boundary drawn **once**. A single pass along each subpath. Answers "draw the boundary".

Contour fill (Ch. 3 §3.4)

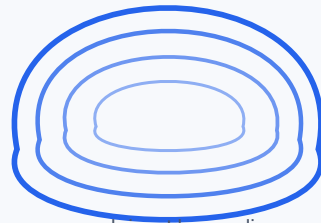
The boundary drawn **many times**, each inset inward by one line spacing, to **cover the interior area**. Answers "fill the interior by echoing the boundary".

Outline - boundary drawn ONCE



1 pass along the boundary

Contour fill - boundary drawn MANY times, insetting



N passes, each inset by one line spacing

Two features share the boundary-tracing primitive, but serve different purposes: **outline** draws the boundary exactly once to produce a border, while **contour fill** repeats the progressively inset boundary to cover the entire interior.

4. Optional tool compensations (blade/knife/kerf)

These are **optional post-processing steps** applied to already-chosen outline chains, relevant only when the tool is a physical **cutting** blade. A marking tool (pen, laser marking) needs none of these - the mark sits directly on the path. They are not a separate mode or pipeline; for a marking tool, skip them entirely.

Order of operations relative to ordering (Ch. 7)

Apply **kerf offset BEFORE ordering** - it changes the geometry that the ordering and loop re-rooting steps will operate on. Apply **overcut AFTER ordering and loop re-rooting** - overcut extends past the loop's *final* start vertex, so it must run on the oriented, re-rooted chain, re-crossing wherever the seam ended up. Reversing this order (overcut before re-rooting) places the extension mid-loop and fails to sever anything.

4.1 · Overcut

Extend a small distance past the start point of a closed loop so the blade fully severs the seam.

4.2 · Kerf / tool offset

Offset the path by half the kerf width so the cut size matches the drawn size.

4.3 · Corner sharpening

Compensate for corners rounded off because the blade trails the pivot.

4.1 Overcut (fully closing a severed loop)

A drag knife (used in vinyl/decal cutters) has its blade tip positioned **behind** its pivot axis. On a closed loop, the very start of the loop is only *scored* until the blade has rotated to face the next direction; the cut loop may not fully separate at the seam. **Overcut** fixes this:

☆ Overcut definition

After closing a loop, extend the path a small distance ($\approx 0.5\text{--}1.0\text{ mm}$) **past the start point, continuing in the direction of the first segment**, so the blade re-crosses the seam and fully severs it.



A drag-knife blade trails behind its pivot axis, so it hasn't fully cut through right at the start point. Overcut extends the path a small distance past that point, in the direction of the first segment, so the blade re-crosses the seam.

The algorithm, applied after a chain has been closed into a loop:

```
if chain.is_closed and overcut > 0:
    dir = unit(chain[1] - chain[0])
    chain.append(chain[0] + dir * overcut) // extend past the start point
```

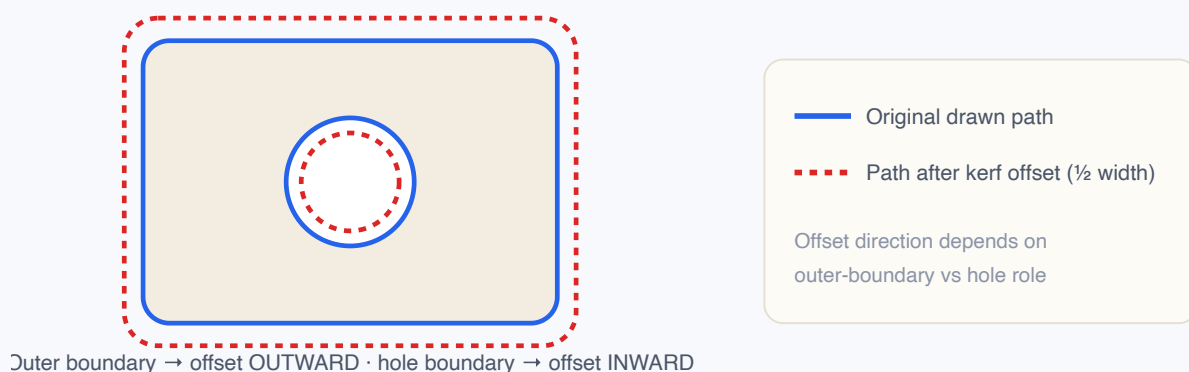
Apply only to **closed** chains and only when the tool is a **cutting** tool. For a marking tool, this extension would leave a stray tick mark - do not apply. It is a no-op for open chains or a negligible overcut value.

4.2 Kerf / tool offset

The tool removes (or marks) a certain material width. To cut the exact size drawn, offset the path by half the kerf width:

- **Offset outward** when cutting a hole to size, or an outer boundary that must keep its outer dimension.
- **Offset inward** for the opposite intent.

Uses the same **polygon offsetting** primitive as contour fill (Ch. 2 §7). Many simple cutters skip true kerf compensation and instead record the installed **blade offset** as a calibration note, sharpening corners with a small extra travel move; full kerf compensation is the general solution when dimensional accuracy really matters.



Kerf offset uses the same polygon-offsetting primitive as contour fill, but the **offset direction** must match the subpath's role: an outer boundary offsets outward to keep the part's dimension, a hole boundary offsets inward to keep the hole's dimension.

⚠ Offset direction is the easiest thing to get wrong

Offsetting a boundary in the wrong direction changes the part's dimension by a full kerf width. Tie the offset direction to whether each subpath is an outer boundary or a hole (winding/even-odd depth, Ch. 2 §4).

4.3 Corner sharpening (drag knife)

Because the blade trails the pivot, sharp corners get rounded off. Standard fix: at each corner, lift-and-swivel or add extra travel equal to the blade-offset length so the blade tip reaches the true corner before proceeding onto the next edge. This is a per-corner refinement of the same idea behind overcut.

5. Multi-pass

Thick material (or a need for a darker mark) requires several passes over the same outline. Emit the **ordered** chains, then repeat that entire set **N** times (usually lowering the tool slightly each pass, if the machine has a Z axis).

✓ Order once, repeat the result

Order the chains **exactly once**; repeat that ordered result so every pass follows *the same* travel-optimised path. Do not re-order per pass.

6. Stroke variants: dashed & variable-width

Two common stroke styles need pre-processing **before** the boundary becomes chains:

6.1 Dashed / dotted strokes

A dash array turns a continuous boundary into **many short open chains** with gaps between them. Walk along the path by arc length, emitting an "on" segment for each dash length and skipping each gap length. Each dash is an independent chain (the ordering step will travel between them). Do this **after flattening** (measuring length along the polyline).

Continuous boundary



Many short open chains



each "dash" = one independent chain, separate

A dash array turns a continuously drawn boundary into many short open chains, one per "on" segment of the dash pattern; the ordering step (Ch. 7) decides the travel order between them.

6.2 Variable-width strokes (calligraphic, pressure, tapered)

Two choices:

- **(a)** treat the stroke as a **filled region**: expand the centerline into its variable-width outline and fill it (Ch. 3); or
- **(b)** if the tool's mark width already matches, just draw the **centerline** once and ignore the width.

Choose (a) when the width variation is visible at tool scale; choose (b) when it isn't.

7. Summary: open & closed handling

Case	Outline behaviour
Closed subpath, marking tool	Trace once; no overcut.
Closed subpath, cutting tool	Trace once + overcut ; optional kerf offset.
Open subpath	Trace from one endpoint to the other; ordering may reverse it; never overcut.
Hole subpath	Emitted like any other subpath; for a cutting tool, offset in the <i>opposite</i> direction to the outer boundary.

□ "Marking tool" vs "cutting tool"

This is a distinction of **tool capability** (does the tool sever material?), **not** a UI/job mode. It only changes whether the optional compensations in §4 apply; the outline chains themselves are produced identically in both cases.

8. Checklist (when generating outline chains)

- Bake every transform, then flatten at the fine tolerance.
- Emit one chain per subpath (≥ 2 points).
- Apply the presence rules (§2) so outline and fill don't unintentionally overlap.
- Cutting tool only: overcut closed loops; consider kerf offset with the correct direction per subpath role.
- Repeat the *ordered* chains for multi-pass; don't re-order per pass.
- Return chains in world space; ordering and code emission are shared across all chain sources.

Single-Stroke (Centerline) Text

Normal vector text is a set of **filled glyph outlines**: the letter "I" is really a thin rectangle with four sides. A pen or engraving head asked to reproduce it must draw *both edges* of every stroke (and then fill the gap between them) – slow, and it looks doubled. **Single-stroke text** instead draws each glyph's **skeleton / medial axis** – the centerline of every stroke – in a single pass. This is the "engraving font" / "stick font" behaviour.

There is a shortcut: use a real single-line font

If you already have a genuine **single-line font** (Hershey / a dedicated engraving font), use its centerline strokes directly – no extraction needed. The pipeline below is for the general case where you only have filled outlines (any TrueType/OpenType font, imported vector artwork, or an arbitrary shape). This chapter describes how to derive a clean centerline from any filled glyph outline; it is font- and language-independent.

1. The problem: why single-stroke is needed

Look closely at the letter "I". On a point-to-point machine, a filled "I" is not a single straight stroke: it is a *closed loop* of four edges, and filling the interior means the machine sweeps back and forth many times. The result is three drawbacks at once: the stroke looks **twice as thick** as intended, run time **multiplies**, and at small sizes the two edges merge into a single blobby mass. Yet the geometric meaning of "I" is only *a single vertical segment*.

Filled outline (slow, doubled)



2 edges + many fill lines

Single-stroke (one skeleton axis)



1 stroke, 1 pass

— Working stroke (centerline) - - - Interior fill line □ Glyph body (ink)

The same letter "I": on the left, the filled outline forces the tool to trace **both edges** and then fill the gap; on the right, the centerline is **a single stroke** running along the medial axis of the letterform.

□ Medial axis / skeleton

The **medial axis**, also called the **skeleton**, is the set of points equidistant from the *two nearest boundaries* of a shape. For a stroke of even width, the medial axis is the "thread" running down its middle. Extracting that thread and drawing it once is the whole goal of this chapter.

2. When it applies

Single-stroke mode is activated only when both of the following are true:

- The job (or the layer) has the **"stroke text"** option enabled, **and**
- the shape is text (or any shape you choose to skeletonise).

When active, it **replaces** the shape's normal outline/fill chains: instead of tracing the glyph boundary, the system emits the extracted centerlines. This applies regardless of tool type (pen, laser, blade); other shapes are unaffected.

i Where this sits in the overall pipeline

This is a *chain source* at stage 3 (plan chains) of the overall architecture. The input is a flattened glyph outline (Ch. 2); the output is centerline chains in world coordinates (mm), ready for ordering and code emission.

3. Pipeline: raster skeletonisation

Deriving the medial axis *analytically* straight from Bézier curves is hard and brittle. The robust, universal method is **raster-based skeletonisation**: rasterise the glyph into a bitmap, thin it down to a one-pixel skeleton, then trace the skeleton into polylines. The ten steps below form the complete pipeline.

```
outline path
| 1. rasterise # fill glyphs to a bitmap
    ▼
    1-bit ink grid
| 2. thin (Zhang-Suen) → 1-pixel skeleton
| 3. prune # remove short spurs
    ▼
    clean skeleton
| 4. trace # skeleton → polylines
| 5. join # merge fragments at junctions (straightest
    continuation)
    ▼
    raw centerline polylines (pixel space)
| 6. smooth (moving average) # kill staircase wobble
| 7. simplify (Douglas-Peucker) # drop redundant points
| 8. round (Chaikin) # smooth the curve
    | 9. map back to world mm
| 10. extend terminals # reach terminals out to the glyph boundary
    ▼
    centerline chains (world mm)
```

The ten steps of the skeletonisation pipeline. Each step is a standard named algorithm; the step-by-step section below details each one.

1 · rasterise → 2 · thin → 3 · prune → 4 · trace → 5 · join → 6 · smooth →

7 · simplify



8 · round



9 · map mm



10 · extend

4. Step by step

4.1 Rasterise (fill the glyph into a bitmap)

Render the filled glyph path into a grayscale / 1-bit bitmap. Choose a resolution `res` (pixels per mm) high enough to resolve the thinnest stroke (a few px across) but capped so the bitmap does not balloon (e.g. max dimension around 2600 px, at most about 9 million pixels). Disable anti-aliasing, since we want a crisp ink/no-ink boundary; fill using the **non-zero winding** rule (so overlapping contours merge into solid ink); threshold to `ink = 1` / `background = 0`. Add a small padding border so edge pixels also have neighbours.

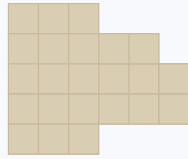
4.2 Thin - Zhang–Suen

The classic **Zhang–Suen thinning** algorithm iteratively removes boundary pixels that are *not essential* to connectivity, until only a **1-pixel-wide skeleton** remains. Each iteration has two complementary sub-passes with different deletion masks; it uses:

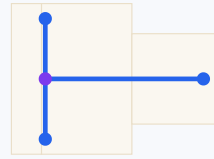
- **B(p)** = the number of 8-neighbours that are ink (must lie in the range **2-6** to be deleted),
- **A(p)** = the number of 0→1 transitions going around the 8-ring (must be **exactly 1**),
- two corner-condition products, which differ between the two sub-passes.

Repeat until no pixel changes (guarded by a max-iteration cap). Keywords: *Zhang–Suen thinning* / *morphological skeletonisation*.

Thick ink region



1-pixel skeleton



thin



■ Ink pixel — Skeleton (centerline) ● Junction

Zhang–Suen progressively removes non-essential boundary pixels, turning a thick ink region into a skeleton exactly one pixel wide while preserving connectivity. The purple point is a **junction** (a branch point).

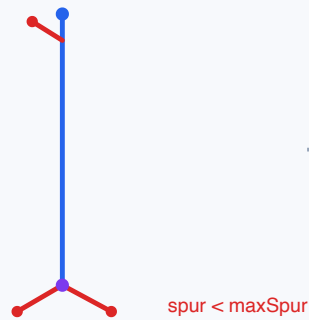
4.3 Prune spurs

Thinning spawns short dead-end "twigs" at serifs, stroke terminals, and junctions, making the centerline look hairy. **Prune** each branch that runs from an **endpoint** (1 neighbour) into a **junction** (>2 neighbours) within `maxSpur` pixels: delete the twig, keep the junction. Set `maxSpur` a little above a typical stroke half-width (about **0.7 mm** converted to pixels) so real strokes (much longer) survive while noise artefacts vanish. Run a few passes.

⚠ Do not prune an entire stroke

Never prune a branch that runs between *two genuine endpoints*: that is a whole stroke (like the letter "I"), not a spur. Only a branch with one endpoint end and one junction end, short enough, is a pruning candidate.

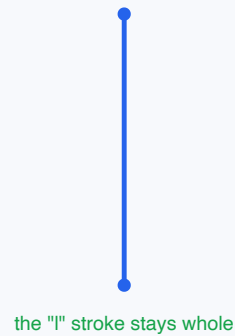
Before prune (hairy)



prune



After prune (clean)



— Real stroke (kept) — Spur (removed) ● Junction

The red twigs at the serifs are shorter than `maxSpur` so they are removed; the long vertical stroke of the letter "I" runs between two genuine endpoints, so it is kept intact.

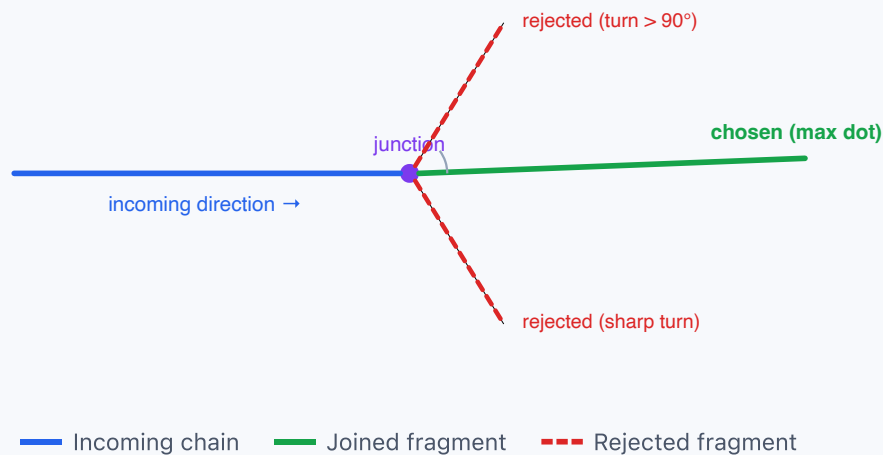
4.4 Trace (skeleton → polylines)

Walk along the 1-pixel skeleton to trace out polylines. Start from every **node** (endpoint or junction) and follow the runs with exactly 2 neighbours, marking each undirected edge used at most once; then mop up the remaining **pure loops** (rings with no node, like the letter "O"). Result: many short polylines in pixel space.

4.5 Join at junctions (straightest continuation)

The raw trace is fragmented (every junction breaks the chain). Rejoin fragments meeting at a shared endpoint using **straightest continuation**: at a junction, extend the chain into whichever fragment's direction best matches the incoming direction (maximum dot product, requiring a turn of $< 90^\circ$). This turns a scribble of disjoint fragments into a few flowing strokes, giving *far fewer pen lifts* and smooth runs through junctions.

At a junction: pick the best-aligned fragment



A chain arriving at a junction joins the fragment whose direction matches best (maximum dot product, turn < 90°). Sharply turning fragments are left for a different chain, so the stroke flows smoothly through the junction.

4.6 Smooth - moving average

The raw skeleton has a ± 1 px "staircase" wobble. Apply a low-pass **moving average** filter (e.g. convolution with the `[1,2,1]/4` kernel) a few times. Pin the endpoints of open polylines (don't let them move); for closed loops, wrap around. Keywords: *Laplacian smoothing* / *box smoothing*.

4.7 Simplify - Douglas–Peucker

Reduce the point count with the **Ramer–Douglas–Peucker** algorithm at a small tolerance (a finer tolerance keeps rounded curves more faithfully). It removes collinear / near-collinear vertices without changing the shape.

4.8 Round - Chaikin

Apply **Chaikin corner-cutting** (1-2 iterations) to turn the simplified polyline into a smooth curve. Each iteration replaces every corner with two points at 1/4 and 3/4 along its edges. Keyword: *Chaikin subdivision*. See also curve continuity in Ch. 10.

4.9 Map to world space (back to mm)

Convert pixel coordinates back to world mm using the inverse of the rasterise transform (accounting for padding and `res`).

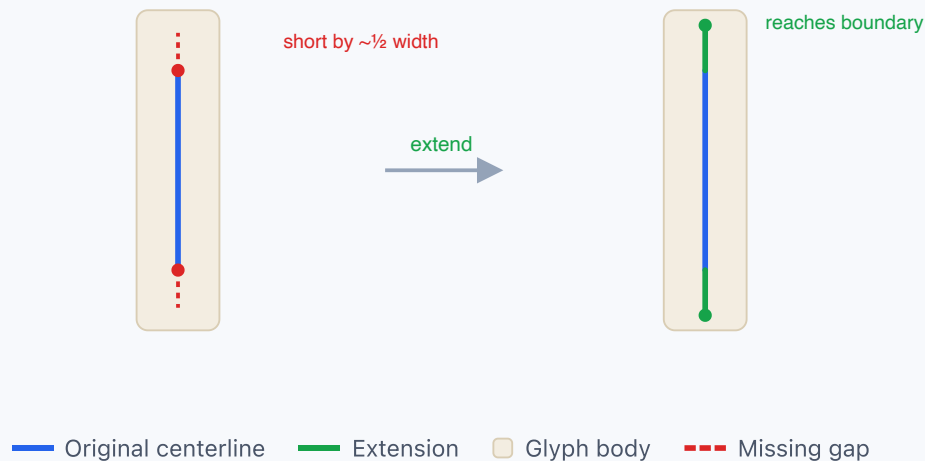
4.10 Extend terminals

The medial axis stops *about half a stroke-width short* of a stroke's visual endpoint. This is the "missing stroke tip" problem. The fix: extend every endpoint that is a genuine **terminal** outward along its tangent until it reaches the glyph boundary:

- A **terminal** is an endpoint that *no other* stroke touches within the **share tolerance** (about **0.35 mm**, see §5). An endpoint that lies within that tolerance of another polyline is a **junction**: leave it alone (extending would overshoot into the glyph). Closed loops have no terminals.
- Estimate the tangent from a short back-off along the stroke, then step outward in small increments as long as the point remains inside the filled glyph, up to a maximum extension.

Before: missing tip

After: extended to boundary



The skeleton axis stops about half a stroke-width short (the **missing stroke tip** problem). We extend each *true terminal* along its tangent out to the glyph boundary. At a junction we do **not** extend, since that endpoint is already shared with another stroke.

⚠ Robustness warning: choose the share tolerance carefully

Keep the share tolerance *well below* the narrowest gap between distinct strokes. Otherwise, glyphs (or serifs) that touch or overlap will cause a true terminal to be misclassified as a junction (producing a **missing tip**) or vice versa (producing an overshoot spike). For glyphs that touch themselves, the max-extension cap (§5) bounds how far a mis-stepped extension can run along an unintended interior path.

5. Cleanup thresholds (with rationale)

Every tunable parameter has a reason behind it; understanding the reason is what lets you re-tune it intelligently.

Parameter	Typical value	Rationale
<code>res</code> (px/mm)	chosen so the thinnest stroke is $\geq \sim 4\text{-}8\text{ px}$ (2 is only a hard floor), capped by max bitmap size	too-coarse raster loses thin strokes; too-fine blows up the bitmap
<code>maxSpur</code>	$\sim 0.7\text{ mm}$	just over a stroke half-width: kills serifs, keeps strokes
min stroke length	$\sim 0.6\text{ mm}$	drops stray nubs / tiny loops
DP tolerance	$\sim 0.045 \cdot \text{res px}$ ($\geq 0.7\text{ px}$)	small $\Rightarrow$ curves stay round
Chaikin iterations	2	smooth without over-shrinking
terminal/junction share tol	$\sim 0.35\text{ mm}$	endpoints closer than this = a shared junction (don't extend); farther = a free terminal
terminal max extension	$\sim 2.2\text{ mm}$	enough to reach the boundary without overshooting

Resolution for small type

`res` is chosen *per glyph* so the thinnest stroke has enough pixels. Small type (cap height $\leq 4\text{-}5\text{ mm}$) needs a **high** `res` (tens of px/mm), or is better served by a real single-line font (§1). Smoothness measured in mm is preserved by DP + Chaikin regardless of `res`; what a too-coarse raster loses is **fidelity** (whether the centerline actually matches the glyph).

6. Why this order

Thin first (topology), then prune (remove noise), then trace (extract), then join (connect), *then* apply geometric smoothing (appearance), and extend terminals as the **very last step** (because it needs

the finished tangents and the "is this a shared junction?" relationship between completed polylines). Reordering breaks these assumptions: for example, smoothing *before* pruning would blur the very spurs you want to detect and cut.

☆ **Principle: topology first, geometry second, relationships last**

Three layers of work must be ordered correctly. The **topology** layer (thin, prune, trace, join) decides "how many strokes and how they connect"; the **geometry** layer (smooth, simplify, round) decides "how smooth they look"; the **relationship** layer (extend terminals) needs every stroke finished before it can tell terminal from junction.

7. Checklist (AI generating centerline text)

1. Rasterise the filled glyph (non-zero fill, no AA, padded, bounded size).
2. Zhang–Suen thin down to a 1-px skeleton.
3. Prune spurs into junctions (a few passes).
4. Trace nodes → polylines; mop up remaining loops; use each edge once.
5. Join fragments by straightest continuation (turn < 90°).
6. Smooth (moving average) → Douglas–Peucker → Chaikin.
7. Map back to world mm.
8. Extend *true terminals* to the boundary; leave junctions alone.
9. Return the chains in world space; they replace the glyph's outline/fill.

✓ **Result: elegant text in a single stroke**

Done correctly through all ten steps, any filled glyph becomes a handful of smoothly flowing centerline chains, each stroke traced once, tips landing right at the boundary. The output feeds into the same downstream flow as every other source: minimise travel in Ch. 7, then emit code in Ch. 8. The time cost drops sharply because the fill scan lines disappear entirely, exactly as the cost model in Ch. 11 predicts.

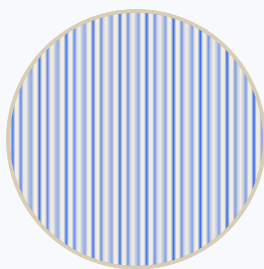
Pattern, Texture & Shade Fill

Beyond a plain solid fill (Ch. 3), a shape may be filled with a **decorative pattern** (lines, grid, weave, brick, dots, wave, ...) that should be reproduced *literally* on the machine, or shaded with a **tone/level** that maps to fill density. This chapter covers both, and how they interact with the solid-fill engine.

1. Three kinds of "fill" - pick the right path

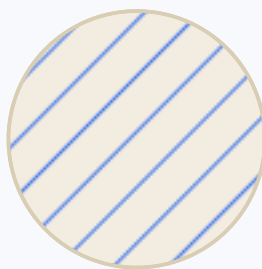
When a shape has a fill attribute, the first step is to **classify** it before generating chains:

Fill kind	Detection	Chain source
Solid	Pattern is empty / "solid" sentinel	Fill engine - Auto strategy (Ch. 3)
Simple hatch look	Pattern is a line/grid family (line-h, line-v, diagonal, grid, cross)	Hatch at the pattern's angle set
Decorative texture	Pattern has real repeating stroke geometry (weave, brick, wave, circles...)	Pattern tiling & clip (this chapter)



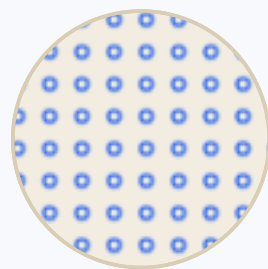
Solid (Auto)

fill engine, fastest path



Simple hatch look

disjoint parallel lines



Decorative texture

real repeating stroke geometry (e.g. dots)

All three fill kinds look alike in that they "cover" a shape, but their **underlying geometry** is completely different: solid is a closed chain optimised for speed, hatch is disjoint parallel lines, and texture is a repeated tile reproduced as-is.

Why classify first

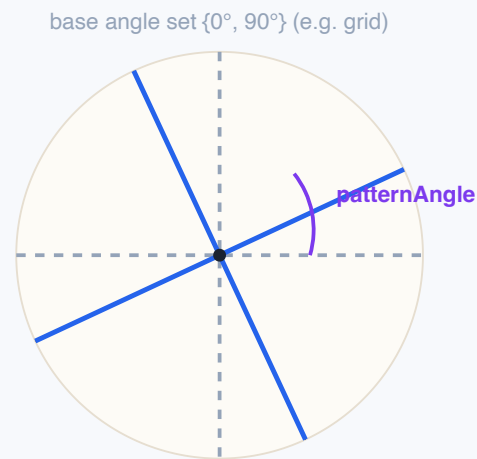
A solid fill should be drawn the **fastest** way (Auto), while a decorative texture must be reproduced *exactly as drawn on screen* - its geometry *is* its identity, so it cannot be substituted with a faster strategy that produces a different shape.

2. Simple line/grid patterns → hatch angle sets

The line/grid pattern family maps to a small set of **hatch angles**; generate one Hatch pass (Ch. 3 §3.1) per angle:

Pattern	Angle set (deg)
horizontal line / dash-h	{0}
vertical line / dash-v	{90}
forward diagonal	{45}
back diagonal	{-45}
grid / plus / lattice / plaid / weave	{0, 90}
diagonal grid / cross	{45, -45}
(anything else)	{45}

Then add the shape's **pattern rotation** (`angle + patternAngle`) to every angle in the set, so the whole set rotates together with the user's setting. Spacing between lines comes from the job's hatch interval (or the shade mapping, §4). These are *deliberately disjoint* hatch lines - the look depends on those parallel lines existing exactly as such, never rejoined.



$$\text{actual angle} = \text{base angle} + \text{patternAngle}$$

The pattern's base angle set (dashed grey) has `patternAngle` added to produce the actual angle set (solid blue) - the whole set rotates *together* as one rigid body.

3. Decorative texture: tile & clip

A texture is defined once as **tile geometry**: a set of stroke polylines inside a repeating rectangular cell (`tileW × tileH` , in the texture's own pixel space). To fill a shape with it, we must reproduce *exactly* the brush as seen on screen:

□ Tile

The smallest geometric unit of a texture, containing one or more stroke polylines, repeated regularly on a `tileW × tileH` grid to cover any area - exactly the way an editor renders a repeating brush on screen.

```

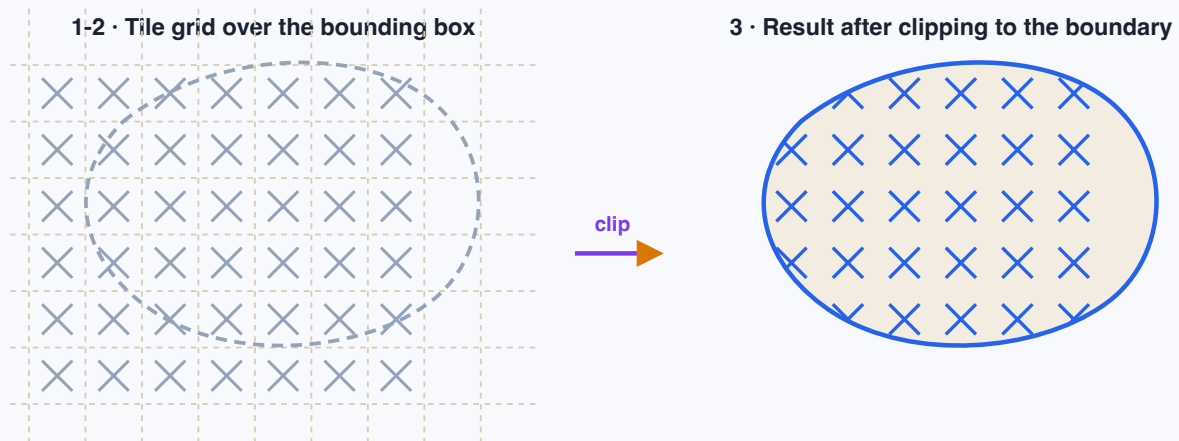
// Step 1: compute the tile → shape transform
scale = texBase × patternScale / shapeScale // match on-screen brush size
rotate = patternAngle
// (texBase is the fixed base brush scale; shapeScale removes the shape's own
// scaling so the texture size stays stable regardless of shape size)

// Step 2: map the shape's bounding box back into tile-pixel space (inverse
// transform)
for i in [i0..i1], j in [j0..j1]: // tile indices that can overlap the shape
    if tileCount is implausibly large: bailOut() // fallback: caller reverts to a
    plain fill

// Step 3: for each tile (i, j), for each stroke polyline in the tile
for each stroke in tile:
    stroke' = translate(stroke, i·tileW, j·tileH)
    stroke' = applyTransform(stroke', scale, rotate)
    runs = clipToShapeInterior(stroke') // segment-polygon clip, see Ch.2 §6
    append(chains, runs)

```

Result: the texture's strokes are tiled across the shape and clipped to its exact boundary - pixel-for-pixel the same as the rendered brush, so the preview and the machine output always agree. Work in the shape's **local** space, then map local→world→machine at emission.



Left: the tile grid (dashed) containing sample strokes (X marks) covers the whole bounding box, extending past the shape's boundary. Right: after **clipping** to the boundary (segment-polygon clip), only the strokes falling inside the shape are kept and become chains.

⚠ Critical detail: the scale must match the on-screen brush

The tile→shape transform **must** use the exact on-screen brush scale: the same `texBase × patternScale / shapeScale` formula. If this scale drifts, the generated machine code will *not match* the design - the texture will come out too big or too small compared to what the user approved on screen.

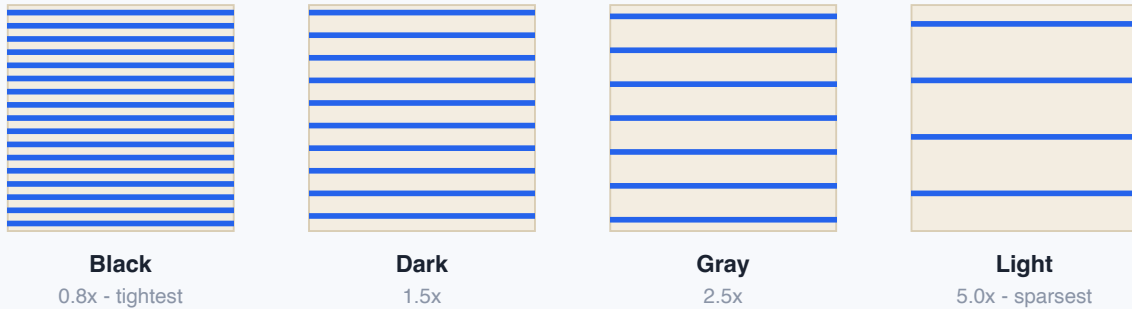
4. Shade / tone via fill density

Independently of *which kind* of pattern is used, a shape can also carry a **shade level**. As established in Ch. 3 §2:

```
spacing = shadeFactor(level) × lineWidth
```

The darker the level, the smaller the factor, the tighter the lines, and the darker the result. A representative 4-level map:

Level	shadeFactor (× line width)
Black	0.8
Dark	1.5
Gray	2.5
Light	5.0



All four cells use the **same strategy** (Auto's horizontal hatch) - only **spacing** changes with **shadeFactor**. The stroke shape stays identical, only the spacing changes, and the eye reads four different "gray levels."

☆ Two independent axes, never conflate them

Pattern / strategy decides the *shape* of the fill lines. **Shade / density** decides the *spacing* between them. So "light gray solid via Auto" and "black solid via Auto" use the *same strategy*, only different spacing. "Architecture fill" (Ch. 3 §5) is a special case: it fixes the *shape* to a plain hatch at lighter levels, leaving density alone to carry the shade.

5. Colour / monochrome note

For single-tool machines, the output is inherently **monochrome**: the tool either marks or it doesn't. On-screen colour (from a layer/theme ink) is purely a **display** concern; the generated toolpath carries no colour. Shade is expressed by **density**, not by any colour value.

ⓘ shade == density

Keep colour **entirely outside** the geometry. It only ever affects the preview rendering, never the generated chain of points.

6. Interaction with the solid-fill engine

The dispatch (Ch. 1 §4) tries the following branches in order, for any shape that **has a fill**:

```

if pattern is decorative_texture and tiling.producedStrokes():
    use pattern chains                                // §3 - tile & clip
elif pattern is solid:
    use fill-engine chains, Auto or override          // Ch.3
else: // simple hatch look, OR a texture that tiled empty
    use hatch chains at the pattern's angle set      // §2

```

The middle branch is where the **smart Auto fill** operates; it applies only to plain solid fills. Decorative textures deliberately bypass Auto (their appearance is the whole point). The final branch also acts as the **safety fallback** if a texture's tiling hits its cell-count cap and produces no strokes.

7. Checklist (when generating code for pattern / shade fills)

1. Classify the fill: solid, simple hatch look, or decorative texture.
2. Solid → fill engine (Auto/override); density from the shade level.
3. Simple hatch → angle set + pattern rotation; density from hatch interval/shade level.
4. Texture → tile with the exact on-screen brush transform, clip to the shape, cap the tile count, fall back to hatch if empty.
5. Keep pattern-shape and shade-density as two independent axes.
6. Keep colour out of the geometry; shade == density.

✓ In short

Classify correctly before generating chains, honour the original geometry of a decorative texture, and keep the "shape" axis separate from the "density" axis - these three principles are enough to make every pattern and shade level come out of the machine exactly as designed on screen.

Toolpath Ordering

After every chain source has run (Ch. 3 - Ch. 6), you hold an **unordered set of chains**. Drawn in arbitrary order, the tool wastes most of its time on pointless travel between distant chains. Ordering finds a good visiting sequence *and* the best orientation for each chain - purely as geometry, with no dialect involved.

1. The problem

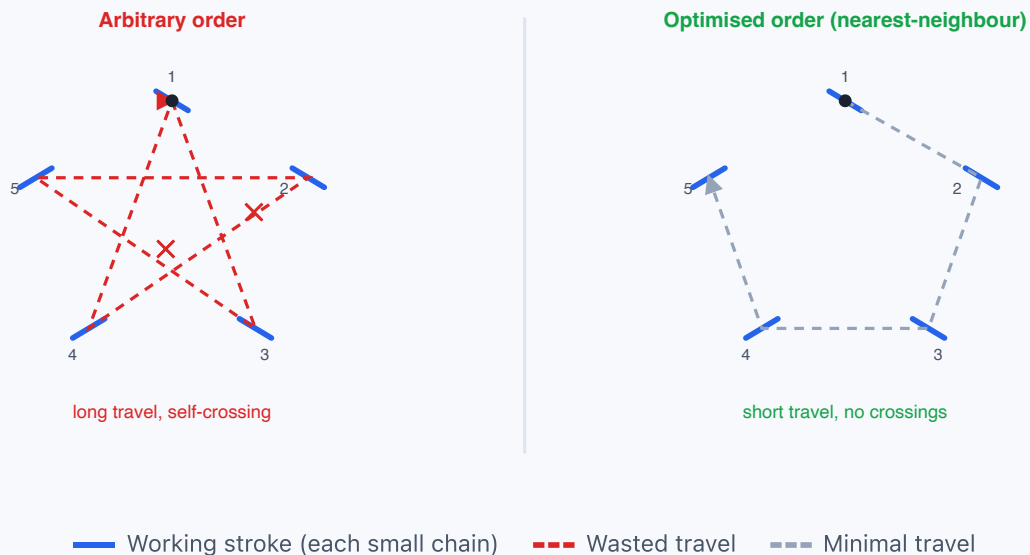
Given chains $C_1 \dots C_n$, each with two ends (or, for a closed loop, any vertex can be the starting point), and a start position, we need to choose:

1. a **permutation** (visiting order), and
2. for each chain, a **direction** (which end to enter from; where to start a loop),

that minimises the total **pen-up travel** (the distance between consecutive chains).

□ Travelling Salesman Problem (TSP)

This is a variant of the **Travelling Salesman Problem** (TSP), with one twist: each "city" is not a point but an edge/loop with *multiple possible entry points*. Exact TSP is NP-hard, so we use a fast **greedy heuristic** - the travel savings from *any* reasonable ordering already dwarf the gap between the heuristic and the true optimum.



The same set of 5 chains, only the visiting order differs. The arbitrary order (left) connects them in a star pattern, crossing itself repeatedly; the nearest-neighbour order (right) traces the perimeter, with no overlapping strokes. No geometry changed - only the sequence did.

2. Greedy nearest-neighbour (the workhorse)

The workhorse algorithm, simple and cheap, is the **greedy nearest-neighbour heuristic**: always move to the unplaced chain whose end is nearest to the current position.

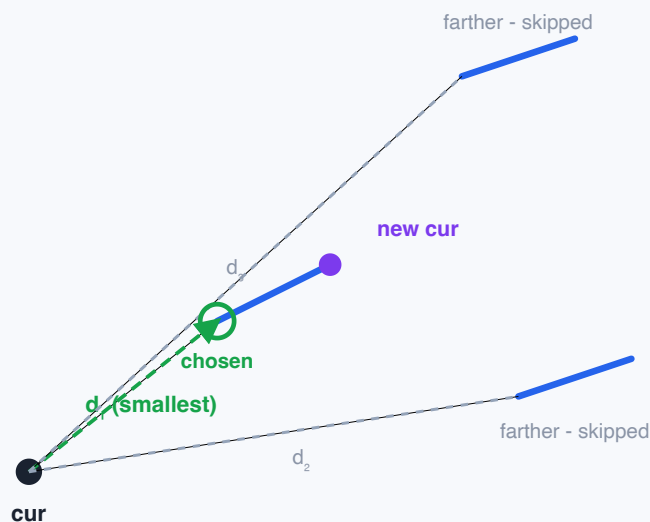
```

cur = startPoint
while unplaced chains remain:
    // pick the unplaced chain whose NEAREST end to "cur" is closest
    pick chain c minimising min(|cur - c.first|, |cur - c.last|)
    orient c to start at its nearer end    // see §3
    append c to the result; cur = c.last  // the new end after passing through c
  
```

- Complexity $O(n^2)$ (each step scans all remaining chains). Fine up to a few thousand chains.
- Uses **squared** distance for comparisons (no `sqrt` needed).
- Deterministic and good enough in practice: nearest-neighbour usually lands within a small constant factor of the optimum (the theoretical worst case is much looser, but real toolpaths are not adversarial) - moreover the fill sources already emit chains in a locally-coherent order (scanline / ring order), so the greedy pass is mostly just *stitching and flipping* them.

★ Keyword

greedy nearest-neighbour TSP heuristic



From `cur`, three candidate chains have their nearest end at distances `d1`, `d2`, `d3`. The algorithm picks the chain with the smallest `d1`, orients it to start at the nearer end, then updates `cur` to that chain's other end.

3. Chain orientation & loop re-rooting

Choosing *which end to enter from* matters just as much as the visiting order:

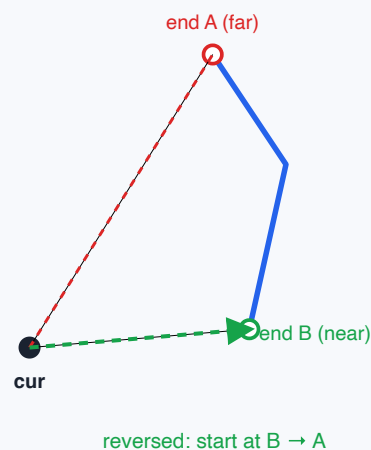
- **Open chain:** reverse it *if and only if* its *last* point is nearer to `cur` than its first point. (Only two choices; pick the nearer entry.)
- **Closed loop:** a loop can start at **any vertex**. Re-root it to begin (and end) at the vertex **nearest** `cur`, which nearly eliminates the travel needed to reach that loop:

```
// re-root a closed loop to the vertex nearest "cur"
drop the duplicate closing vertex (last point = first point)
find the index of the vertex nearest "cur"
rotate the ring so that vertex is first
re-append the first vertex to re-close the loop
```

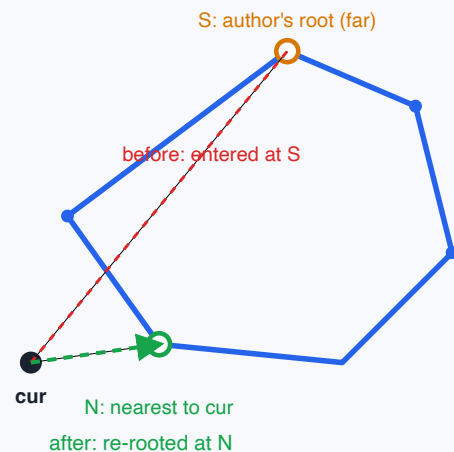
i A big win, at almost no cost

Re-rooting closed loops is a large improvement for near-zero cost. Without it, every loop is entered at whatever vertex its author happened to set as the starting point - which could be on the far side from `cur`.

Open chain: reversal



Closed loop: re-rooting



Left: an open chain has two possible entries; end B is chosen because it is nearer to `cur`, so the chain is reversed. Right: a closed loop can be entered at *any* vertex; re-rooting moves the entry point from **S** (the author's original root, far away) to **N** (the vertex nearest `cur`).

4. Scaling up: when n is too large

Dense fills (fine line spacing over a large area) can produce tens of thousands of tiny chains.

Beyond a cutoff (e.g. around ~4000 chains), the $O(n^2)$ pass becomes too slow. There are two safe fallbacks:

Method	Complexity	When to use
Greedy nearest-neighbour	$O(n^2)$	Below the cutoff (~4000 chains) - the best quality for an acceptable cost.
Orientation-only	$O(n)$	Above the cutoff, when the source already emits in a coherent scanline/ring order - keep the source order, only flip/re-root each chain relative to the running position.
Spatial bucketing (grid/Hilbert)	$O(n \log n)$	Above the cutoff, when the source is <i>not</i> already locally sorted - sort chain midpoints by grid or Hilbert curve, then greedily order within each bucket.

The first option (*trusting the source's locality*) leverages the fact that fill sources already emit chains in a scanline / ring order that is nearly travel-optimal. Skip the reordering step and apply only per-chain **orientation** (still flipping/re-rooting relative to the running position); this preserves *boustrophedon* ("ox-plough" back-and-forth) locality at $O(n)$ cost.

⚠ Always set a cap

Always cap the expensive pass; never let ordering itself become the bottleneck of the whole pipeline. This ties directly into the system's overall performance budget - see also Ch. 11 · Performance & Complexity.

5. Per-group ordering

Order **within** a coherent group, not across the whole document, because groups (layers/tools) carry different machine settings and must be emitted contiguously:

```
for each layer/tool-group in output order:
    collect that group's chains
    ordered = orderChains(chains, currentPosition)
    emit ordered    // carrying this group's machine settings
    currentPosition = last emitted point    // carried into the next group
```

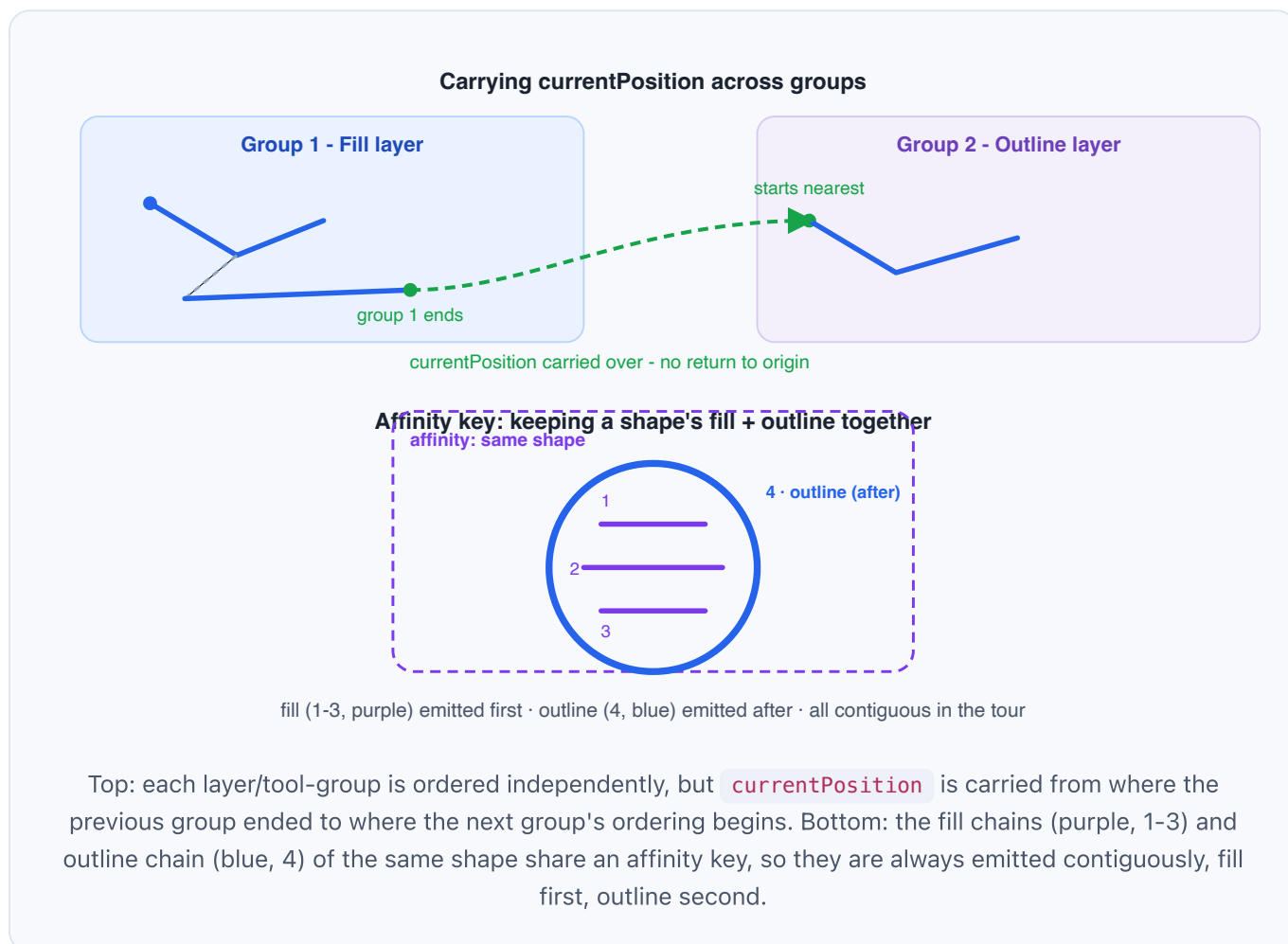
Carrying `currentPosition` across groups means the first chain of the next group is chosen *relative to where the previous group just ended* - with no wasted return-to-origin trip.

Optional: intra-group affinity - keeping compound paths together

Greedy ordering treats every chain as an independent "city", which can scatter the subpaths of one **compound path** or one **glyph** (outer boundary + its holes) across the whole tour. If you need them to stay contiguous (e.g. cut a glyph's counter immediately after its outline), tag chains with an optional **affinity key**, then order *groups of chains sharing a key* as a unit: order within the affinity group first, then treat the whole group as a single "super-city" in the outer nearest-neighbour pass. Skip this technique if raw travel-minimisation is the only goal.

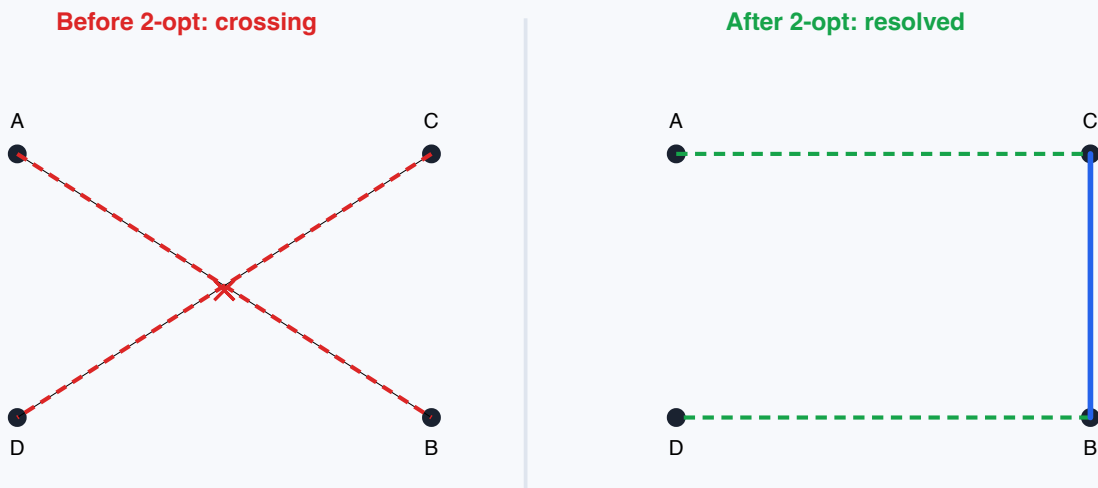
Heterogeneous sources within one layer

A layer often mixes a shape's **fill** chains, its **outline** chains, and nearby **single-stroke text**. Pure nearest-neighbour will happily interleave them (drawing text in between a fill region and its outline). For a job at a single power level, this only affects travel and is therefore harmless - but when *order affects quality* (fill **before** outline so the edge stays crisp; see Ch. 4 §2b), give the fill and outline of the same shape a **shared affinity key** and emit fill-then-outline within that group. Ordering by itself gives no such guarantee - the affinity key is the mechanism that enforces it.



6. Optional refinement (2-opt)

If output quality matters more than generation speed, run a few extra **2-opt** passes over the greedy tour: repeatedly reverse a sub-sequence of the tour if doing so shortens the total travel. Cheap, and it removes the obvious "self-crossing" moves that the greedy algorithm leaves behind. Rarely needed for drawing/engraving, but a standard technique when you want a tighter tour.



The tour $A \rightarrow B \rightarrow C \rightarrow D$ (left) has two travel moves crossing each other. Reversing the sub-sequence B..C into $A \rightarrow C \rightarrow B \rightarrow D$ (right) keeps the same set of points but removes the crossing and shortens total travel.

7. What ordering must NOT do

⚠ Boundaries of responsibility

- It must not **merge or split** chains (that changes the artwork).
- It must not **emit any command** (that is the emitter's job).
- It must not **reorder across a group boundary** (that breaks per-group machine settings).
- It must not **change a chain's point sequence** except by reversal / loop re-rooting (an angle-preserving relabelling).

In other words, ordering is a **purely geometric** stage: the input is a set of chains, the output is *the same set of chains*, only their order and (possibly) their direction differ. The next stage - code

emission (Ch. 8) – is the only place that knows about a specific machine dialect.

8. Checklist (for AI generating the ordering stage)

✓ Checklist

- Greedy nearest-neighbour on minimum endpoint distance, using squared distances.
- Orient open chains by their nearer end; re-root closed loops to the nearest vertex.
- Cap the $O(n^2)$ pass; above the cap, fall back to orientation-only ($O(n)$).
- Order per layer/tool-group; carry the running position across groups.
- Keep it pure geometry: the output is an ordered chain list, nothing is emitted.

With a well-ordered set of chains in hand, all that remains is to translate them – along with the modal tool state and coordinate formatting – into a concrete machine language. That is the subject of the next chapter. For pathological geometric cases (empty chains, single points, duplicates) that can affect both ordering and emission, see Ch. 12.

Machine-Code Emission

This is the **only** stage that knows about a specific machine language. It takes the ordered chain list (Chapter 7) and serialises it into a concrete command language – GRBL G-code, HPGL, Marlin, and so on. Everything upstream is entirely machine-neutral; everything machine-specific lives here, and only here.

1. The emitter interface

Model the emitter as a small set of operations. Each dialect (concrete machine language) implements every operation; the driver loop calls them uniformly, with no idea which machine it is talking to:

```
interface Emitter:
    preamble()                # units, absolute mode, tool init, return home
    travelTo(point)           # move while the tool is DISENGAGED (rapid)
    engage([power])           # pen down / laser on / blade down
    workTo(point[, power])    # move while the tool is ENGAGED
    disengage()               # pen up / laser off / blade up
    setFeed(rate)             # working speed (if the dialect has it)
    comment(text)             # human annotation (if the dialect supports
it)
    postamble()               # disengage tool, return home, end
    format(point) -> string    # world->machine transform + unit scaling
```

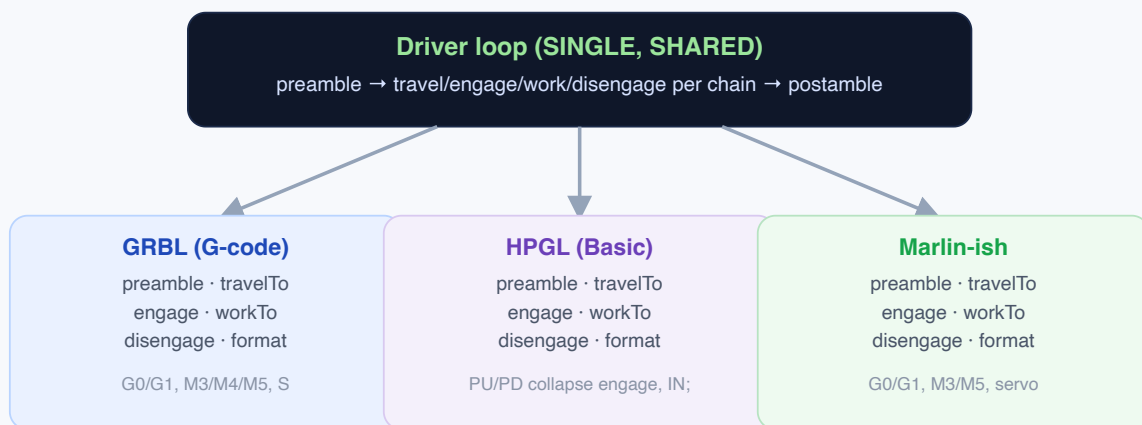
The driver loop is **identical** for every dialect:

```

preamble()
cur = home
for each group:
    setGroupParams(power, feed, force, passes)
    for pass in 1..passes:
        for chain in orderedChains(group):
            if dist(cur, chain.first) > ε:
                disengage(); travelTo(chain.first)
            engage(power)
            for p in chain[1:]: workTo(p[, power])
            cur = chain.last
postamble()

```

Only `preamble` / `travelTo` / `engage` / `workTo` / `disengage` / `format` differ between dialects. Everything else - modal state, group ordering, pass count - only needs to be written once.



setFeed & comment: same meaning in every dialect - only the syntax differs

The driver loop is written once, ever. Each dialect only needs to plug in six functions: `preamble` , `travelTo` , `engage` , `workTo` , `disengage` , `format` .

2. The dialect mapping table

For three representative dialects, here is how each abstract operation translates to a concrete command:

Operation	GRBL (G-code)	Basic HPGL	Marlin-ish
units / init	<code>G21 G90</code>	<code>IN;</code>	<code>G21 G90</code>
travel (disengaged)	<code>G0 X.. Y..</code>	<code>PU x,y;</code>	<code>G0 X.. Y..</code>
engage	<code>M3 S..</code> (vector) / <code>M4 S..</code> (raster)	- (no separate command)	<code>M3</code> / servo down
work (engaged)	<code>G1 X.. Y.. [F..]</code>	<code>PD x,y;</code>	<code>G1 X.. Y..</code>
disengage	<code>M5</code>	<code>PU;</code>	<code>M5</code> / servo up
power/force	<code>S</code> word, <code>0 .. \$30</code>	<code>FS</code> (force), <code>VS</code> (speed)	<code>S</code> / servo angle
return home	<code>G0 X0 Y0</code>	<code>PU 0,0;</code>	<code>G0 X0 Y0</code>
comment	<code>; text</code>	(none - see §6)	<code>; text</code>

□ HPGL has no distinct "engage" command

In HPGL, pen-down *is* the working move: `PD x,y;` lowers the pen (if raised) **and** draws to `x,y`; `PU x,y;` raises the pen **and** travels there. So the abstract `engage` + `workTo` collapse into a single `PD` command - and, symmetrically, `disengage` + `travelTo` collapse into a single `PU` command. In the driver loop, model HPGL's `engage` / `disengage` as **state-only no-ops** (they emit nothing); the actual pen up/down rides the next `PU` / `PD` coordinate command. The modal bool tracker still produces the correct `PU` (travel) → `PD` (work) sequence.

⚠ GRBL laser mode

Enable `$32=1` so the beam only fires during motion. Set power `S` on the `M3` / `M4` that begins a working run (it takes effect on the following move); never leave the laser on across a `G0` travel. `G0` travels use the controller's rapid rate – do not attach a working `F` to them (feed applies only to `G1`).

GRBL's `S` range is `0 .. $30`, where `$30` is the controller's *configured max-spindle-speed* setting (the value corresponding to 100% power – commonly `1000`). This is **not** a fixed ceiling of 30. The formula from percent power to `S` is: `S = round(power% / 100 × $30)`.

GRBL engage mode: use `M3` (constant power) for vector working moves, and `M4` (dynamic power, beam scales with actual speed) for raster (see Chapter 9 §5).

This abstraction holds because all these machines share the same engage/travel/work/disengage model (see Chapter 0 §1).

3. Modal state tracking (never emit redundant commands)

Most controllers are **modal**: a setting persists until it is changed. Track it and emit a command only when something actually changes:

- **Engage state.** Keep an `engaged` bool. Only emit `engage` when going disengaged→engaged, only emit `disengage` when going engaged→disengaged. Redundant toggles waste time and can dwell the tool.
- **Power (S).** Track `lastPower`; omit the `S` word on a working move that keeps the same power. (Essential for raster, Chapter 9, where power changes per scanline.)
- **Feed (F).** Emit once per group (it is modal); re-emit when a group's feed differs from the previous group's.
- **Coordinates.** Some dialects allow omitting an unchanged axis; usually not worth the added complexity – just emit both X and Y.

i Why this matters, beyond aesthetics

Shorter lines pack more moves into the controller's receive buffer, giving the planner deeper look-ahead and smoother motion on dense paths (see Chapter 10 and Chapter 11). Trimming coordinate precision helps too (§4).

✗ Before - re-emit every command

```
M3 S500
G1 X10 Y10 F800
G1 X20 Y15 F800
G1 X30 Y10 F800
M3 S500
G1 X40 Y20 F800
```

Strikethrough = redundant word/command
(F800 and S500 repeat
while state has not changed)

✓ After - emit only on change

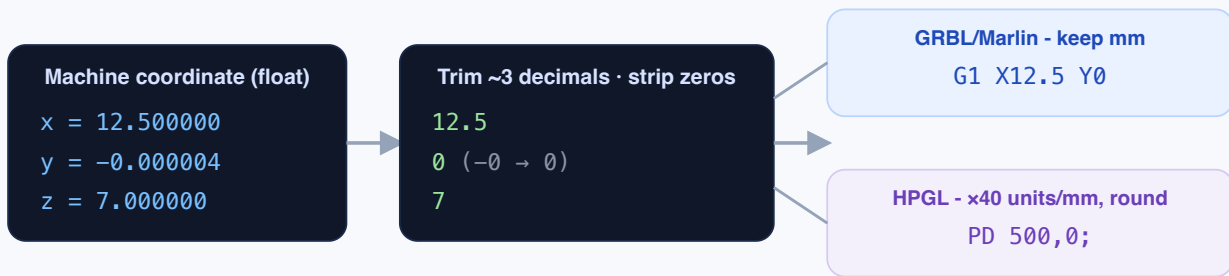
```
G1 F800
M3 S500
G1 X10 Y10
G1 X20 Y15
G1 X30 Y10
G1 X40 Y20
```

F and S emitted only once -
when the value actually changes

The same motion, but the right column is noticeably shorter thanks to tracking `engaged`, `lastPower` and `feed`, then emitting a command only on a genuine change.

4. Coordinate formatting

- **Precision:** emit at most ~3 decimals (micrometre scale) and **strip trailing zeros** (`12.500` → `12.5`, `7.000` → `7`). Parsers read the short form identically, and shorter lines stream better. Normalise `-0` → `0` .
- **Unit scaling:** apply the machine's unit. GRBL/Marlin use mm directly; basic HPGL uses **plotter units** = `1016/25.4 ≈ 40 units/mm` (1 unit = 1/1016 inch) and integer coordinates, so round after scaling.
- **Do this once, at the end.** `format()` applies the world→machine transform (§5) and the unit scale to each point, exactly at emission time - never earlier.



`format(point)` does both jobs - trim precision and scale units - exactly once, at emission

Example: `12.500000` → `12.5` ; `-0.000004` → `0` ; for HPGL, `12.5mm` multiplied by `40 units/mm` then rounded to the integer `500` .

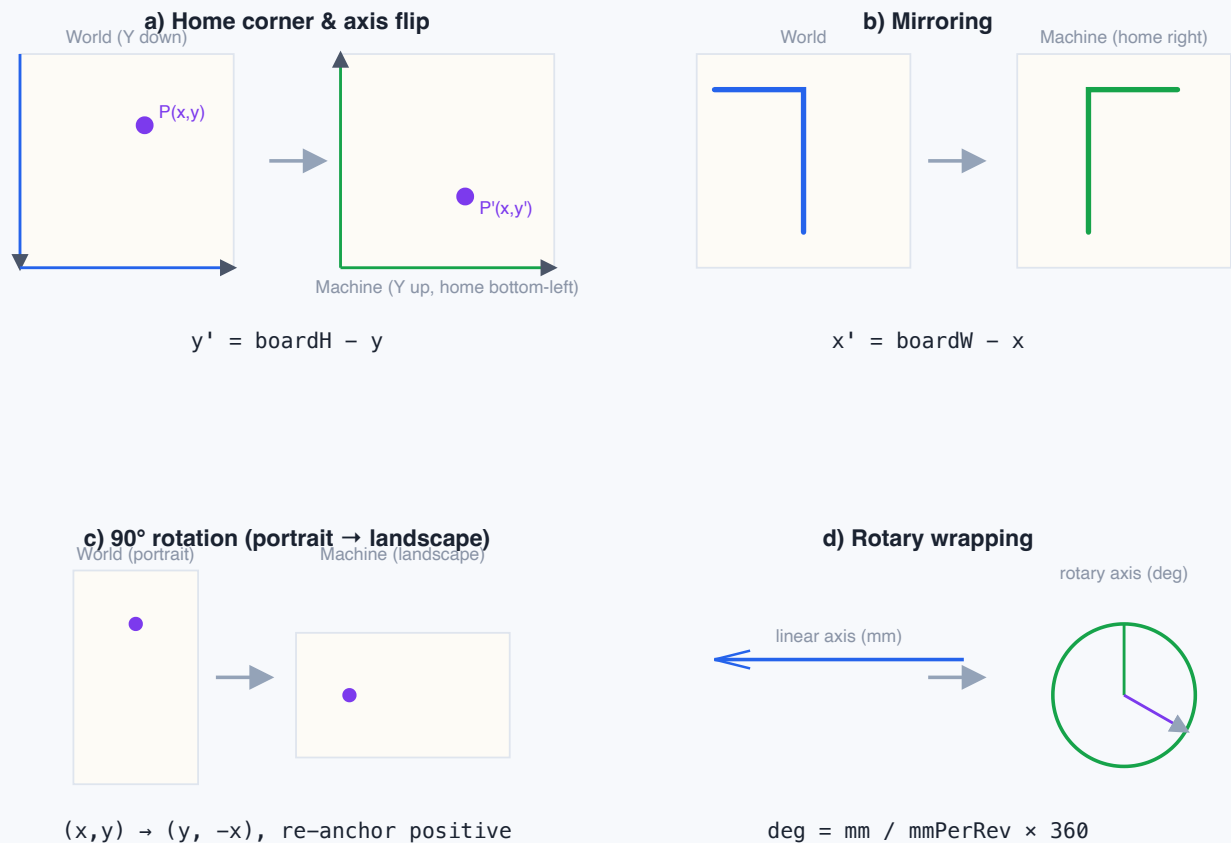
5. World → machine coordinate transform

Design space (world) and machine space rarely coincide. Compose a small affine transform, applied inside `format()` :

- **Home corner / axis direction.** Translate + flip so the origin is the machine's home and the axes point the way the hardware expects. (Example: design +Y points down, machine +Y points up ⇒ `y' = boardH - y` .)
- **Mirroring.** Some devices home at one side with an inherent reflection (e.g. a cutter homing at the right with +X running left). **Pre-mirror** in the emitter to cancel it out (`x' = boardW - x`) so the output isn't reversed. A separate, user-facing "mirror" toggle (for iron-on/HTV) is applied *before* this step, affecting only the emitted text - not the preview.
- **Rotation.** To fit portrait artwork onto a landscape bed, or vice versa, rotate the whole job (e.g. 90° clockwise: `(x,y) → (y, -x)` , then re-anchor to the positive quadrant).
- **Rotary wrapping.** If one axis is a rotary attachment, convert that axis' linear length (mm) into **degrees**: `deg = mm / mmPerRev × 360` , and emit it on the rotary axis letter (e.g. replace `Y` with `A`), leaving the remaining linear axis unchanged.

☆ Keep home moves in raw machine coordinates

Moves that return home must stay in **raw machine coordinates** (not user-mirrored) so that "home" is always the device's true origin. Preview parity: keep the simulator in **world** space (readable to the user); only the emitted text carries these transforms (see Chapter 1 §8-9).



Four common transforms when moving world coordinates into machine space: flipping per home corner, mirroring to cancel hardware reflection, rotating to fit the bed, and wrapping a rotary axis into degrees.

6. Preamble, postamble & comments

- **Preamble:** set units + absolute positioning mode, initialise the tool (force/speed/power/pen select), and move to home.
- **Postamble:** disengage the tool, return home, deselect/end.

- **Comments/annotations:** dialects with a comment syntax (G-code `;`) can carry human-readable notes (layer names, settings, coarsening warnings). Dialects **without** comments (basic HPGL) must be sent only **bare commands** - so keep annotations as a separate, clearly-marked list, and **strip** them before streaming to the device (e.g. drop any line starting with `;` or any blank line). Never send a comment to a machine that can't parse one.
-

7. Passes & groups

- **Passes:** repeat a group's *already-ordered* chains N times for thick material or a bolder mark. Order once, repeat the result.
 - **Groups (layers/tools):** emit as contiguous blocks, each carrying its own power/feed/force/interval. Order within a group; keep position continuous across groups (see Chapter 7 §5).
-

8. Checklist (when an AI generates an emitter)

- Implement the `Emitter` interface; keep the driver loop dialect-free.
- Track engage/power/feed modal state; emit only on change.
- Format coordinates: trim precision + strip trailing zeros, scale to device units.
- Apply the world→machine affine (home/mirror/rotate/rotary) only inside `format()`; keep home moves in raw machine coordinates.
- Emit preamble + postamble; strip comments for dialects that don't support them.
- Repeat ordered chains for passes; emit groups contiguously with their settings.
- Drive the simulator from the same ordered chain list, in world space.

Raster Engraving

A bitmap image is **not** a chain source. You cannot meaningfully "outline" or "fill" a photograph with polylines; instead the tool sweeps across the image row by row and **modulates power** per pixel (dark = strong, light = off). This approach only applies to machines that can vary power *while moving* (laser, variable-pressure engraver) - a pen or blade normally cannot render tone, so raster shapes are simply skipped.

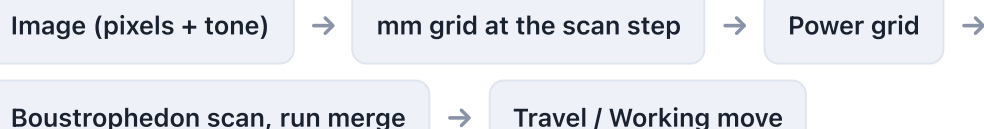
This is the one part of the system that *bypasses* the chain/ordering pipeline learned in earlier chapters; it emits its own moves directly. The chapter still belongs in the book because "convert artwork to machine code" must handle images too, not just vector shapes.

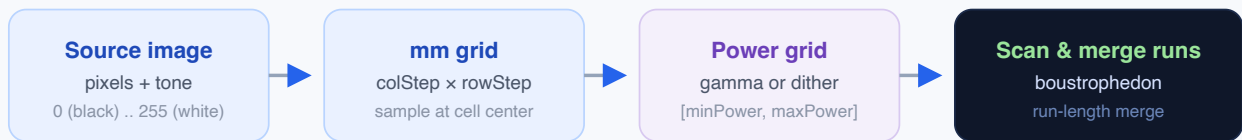
ⓘ Skipped for non-modulating tools

If the selected tool is a pen or blade (unable to vary intensity while moving), the raster stage is **skipped entirely** - it does not try to "fake" tone with stroke density, since that is the job of pattern/hatch fill (Chapter 6), not this raster engraver.

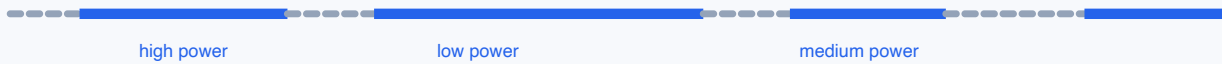
1. The model

The whole path from image to machine code fits into four consecutive transforms: sample the image onto a physical grid, turn the grayscale values into a power grid, then scan that grid boustrophedon-style and merge cells of equal power into a single stroke.





Output (one scan row):



The emitter (Chapter 8) translates travel/working + power into G0/G1+M3/M4 of the target dialect.

— Working move (engaged, power > 0) - - - Travel (through white/clipped area)

Four stages turn an image into machine motion. As with every other source, the algorithm only emits abstract **travel/engaged** moves with a power value; the emitter (Chapter 8) is what maps them to a concrete dialect (GRBL `G0 / G1 + M4` for instance - the dialect letters here are only for illustration).

2. Sampling grid: never oversample

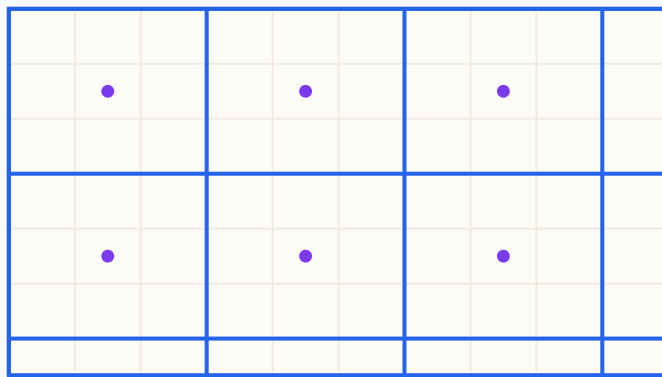
The scan step is chosen from **both** the desired engraving spacing and the image's source resolution:

```
colStep = max(interval, imageWidthMm / imagePixelsWide)
rowStep = max(interval, imageHeightMm / imagePixelsTall)
```

- `interval` - the spacing between engraving lines (material/beam dependent).
- `max(...)` prevents **oversampling** the bitmap: scanning finer than one source pixel is pointless, it only adds motion without adding information.
- **Bounded work:** cap the total cell count (`rows × cols ≤ kMaxCells`). If exceeded, coarsen both steps by a factor of `√(cells / kMaxCells)` and record a human-readable note. This stops a large image at a tiny scan interval from producing millions of moves.

Sample each cell at its **center**, in the image's own local space, so the scan rides the image's own transform (rotation/scale). Cells outside an optional **clip mask** are treated as white (skipped, not engraved).

Source pixels (much finer than colStep/rowStep)



• cell center = sample point (image-local space)

colStep

= max(interval,
imgWmm / pxWide)

If rows×cols > kMaxCells:

coarsen x= √(cells / kMaxCells)

→ note for the user

The scan grid (blue, thick) is always coarser than or equal to the source pixel grid (faint, thin) - never finer. Each cell is sampled at its center (●), and if the total cell count exceeds the **kMaxCells** cap, both steps are coarsened uniformly by the square root of the overshoot ratio (see also Chapter 11 on bounding work).

3. Tone reproduction: two modes

There are two ways to turn each cell's gray value into engraving power: continuous modulation through a gamma curve, or quantisation to 1-bit via dithering.

Mode	Power output	Suited when
Grayscale + gamma	Continuous, within [minPower, maxPower]	Smoothly modulating tool (diode laser), image with a wide gray range
1-bit dithering (Floyd–Steinberg)	Binary, 0 or full power	On/off-only tool, or a crisper "dot" look is preferred over a soft gray field

3.1 Grayscale power modulation (with gamma)

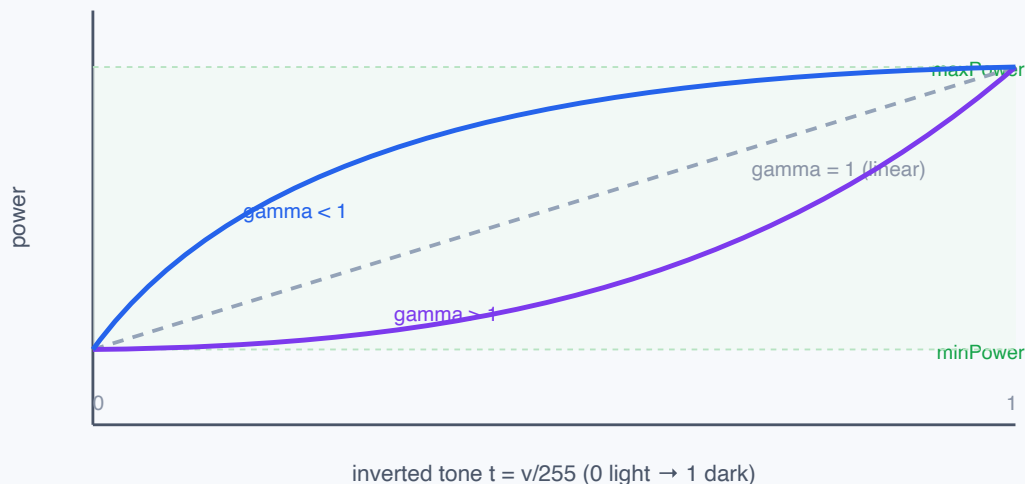
Map cell tone directly to a power level within [minPower, maxPower] :

```

v = 255 - gray          # invert: darker burns hottest
if v <= 0: power = 0     # white → off (hard skip, no engrave)
t = v / 255
if gamma != 1: t = t^gamma # tone curve: correct for the material's non-linear
                           # response
power = minPower + round(t × (maxPower - minPower))

```

- **minPower** - a power floor so light gray areas still leave a mark (below the tool's activation threshold, nothing shows up at all).
- **gamma** - bends the tone curve to match how the material darkens with actual power (non-linear).



The gamma curve bends the tone→power mapping within the `[minPower, maxPower]` band. `gamma > 1` keeps power low for longer (light strokes stay light); `gamma < 1` ramps power up earlier (dark strokes bloom sooner). `gamma = 1` is a pure linear mapping.

□ Terminology

Power/PWM modulation - modulating output power (usually via PWM) to the desired value at each point. **Gamma correction** - the power-law transform t^{gamma} that compensates for the non-linear response between control signal and observed darkness on the material.

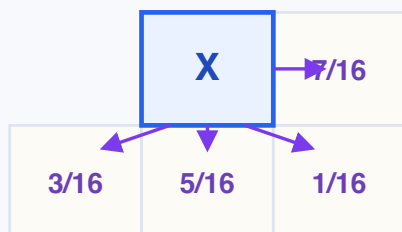
3.2 Dithering (1-bit) - Floyd–Steinberg

For an on/off-only tool, or for a crisper look, convert to on/off dots that *approximate* tone through their density. **Floyd–Steinberg error diffusion**: threshold each cell to black/white, then push the quantisation error to the *not-yet-visited* neighbouring cells using the `7/16, 3/16, 5/16, 1/16` kernel:

```
for each cell (row-major):
    old = buf[cell]; new = (old < 128) ? 0 : 255
    err = old - new; buf[cell] = new
    buf[right]      += err × 7/16
    buf[below-left] += err × 3/16
    buf[below]      += err × 5/16
    buf[below-right] += err × 1/16
```

Dithered cells are already 0/255, so they engrave at **full power** (no gamma applied). Other diffusion kernels (Atkinson, Jarvis) or ordered/Bayer dithering are alternative choices.

cell X already quantised; error $err = old - new$



right, below-left, below, below-right - all "not yet visited" cells

Visit order: row-major, left→right, top→bottom

start →

The quantisation error at cell X is spread to four neighbouring cells by fixed weights, always toward *not-yet-scanned* cells - so the error never feeds back into an already-processed cell.

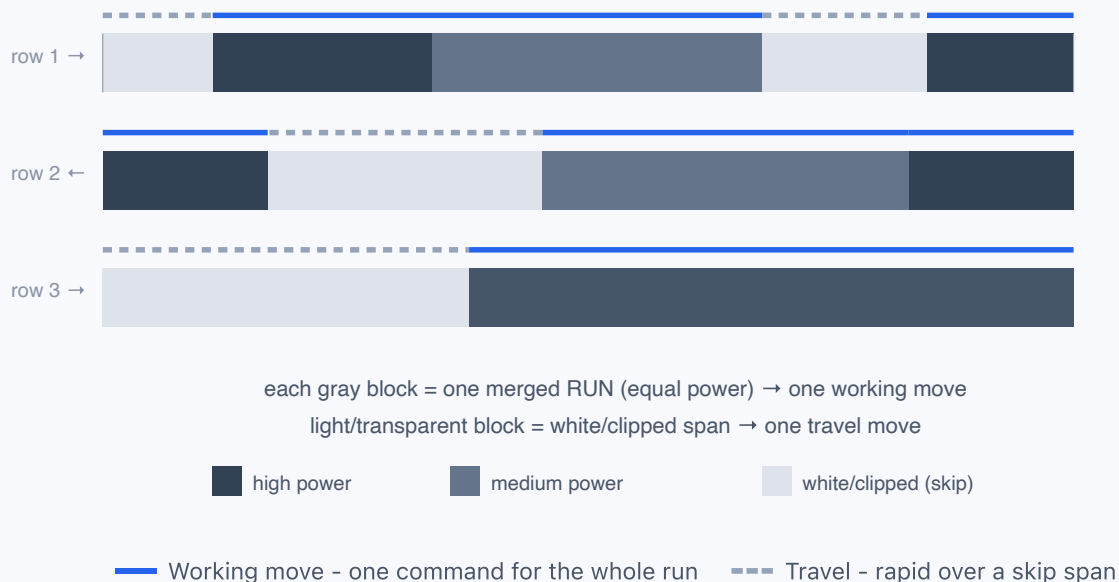
□ Terminology

Error diffusion - the technique of spreading quantisation error to nearby unprocessed samples so the local average tone is preserved. **Halftoning** - reproducing continuous tone through the density of binary dots, a concept originating in printing.

4. Scanning: boustrophedon and run merge

```
leftToRight = true
for each row r:
    compute power for every cell in the row
    find firstBurn..lastBurn (# skip a fully-white row, still flip direction)
    walk that span in the current direction:
        merge a maximal RUN of cells with equal power
        power > 0 → engaged working move to the run's end, at that power
        power = 0 → travel move over the gap (white/clipped cells)
    leftToRight = !leftToRight      # flip direction each row
```

- **Boustrophedon** - "ox-plough" back-and-forth rows, avoiding a return trip to the margin on every scan line.
- **Run-length merge** - collapses adjacent cells of equal power into *one* move, dramatically reducing the line count. Combined with modal power (Chapter 8 §3), a smooth gradient becomes a handful of moves per run instead of one per pixel.
- Gaps (white or clipped) are traversed while **disengaged** (rapid), so masked regions are skipped rather than mistakenly engraved.



☆ Why run merge matters

Without merging, a 1000×1000-pixel image could produce a million individual moves. Run-length merge turns a smooth gradient into a small number of constant-power segments - the move count scales with the *number of power changes* per row, not with the pixel count.

5. Dynamic power (M4) vs constant power (M3)

For raster, prefer the controller's **dynamic power mode** (GRBL **M4**) over constant (**M3**): it scales beam power with the *actual feed rate*, keeping engraving density even through the acceleration/deceleration segments at each row's ends. Vector shapes typically use constant power (**M3**).

M4 - dynamic power

Power scales with the instantaneous feed rate. Used for **raster**: the slower accel/decel segments at the start/end of a scan row still yield even engraving density, without "over-burning" at the ends.

M3 - constant power

Power stays fixed regardless of speed. Used for **vector** (chains): speed along a cut/draw path is usually controlled separately, so power doesn't need to compensate for feed rate.

i Track modal state

Just like power and feed in Chapter 8, the M3/M4 mode is **modal state**: emit the mode-switch command only once when transitioning from a vector segment to a raster segment (or back), not on every scan row.

6. Integration with the vector pipeline

Raster bypasses chains and ordering, but it still lives **inside the same job** - don't let this bypass drop it out of the shared machinery:

Coordinate transform

Raster points go through the *same* world→machine `format()` transform as chains (Chapter 8 §5): sample in the image's local space, map local→world, then let the emitter apply the machine affine (home/mirror/rotate/rotary). Skipping this step lands the engraving in the wrong place or orientation.

Per-layer sequencing

Within a layer, emit the ordered vector chains **first**, then that layer's raster shapes (Chapter 12 §7); carry the running position so the first raster row starts near where the vectors just ended.

Passes

Raster participates in the layer's pass count just like chains: re-emit the whole raster scan once per pass.

Settings

Raster uses the layer's power as the *max/on* level and the layer's feed rate; min-power/gamma/dither are the layer's own image controls (§3).

⚠ Common mistake

Because raster "emits its own code," it's easy to accidentally miss the machine transform (home/mirror/rotary) that vector chains always pass through in Chapter 8. Reuse a single `format()` function for both paths - don't write a separate copy for raster.

7. Not animated in preview

Raster produces an enormous number of moves; don't feed them into the toolpath simulator. Instead, show raster as a static image overlay on the preview. The vector simulator then only animates chains, keeping the interface responsive (raster moves are emitted with no simulator index assigned).

✓ General principle

The preview needs to be honest about output *shape*, not necessarily simulate *every single motion*. With tens of thousands of raster moves, a static overlay image conveys the right information at nearly zero cost.

8. Checklist (AI generating a raster engraver)

- Skip raster entirely for non-modulating tools (pen/blade).
- Scan step = $\max(\text{interval}, \text{one source pixel})$; cap total cell count and coarsen if needed (with a note).
- Sample cell centers in the image's local space; honour the clip mask.
- Tone: grayscale+gamma into `[minPower, maxPower]`, OR Floyd–Steinberg dither to 1-bit.
- Scan boustrophedon; merge equal-power runs; working move over engraved spans, travel over gaps (GRBL `G1` / `G0`).
- Use dynamic power (M4) for even tone; track modal power/feed.
- Keep raster out of the animated simulator.

Curve Fidelity & Continuity

The single most visible quality difference between a good generator and a bad one lies in how it handles **curves**. Quality output must satisfy three things at once: curves read as curves (no faceting), joins read as joins (no breaks), and the result stays faithful to the design within a controlled tolerance. This chapter collects every technique that protects those three properties, from curve flattening through chain assembly to the final emitted motion.

Smooth

Curves read as curves, not jagged polygons. No visible flat facets, no aliasing ("*không răng cưa*").

Continuous

No gaps, no accidental breaks between segments that should be one flowing stroke ("*không đứt đoạn*").

Faithful

The toolpath tracks the design within a controlled tolerance - no missing tips ("*mất nét*"), no bloating or distortion.

Applies to every chain source

The principles here apply to *all* chain-generating sources: fill, outline, text, pattern. Wherever a chain comes from, all three properties above must be protected uniformly.

1. Where jaggedness and breaks come from

Before fixing anything, classify the symptom and trace it back to its root cause. Nearly every curve-quality defect falls into one of seven groups below, each with a specific fix in the sections that follow.

Symptom	Root cause	Fixed in
Faceted curves (visible flat segments)	Flattening tolerance too coarse for the actual scale	§2
Staircase aliasing on diagonal lines	Skeleton/raster steps not smoothed; or integer rounding	§4, §7
Wobble along a stroke that should be smooth	± 1 -sample noise (raster skeleton) not low-passed	§4
Gap between two segments that should join	Two endpoints differ by $> \epsilon$ and are never welded	§3.1
One fill that should be a single stroke splits into many	A valid connector rejected by an overly strict inside/outside test	§3.4
Loop not closed	First/last vertex differ by $> \epsilon$; closing vertex dropped	§3.2
Corner rounded off	Over-smoothing, or the tool's drag-offset trailing behaviour	§4

Faceting

Staircase

Wobble

Gap

Loop not closed

Corner rounded off

Six common defects: the red stroke is the faulty output, the dashed grey stroke is the shape that should have resulted. Memorising their silhouettes speeds up diagnosis when looking at real output.

2. Flattening quality (the anti-facet foundation)

A curve turns into line segments *exactly once*, at the flattening step. That is where smoothness is won or lost - poor flattening cannot be rescued by anything downstream.

2.1 Control tolerance in world units

A curve flattened with a tolerance fixed in *path units* facets badly once viewed at mm scale. Make the effective tolerance a real-world distance sitting below the tool's mark width. This is the **scale-up trick** (see Ch. 2 §1): multiply the path by a factor k (e.g. $k \approx 25$), flatten, then scale back down. The result is an effective tolerance $\approx \text{libraryTolerance} / k \approx 0.02 \text{ mm}$. At that size, facets are no longer physically resolvable, so the curve looks perfectly smooth.

□ flatness test

A Bézier span is considered "flat enough" once the deviation of its control points from the chord joining its two endpoints falls below tolerance. If not, subdivide the span (de Casteljau) and re-test each half.

2.2 Prefer curvature-adaptive subdivision

The best flatteners place **more points where curvature is high** and fewer on near-straight spans. This is **curvature-adaptive subdivision**: recursively subdividing a Bézier span (recursive de Casteljau subdivision) until the control-point deviation falls below tolerance. This approach gives a smooth curve *and* compact output at the same time. A uniform-step flattener wastes points on straight stretches while still being able to facet sharp corners - adaptive subdivision is strictly better.

Coarse tolerance → faceted



4 points - facets clearly visible

Fine tolerance → smooth



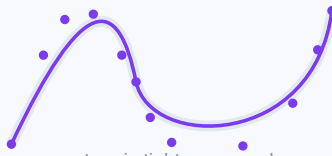
many points - facets invisible

Uniform-step



points spread evenly - wasted on straight spans

Adaptive



Top: the same curve - a coarse tolerance leaves facets visible (red), a fine tolerance is smooth (green).
Bottom: uniform stepping spreads points evenly even along straight spans; **curvature-adaptive subdivision** concentrates points where curvature is sharp and saves points where the curve is nearly straight, giving the same smoothness with fewer points.

2.3 The tolerance trade-off

Two directions of trade-off

SMALLER tolerance → smoother curve, MORE points → bigger output, slower stream

LARGER tolerance → fewer points → faceting

Pick the tolerance as "just below the tool's mark width": $\approx 0.02 \text{ mm}$ as the fine default (Ch. 2 §1), and $\approx 0.05 \text{ mm}$ as a coarser tier, acceptable only for boundaries used in **non-output decisions** (metrics, tile bounds, inside/outside tests).

⚠ The coarse 0.05 mm tier is never used for an emitted vertex

Never use the coarse tolerance for a boundary whose intersections become the endpoints of an emitted fill/pattern stroke, or that must align with an outline. Doing so reproduces exactly the misaligned seam this section warns against. Both figures are estimates tied to tool resolution, not hard constants. Do not shrink the tolerance arbitrarily either: beyond the tool's resolution you only add streaming load (Ch. 11) with zero visible gain.

2.4 Keep a single source of truth

Flatten every curve through *one* routine with *one* tolerance policy (a fine tolerance for anything the eye follows). If different features flatten at different tolerances, a fill line and the outline it meets won't align, producing visible seams.

☆ One routine, one policy

The "single source of truth" principle defends against both failures at once: uneven faceting *and* mismatched seams between features. Don't let each module pick its own tolerance.

3. Continuity: no gaps, no accidental breaks

"Continuity" has two levels, borrowed from CAD's **geometric continuity** classes (written G^0/G^1 here):

⚠ G^0/G^1 is NOT the G-code commands G0/G1

Don't confuse the two. G^0/G^1 here are *geometric continuity classes* (positional / tangential continuity). $G0 / G1$ in Ch. 8 are *G-code motion commands* (rapid move / linear interpolation). They are entirely unrelated - the matching characters are pure coincidence.

G^0 (positional)

Two consecutive segments *share exactly one endpoint* - no gap. But the direction of travel through that shared point can change abruptly (still a visible kink).

G^1 (tangential)

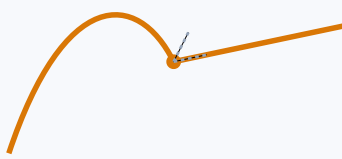
Beyond sharing the point, the two segments also *share a direction* at the join - no visible kink. The stroke flows smoothly through the join.

Gap (fails G^0)



two endpoints offset from each other

G^0 : shared point, kinked direction



no gap, but a visible corner

G^1 : shared point & direction



stroke flows smoothly through the join

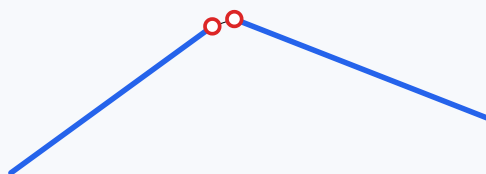
Three situations at a join. Left: the two endpoints were never welded, leaving a **gap** (fails even G^0). Middle: G^0 - they share a point so there's no gap, but the tangent changes abruptly, showing a corner. Right: G^1 - they share both point and tangent (dashed grey line), and the stroke flows smoothly through the join.

3.1 Weld coincident endpoints (kill gaps)

Whenever two segments/chains are supposed to meet, force their shared point to be **identical**, not merely close. Snap endpoints within an epsilon to a common value (this is **endpoint welding / vertex snapping**). Never rely on two independently computed points being bit-equal. A sub-tolerance gap turns into a visible pen-up + travel + pen-down move - the classic "*đứt đoạn*" (break) bug.

Before: gap under ϵ

stray up+travel+down



two points offset $> 0 \rightarrow$ machine lifts the pen

After: welded to one point



one shared point $\rightarrow$ one continuous stroke

— Working move - - - Travel (pen up) — Welded

A small sub-tolerance gap (left) forces the machine to lift the pen, travel a tiny distance, and re-engage: the path looks broken and wastes time. After welding (right), the two segments share exactly one vertex and become one continuous stroke.

3.2 Close loops explicitly

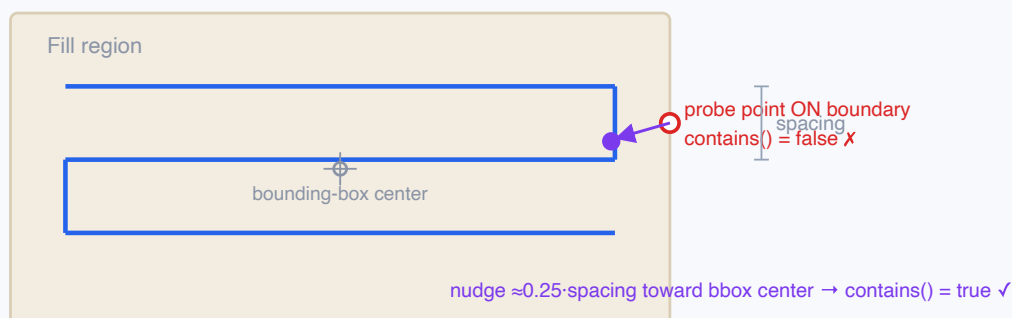
A closed subpath must have `first == last` within ϵ . If flattening or an intermediate operation left them slightly apart, re-append the first vertex to close the loop. Test closure with a squared-distance epsilon over at least 3-4 points (Ch. 2 §2). An unclosed loop leaves a small gap right at the start/end join - exactly the defect described in §3.1.

3.3 Don't fragment contiguous runs

When clipping fill/pattern lines to a region (Ch. 2 §6), **merge** adjacent inside sub-segments into one chain rather than emitting each tiny piece separately. Only a genuine exit-and-re-enter should ever break a chain. Emitting per-piece produces hundreds of micro-chains and a stuttering, lift-heavy path.

3.4 The boundary-probe nudge (a subtle break bug)

A connector that runs *along* a region's edge (e.g. a boustrophedon turn) has its midpoint sitting *exactly on* the boundary, where a strict `contains()` returns false and the connector gets wrongly rejected - splitting one smooth fill into many separate strokes. **Fix:** before the inside/outside test, nudge the probe point a hair ($\approx 0.25 \cdot \text{spacing}$) toward the region's **bounding-box center** (a cheap approximate centroid). This preserves valid edge connectors and keeps the fill continuous.



The midpoint of a connector running along the edge sits *exactly on* the boundary, where `contains()` returns false and wrongly rejects the connector. Nudging it by $\approx 0.25 \cdot \text{spacing}$ toward the bounding-box center moves the probe point solidly inside the region, the test returns true, and the fill stays continuous.

⚠ **Caveat: concave or holed regions**

The bounding-box center is only a valid interior reference for convex-ish regions. On concave or holed regions it can lie outside the material; in that case the nudge merely biases the test toward *rejecting* the connector - which is *safe*, since it never validates a move that crosses a gap. See Ch. 3 §3.3 for the local-test fallback.

3.5 Join fragments into flowing strokes

Skeleton tracing and per-segment extraction naturally produce many fragments. Rejoin them at shared endpoints following the **straightest continuation** rule (max direction dot-product, turn $< 90^\circ$), so a scribble of pieces becomes a handful of flowing strokes (Ch. 5 §3.5). The result is G¹-continuous runs with far fewer pen lifts.

4. Smoothing generated polylines (kill wobble & staircase)

Some chain sources (raster skeletons, traced bitmaps) are born jagged. Apply these three steps in **exactly this order**: denoise → simplify → round.

1 · Moving average [1,2,1]/4 → 2 · Douglas–Peucker → 3 · Chaikin

1 · Moving-average low-pass

Moving-average low-pass ([1,2,1]/4 , a few passes) - removes ± 1 -sample staircase wobble. Pin the endpoints of an open path; wrap around a closed loop.

2 · Simplify

Douglas–Peucker at a small tolerance - drops redundant collinear points *without* changing the shape (fewer points = faster stream, no quality loss).

3 · Round corners

Chaikin corner-cutting (1-2 iterations) - rounds remaining corners into a smooth curve.



Mandatory order: **smooth** (remove staircase noise) → **simplify** (drop redundant points) → **round** (beautify).
Simplifying *before* smoothing locks the staircase in place; rounding *before* simplifying wastes work.

⚠ Caution: over-smoothing

Smoothing too aggressively will *shrink* the shape and round off intended sharp corners. Keep tolerances small and iteration counts low. When a shape has *intended* sharp corners, a corner-preserving smoother (angle-thresholded) beats blanket Chaikin applied everywhere.

5. Consistent point density

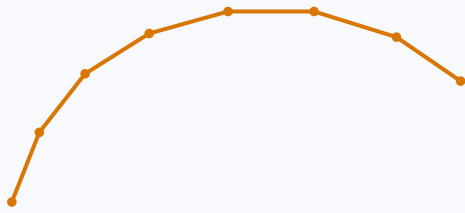
Very uneven segment lengths make a controller's motion planner surge and stall (one long move followed by a burst of tiny ones), which *looks* rough even though the geometry itself is smooth. Two mitigations:

- Adaptive flattening already spaces points by curvature - which is exactly what's needed.
- Optionally **resample** a chain to a maximum segment length on long straight runs, so the feed rate stays even. Balance this against output size.

6. Arc fitting (optional, advanced)

Instead of many short line segments, fit **circular arcs** to smooth spans and emit native arc commands (**G-code G2/G3**; HPGL **AA/AR/CI**).

Many line segments



dozens of vertices, still faceting risk

One G2/G3 arc



one command, exactly curved, tiny output

The controller interpolates a *true* arc: no faceting regardless of feed rate, and one arc replaces dozens of segments, so output is much smaller and look-ahead runs deeper (Ch. 11).

Cost: arc fitting (e.g. **biarc fitting** onto Bézier spans) is more complex, and not every controller supports arcs well. Use line flattening as the robust default, and arc fitting as an optimisation where the target dialect supports it. **Keywords:** *biarc/arc fitting, G2/G3 arc interpolation*.

7. Avoid integer-rounding aliasing at emission

Some dialects use integer device units (e.g. HPGL: 40 units/mm). Rounding each point independently can reintroduce a fine staircase on shallow diagonals. Mitigate with two rules:

- Emit at the **finest device unit** available (40 units/mm $\approx$ 0.025 mm steps, below tool resolution and therefore acceptable).
- Keep all geometry in floating-point mm and **round only at the final `format()` step** (Ch. 8 §4) - never round mid-pipeline and keep computing on the rounded values (errors compound into visible drift/gaps).

✓ Golden rule: compute in float, round only at emission

Every computation in the pipeline stays in floating-point mm. Rounding to device units is the *last step*, done *exactly once*, right at `format()`. This is the surest way to keep rounding error from accumulating into a distorted shape.

8. Checklist (AI protecting curve quality)

☆ Quality checklist

- Flatten with a world-space tolerance ≈ 0.02 mm (scale-up trick or adaptive subdivision); one routine, one policy.
- Prefer curvature-adaptive subdivision over uniform stepping.
- Weld coincident endpoints; close loops explicitly (ϵ test).
- Merge contiguous clipped runs; don't emit per-piece micro-chains.
- Nudge boundary-probe points toward the bounding-box center so edge connectors survive.
- Join fragments by straightest continuation (G^1) into flowing strokes.
- Smooth $\rightarrow$ simplify $\rightarrow$ round, with small tolerances, respecting corners.
- Round to device units only at emission; compute in float mm throughout.
- Consider arc fitting where the target dialect supports $G2/G3$.

📘 Related reading

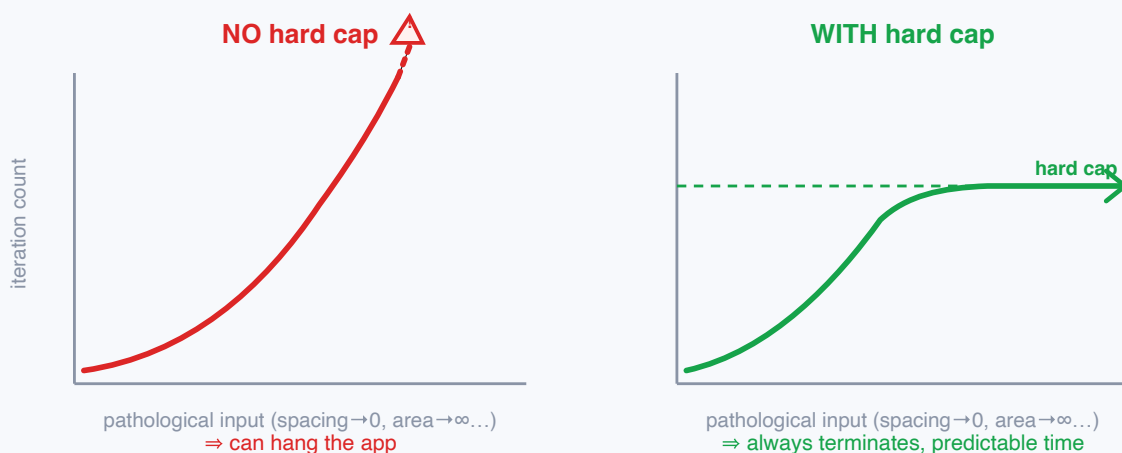
Flattening foundations and ϵ tests in Ch. 2; connectors and the fill engine in Ch. 3; joining skeleton fragments in Ch. 5; coordinate formatting and rounding in Ch. 8; trading point count against stream speed in Ch. 11.

Performance & Complexity

Good output is not enough - generation must also stay **responsive**. A user editing artwork expects toolpath regeneration in interactive time, and heavy jobs (fine fills, large images, thousands of glyphs) must never be allowed to hang the app or exhaust memory. This chapter lists the complexity of every stage, the hard caps that keep them bounded, and the techniques that make them fast.

☆ Golden rule

Every stage must have a known worst-case bound. An unbounded loop that runs on "spacing over area", or an $O(n^2)$ pass with no cutoff, will sooner or later meet an input that freezes the app.



Whatever the stage (flattening, offsetting, clipping, ordering, raster...), a loop with no cap grows without bound when it meets a pathological input. The same loop, given a **hard cap**, always finishes in predictable time - even if the result is slightly coarsened.

1. Complexity budget per stage

The table below lists the asymptotic cost of every stage in the pipeline, which variable drives that cost, and the corresponding cap or mitigation. This is the central lookup table for the whole chapter.

Stage	Cost	Dominant variable	Bound / mitigation
Flatten curves	$O(\text{points})$	tolerance	adaptive subdivision; don't over-refine
Shape metrics	$O(v \log v)$	vertices (hull sort)	one pass; reuse results
Scanline hatch/zigzag	$O(\text{scanlines} \times \text{edges})$	area/spacing $\times$ complexity	cheap; spacing floor
Contour/spiral offset (boolean, Method A)	$O(\text{rings} \times \text{offset cost})$	area/spacing	max-ring cap
Contour/spiral offset (distance field, Method B)	$O(\text{cells}) + O(\text{cells} + \text{contourLen})$	grid cells	max-cell cap + coarsen
Polyline clip (general)	$O(\text{seg} \times \text{boundaryEdges})$	both	bbox reject; spatial index
Space-filling curve	$O(\text{cells})$	grid order	max-order cap
Voronoi	$O(\text{sites}^2)$	site count	max-site cap
Skeletonise (text)	$O(\text{pixels} \times \text{iters})$	image size	image-size cap
Ordering (greedy nearest-neighbour)	$O(n^2)$	chain count	cutoff $\rightarrow O(n)$ fallback
Raster scan	$O(\text{rows} \times \text{cols})$	image $\times$ interval	max-cell cap + coarsen

Stage	Cost	Dominant variable	Bound / mitigation
Emission	$O(\text{moves})$	move count	trim precision; stream in chunks

2. Flattening: fidelity vs. point count

Point count is set by tolerance (see Chapter 10 §2). More points ripple downstream through every later stage (clipping, ordering, emission, streaming). Therefore:

- Use **adaptive** subdivision so straight spans cost almost no points, and only tight curves receive extra ones.
- Never set tolerance below the tool's own resolution – extra points buy no quality but cost throughout the whole pipeline.
- Flatten **once** per shape and cache the result if the shape is unchanged.

3. Fills

Scanline (hatch / zigzag)

For every scanline we test crossings against every boundary edge → $O(\text{scanlines} \times \text{edges})$.

Scanline count equals $\text{height} / \text{spacing}$, so a very small spacing on a tall shape explodes.

Guards:

- A **spacing floor** (never below a sane minimum) caps the scanline count.
- The crossing test is trivial arithmetic; this is the *cheapest* fill.
- Optional: bucket edges by y-range so each scanline only tests edges that can cross it (turns the edge factor into a near-constant).

Offset (contour / spiral)

There are two different cost profiles, one per offset method (see Chapter 2 §7):

Method A - boolean erosion

Each inward offset pass costs one outline stroke plus one boolean subtract - the most expensive fill. Ring count = $\text{meanWidth} / \text{spacing}$.

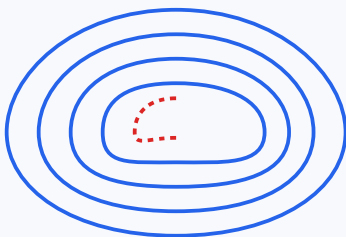
Method B - distance field + iso-contours

One rasterise pass plus one distance transform ($O(\text{cells})$), then a single marching-squares pass ($O(\text{cells} + \text{total contour length})$) yields **all** rings at once.

Method A needs a **max-ring cap** (e.g. a few thousand) so that a tiny spacing on a huge area still terminates; and it stops as soon as an inset pass returns empty.

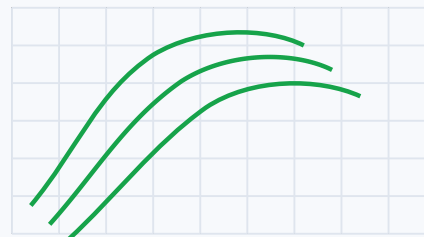
Method B's cost is set by the **grid**, not by the ring count - so it does *not* blow up as spacing shrinks (a smaller spacing just reads more iso-levels off the same field). It needs a **max-cell cap** (e.g. ~500k for a pure-language pass): if $(w+\text{pad})(h+\text{pad})/h^2 > \text{cap}$, enlarge the grid step h by $\sqrt{(\text{area}/\text{cap})}$ and record it; the default grid step is $h \approx \text{spacing}/3$. A chamfer transform is two linear scan passes; marching squares emits crossings in a single pass over cells (only boundary cells produce segments) - both run comfortably under a second within the cell cap.

Method A - concentric rings (boolean)

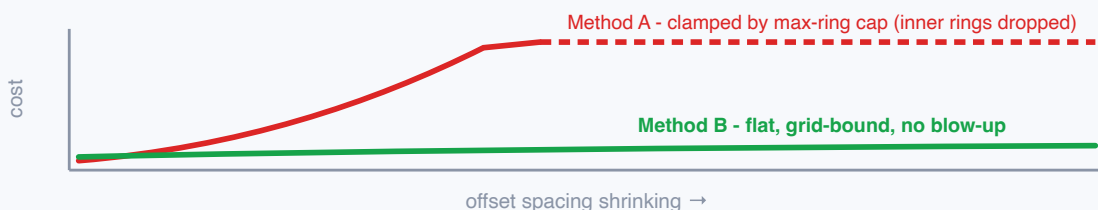


spacing ↓ ⇒ rings ↑↑ (needs max-ring cap)

Method B - grid + iso-contours



every ring is read off one fixed grid at once



Method A pays per **ring**, so shrinking spacing drives cost up steeply, requiring a max-ring cap (trade-off: the innermost rings are lost). Method B pays per **fixed grid size** - a smaller spacing just reads more iso-levels, without inflating the cost.

For both methods: prefer scanline fills for large plain areas; reserve contour/spiral for shapes whose *own shape* calls for them (the auto selector already handles this, see Chapter 3 §4).

Decorative (space-filling curves, Voronoi)

- Hilbert/Peano: cap the curve's **order** (e.g. 2^7 or 3^4 cells/side) so the point count cannot blow up.
- Voronoi: naive cell clipping is **$O(\text{sites}^2)$** ; cap the site count (e.g. ≤ 240). Use a spatial structure (or a real Fortune's algorithm Voronoi) if more cells are needed.

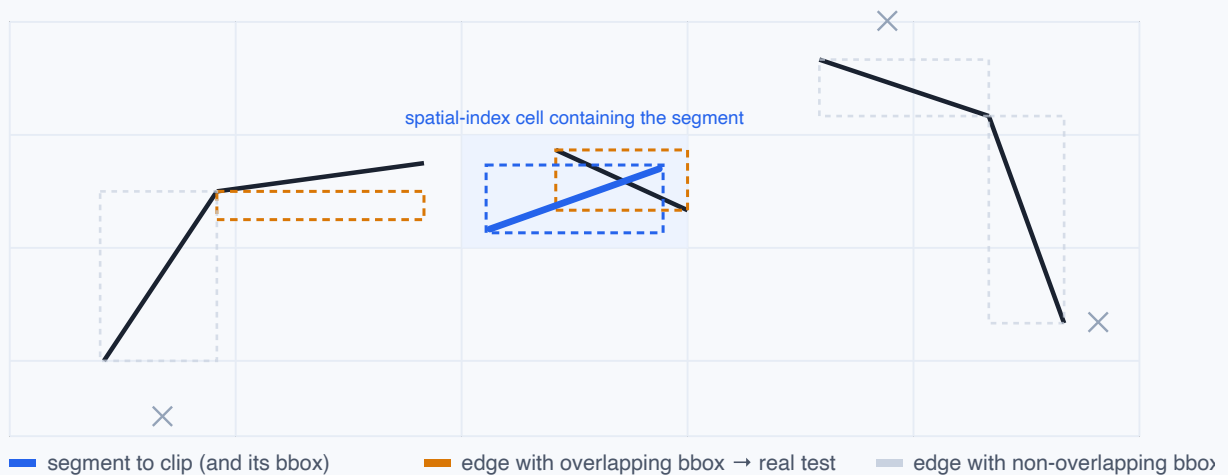
4. Clipping: the hidden quadratic

Clipping a polyline against a general region tests every segment against every boundary edge → **$O(\text{seg} \times \text{boundaryEdges})$** . Pattern tiling multiplies this by the tile count. This is the most common accidental hotspot. Mitigations:

A commonly missed hotspot

A pattern-tiling loop (see Chapter 6) looks harmless but multiplies segment count by tile count *and* by boundary edge count - easily turning a seemingly cheap operation into a quadratic pass that runs for seconds.

- **Bounding-box reject:** skip any (segment, edge) pair whose bounding boxes don't overlap - turns most comparisons into a cheap, early-out test.
- **Spatial index** on boundary edges (uniform grid, quadtree, or an interval tree on y). Query only edges near the segment being tested. Brings cost down to near **$O(\text{seg} \cdot \log)$** .
- **Tile-count guard:** if the estimated tile count is implausibly large, fall back to a plain fill rather than tiling millions of cells.



Bounding-box reject turns most (segment, edge) pairs into an instant cheap rejection; a spatial index (a grid here) narrows the candidate list to only edges near the segment being tested, bringing cost from $O(\text{seg} \times \text{edges})$ down to near $O(\text{seg} \cdot \log)$.

5. Ordering: $O(n^2)$ with a cutoff

Greedy nearest-neighbour (see Chapter 7) is $O(n^2)$. Dense fills produce tens of thousands of chains, so:

- A **cutoff** (e.g. 4000 chains): above it, skip the $O(n^2)$ pass and use the orientation-only $O(n)$ fallback, which trusts the fill's inherent locality (scanline/ring order).
- For large-but-important sets, use a **spatial sort** (grid order or a Hilbert order on chain midpoints) → $O(n \log n)$, giving a much better tour than raw source order.
- Use **squared** distances (no **sqrt**) in the inner loop.

6. Raster: cap and coarsen

Raster is $O(\text{rows} \times \text{cols})$ and can reach millions of cells (see Chapter 9 §2):

- Step $\geq$ one source pixel (never oversample) **and** $\geq$ the interval.
- Max-cell cap:** if $\text{rows} \times \text{cols} > \text{kMaxCells}$, coarsen both steps by $\sqrt{(\text{cells}/\text{cap})}$ and record a note.
- Run-length merging** plus modal power collapsing compress a gradient into far fewer moves.

- Don't feed raster moves into the animated simulator (preview as a static image instead) - animating millions of moves would kill UI responsiveness.

7. Skeletonisation: bound the bitmap

Single-stroke text centerlines (see Chapter 5) rasterise then thin iteratively:

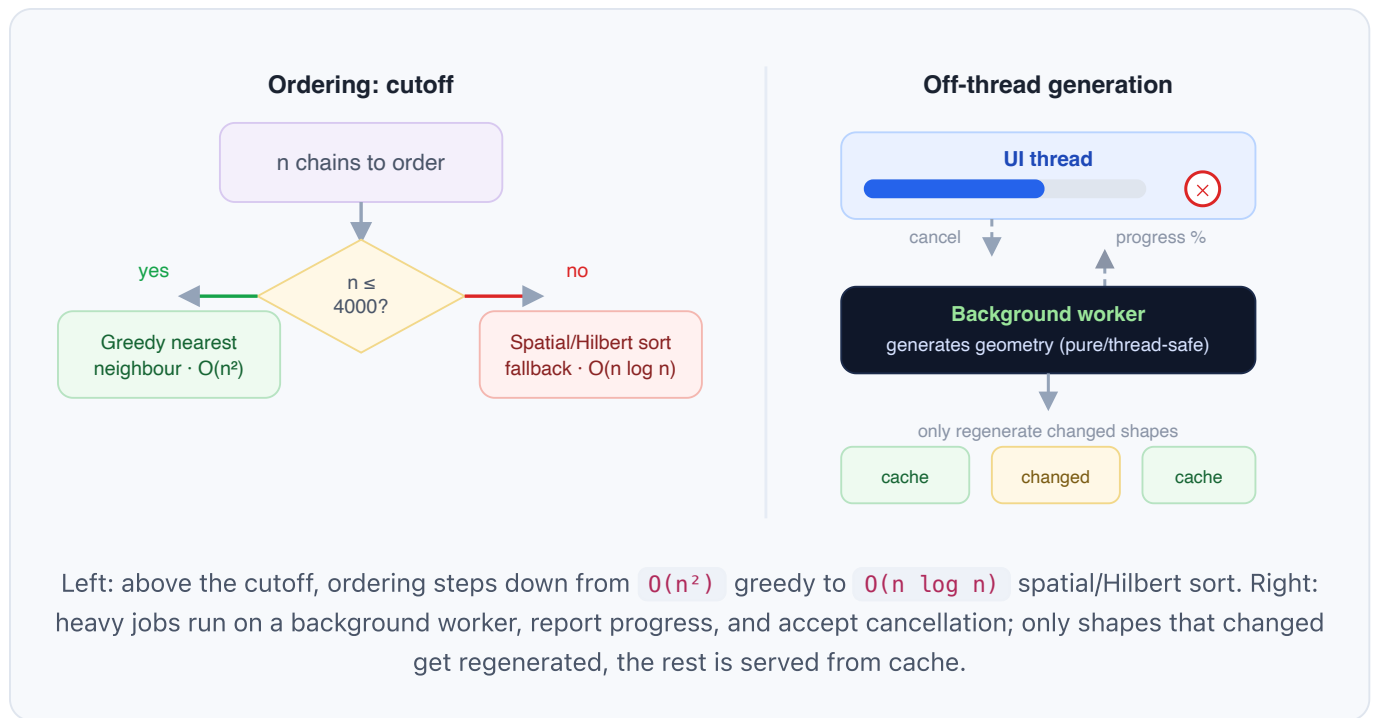
- **Cap resolution and total pixel count** (e.g. max dimension ~2600 px, $\leq$ ~9M px total).
- Thinning is $O(\text{pixels})$ per iteration, with a bounded (guarded) iteration count.
- Skeletonise per glyph, not per whole text block, so every bitmap stays small.

8. Emission & streaming

- **Trim coordinate precision** and strip trailing zeros (see Chapter 8 §4): fewer bytes $\Rightarrow$ deeper controller look-ahead $\Rightarrow$ smoother dense motion, and less string-building work.
- **Modal-state suppression** (see Chapter 8 §3) removes redundant lines - smaller output, faster to build and send.
- Build output with a pre-sized buffer / streaming writer, not repeated string concatenation (avoid $O(n^2)$ string growth).
- For very large jobs, **stream** to the device/file incrementally rather than materialising the whole program in memory.

9. Keeping the UI responsive

- **Report progress** at a coarse cadence (e.g. once per shape, or every N raster rows) so a long job shows a moving progress bar, not a frozen window.
- **Generate off the UI thread** (worker/background) for heavy jobs; post results back afterwards. Keep the geometry pure/thread-safe so this stays simple.
- **Cancellation:** allow a long generation job to be cancelled; check a flag at the same cadence as progress.
- **Incremental regeneration:** only regenerate shapes/layers that changed; cache the rest. Flattening and filling an untouched shape should never be repeated.



10. Determinism - a rule tied to performance

i Controlled randomness

Avoid RNG in geometry (e.g. Voronoi jitter): derive randomness from an index-based **hash** so output is reproducible across runs. Deterministic output makes caching valid and debugging tractable; a nondeterministic generator cannot safely skip "unchanged" work.

11. Checklist (AI keeping generation fast)

- Every iterative construction has a hard cap (rings, order, sites, cells, iterations).
- Adaptive flattening; never refine below tool resolution; cache per shape.
- Clipping uses bbox reject plus a spatial index; guard tile counts.
- Ordering switches to $O(n)$ / spatial sort above a chain-count cutoff; squared distances only.
- Raster caps and coarsens; run-length merges; never fed to the animated simulator.
- Emission trims precision, suppresses modal repeats, streams large jobs.
- Heavy jobs run off the UI thread with progress and cancellation; regenerate incrementally.
- Geometry is deterministic (hash-based, not RNG).

Inputs, Units & Edge Cases

Chapters 2-11 assumed clean, well-scaled geometry. A **complete**, production-grade generator must also handle the messy realities at the edges: artwork in the wrong units, self-intersecting imported paths, degenerate/empty shapes, out-of-bed geometry, unusual strokes, and the lifecycle of a long job that may be cancelled. This chapter consolidates those concerns so every case has a defined, no-crash behaviour.

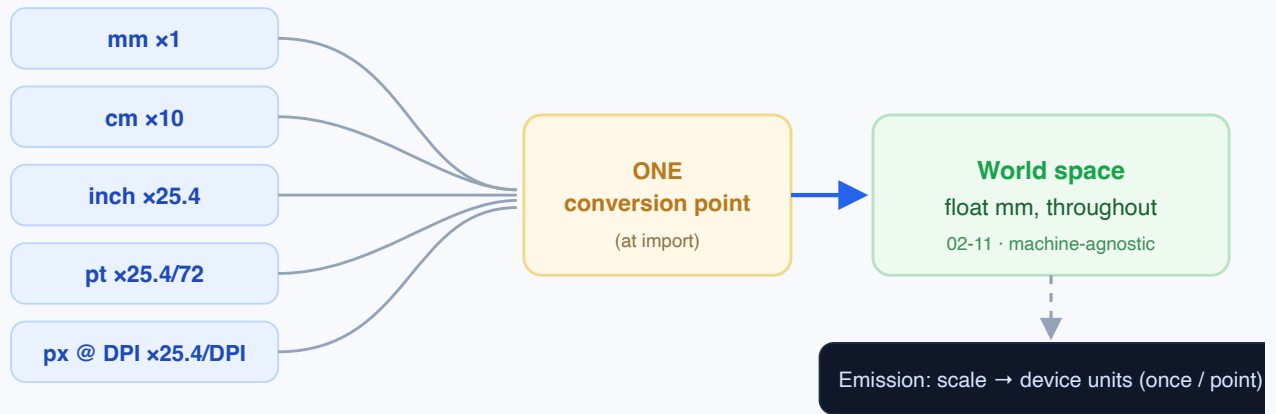
☆ Guiding rule

The generator **must never crash** and **must never silently produce wrong output**. On a bad input it must either normalise it, omit it gracefully, or record a human-readable note - never throw, never hang.

1. Source-unit normalisation → world mm

Artwork arrives from many sources in many units. **Normalise everything to one world unit (mm)** at the earliest possible point, then never think about units again until emission (Ch. 8 §4).

Source unit	Convert to mm by
mm	$\times 1$
cm	$\times 10$
inch	$\times 25.4$
point (pt)	$\times 25.4 / 72$
pixel / user-unit @ DPI	$\times 25.4 / \text{DPI}$



Every source unit converges to **one single conversion point** at import. From there, all geometry lives in world mm; only the emitter scales to device units, and only ever does so once per point.

i DPI ambiguity

Raster images and some vector formats carry their own DPI; if absent, assume a documented default value - commonly **96 DPI** for screen artwork - and *surface that assumption* to the user, rather than silently guessing.

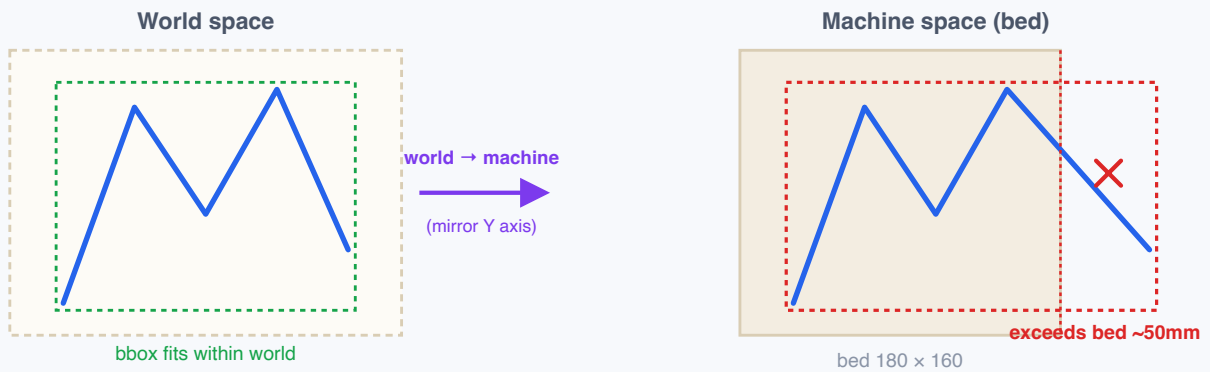
Keep geometry in floating-point mm throughout the pipeline; scale to device units only in the emitter, once, per point (Ch. 10 §7). Multiple conversion points scattered through the pipeline are a top source of scale bugs.

2. Coordinate origin & bed

Origin corner. World space and machine space each have a defined origin corner and axis directions. Make the mapping explicit and configurable (Ch. 1 §8); the emitter applies it last (Ch. 8 §5).

⚠ Check bounds in MACHINE space, not world space

Know the machine's work area. Check bounds in **machine space** - i.e. *after* the world→machine transform (Ch. 8 §5) - not in world space: a job that fits in world space can fall off the bed once mirrored/rotated (or a rotary-wrapped axis changes the meaning of coordinates entirely). In other words: map the artwork's bounding box through the same transform, then compare it to the bed rectangle.



The same shape fits within **world space**. After the world→machine mapping (here, a mirror on the Y axis), the bounding box in **machine space** exceeds the bed. This is why bounds must be checked *after* the transform, not before.

Reject

Reject the job with a clear message ("artwork exceeds bed by N mm"). The safest, most explicit option.

Clamp

Crop the shape to fit the bed. Rarely desirable - it silently distorts the artwork.

Warn and proceed

Let the user decide, recording an annotation.

Pick a policy and make it explicit. Whichever policy is chosen, off-bed working moves must never be emitted silently.

☆ Home always stays in raw machine coordinates

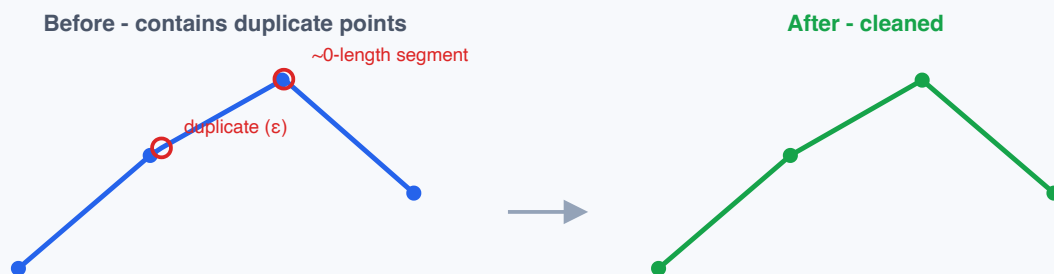
Return-home commands always use raw machine coordinates, never passing through the artwork's mirror/rotation, so "home" is always the true device origin, regardless of how the artwork has been transformed (Ch. 8 §5).

3. Path hygiene (normalise before geometry ops)

Even-odd fill, shoelace area, ray-casting, and offsetting all assume reasonable polylines. Clean the input first:

3.1 Coincident / duplicate points

Drop consecutive duplicate vertices (within ϵ) and zero-length segments. They break tangent/direction math (division by ~ 0) in smoothing, joining, offsetting, and normal computation.



Duplicate vertices and zero-length segments (circled red) are removed before tangent, offset, or normal geometry runs on the chain.

3.2 Self-intersecting (non-simple) paths

Imported artwork (hand-drawn, traced, or malformed SVG) may cross itself. Effects and handling:

- **Fill:** even-odd gives a *predictable, deterministic* result on self-intersecting paths (regions covered an odd number of times are filled), but whether that result is *desired* depends on intent - a self-crossing pentagram or a self-overlapping blob meant to read **solid** needs **non-zero winding** or a **self-union** instead. Pick the fill rule per Ch. 2 §3; the fill itself needs no geometric *repair*, only the correct rule.
- **Signed-area metrics** (Ch. 2 §4) become unreliable on self-intersecting loops (positive and negative sub-areas cancel out). If this metric feeds the auto-fill selector, either (a) compute the metric on the **even-odd rendered region** rather than the raw loop, or (b) detect self-intersection and split the loop into **simple sub-loops** first.
- **Offsetting** (Ch. 2 §7) can produce spurious self-overlaps on non-simple input; the stroke-and-subtract construction is fairly robust but still needs the result validated as non-empty.

□ Keywords

Bentley-Ottmann sweep-line - a sweep-line algorithm for detecting segment intersections. **Polygon self-union** - a boolean OR of a polygon with itself, used to repair the shape and produce a simple boundary.

even-odd



central pentagon hole - covered an even number of times

non-zero winding



entirely solid - covered a nonzero number of times

The same self-intersecting path (a pentagram drawn as one continuous line), two fill rules give two different results. Neither is geometrically "wrong" - pick the rule according to the artwork's intent (Ch. 2 §3).

3.3 Winding & fill rule

Apply the fill rule per Ch. 2 §3: **even-odd** for *nested* holes (winding-independent, deterministic hole detection), but honor the source's **non-zero winding** - or pre-union overlapping same-winding contours - where *overlaps* must read solid. Nesting-depth analysis (Ch. 2 §4) is winding-independent and valid in either case.

4. Degenerate & empty inputs - the no-crash contract

Enumerate every degenerate case and its required behaviour. Each row must be a graceful no-op or a bounded fallback, never a crash:

Input	Required behaviour
Empty document (no visible shapes)	Emit a valid empty job: preamble + return-home + postamble, no working moves.
Shape fully outside the bed	Per the §2 bed policy (reject/warn/clamp); never silently emit off-bed working moves.
Zero-area / collinear "shape"	No fill (area ≈ 0); emit its outline if it has ≥ 2 distinct points, else skip.
Single-point subpath	Not drawable as a line; skip (optionally emit a dot if the tool supports one).
Chain with < 2 points	Skip (a chain needs ≥ 2 points to be a move).
Region thinner than one line spacing	Fill engine keeps the outline as the sole ring / hatch fallback (Ch. 3 §6); never leave empty output for a real area.
Fill spacing ≤ 0	Clamp to a positive floor (Ch. 3 §6).
NaN / infinite coordinate	Reject the shape with a note; never emit NaN to a controller.
Enormous shape at tiny spacing	Caps engage (Ch. 11); coarsen the shape + annotate rather than hang.

This table is the single reference

This is the single reference for "what happens when the input is bad"; each engine implements its own row, while the driver checks the document-level rows (empty document, off-bed).

5. Stroke variants

Cross-reference to Ch. 4 §5b:

- **Dashed / dotted strokes** → many short open chains (walk the boundary by arc length, emit dash segments, skip gap segments).
- **Variable-width strokes** → either expand to a filled region and fill it (Ch. 3), or draw the centerline once if the tool's mark width already matches.

6. Compound paths & grouping affinity

Cross-reference to Ch. 7:

□ Affinity key

A **compound path / glyph** (outer boundary + holes) is multiple subpaths that should often stay **contiguous** in the output. Tag related chains with an **affinity key** and order the whole group as a unit if contiguity matters; otherwise, minimising raw travel is enough.

7. Job lifecycle & cancellation

A single, explicit lifecycle ties together the per-stage behaviour scattered across the previous chapters:

```
START
| normalise units → world mm (§1)
| resolve geometry, path hygiene (§3)
| write preamble (units, absolute, tool init, home) # doc 08
|
| FOR each layer / tool-group (in output order): # doc 07
§5 |   | collect + plan chains (fill/outline/text/pattern) # docs 03-06
|   | order chains (carry running position) # doc 07
|   | emit ordered chains × passes, with group settings # doc 08
|   | [checkpoint] report progress; honour CANCEL # doc 11
§9 |   └ raster shapes (if any), with the same cancel checkpoints # doc 09
|
| write postamble (disengage tool, return home, end) # doc 08
FINALISE
```

⚠ Safe postamble even on cancel

Cancellation is checked at the same cadence as progress reporting (per shape, per N raster rows). On cancel, a **safe postamble** (disengage tool + return home) must still be written so the machine is left in a known state - never aborting mid-working-move.

Progress is reported coarsely so the UI stays responsive (Ch. 11 §9). **Incremental regeneration:** only re-plan shapes/layers that changed; cache the rest (Ch. 11 §9).

8. Checklist (hardening a generator)

- Normalise all source units to world mm at import; a single conversion point.
- Make the origin/bed mapping explicit; define a bed-bounds policy; keep return-home moves in raw machine coordinates.
- Clean paths: drop duplicate points / zero-length segments; apply the correct fill rule per Ch. 2 §3 (even-odd for nesting; non-zero/union for overlaps).
- Handle self-intersection (even-odd for fill; metrics on the rendered region, or split into simple loops).
- Fully implement the degenerate-input table (§4) - every row a graceful no-op or bounded fallback; never crash, never emit NaN.
- Support dashed & variable-width strokes.
- Optional: compound-path affinity in ordering.
- Implement the job lifecycle with cancel checkpoints and a safe postamble on cancel.

✓ Closing the book

From the chain model in Chapter 0, through architecture, geometry, fill algorithms, outlines, single-stroke text, patterns, ordering, machine-code emission, raster, curve fidelity, and performance - this chapter closes the loop by hardening the system against the messy reality of real data. A good generator is not one that has never met a bad input, but one that *already has an answer* for every bad input.